🏗️ The Complete System Design Interview Mastery Guide — HLD + LLD, From Zero to Offer

> **A single, book-length, no-fluff README covering system design from first principles all the way to acing FAANG/MNC/product-company system design interviews — High-Level Design (HLD), Low-Level Design (LLD/OOP design), 20+ fully worked HLD case studies, 12 fully coded LLD case studies, and a massive interview Q&A bank with detailed model answers.**
>
> Pair this with DSA_COMPLETE_INTERVIEW_GUIDE.md (coding rounds), BACKEND_DEVELOPMENT_COMPLETE_GUIDE.md (backend engineering depth) and INTERVIEW_MASTERY_GUIDE.md (behavioral/HR rounds) — this guide is 100% dedicated to **system design**, HLD and LLD, nothing else, in extreme depth.

📖 How To Use This Guide

- **Never designed a system before?** Read Parts 1–3 in order first. They build the mental vocabulary (estimation math, scalability, databases, caching, queues, networking, APIs, microservices, consistent hashing, distributed systems theory, security, observability) that every HLD answer is built from. Skipping this and jumping straight to case studies will make the case studies feel like magic tricks instead of applied logic.
- **Preparing for interviews in the next 2–4 weeks?** Read Part 4 (the interview framework) first, then pick 8–10 HLD case studies from Part 6 that match the company/domain you're interviewing for, and do LLD case studies from Part 7 for backend-heavy roles.
- **Have an interview tomorrow?** Read Part 9 (cheat sheets) and skim Part 8 (Q&A bank) — it's built for fast recall of the things people freeze on.
- **Already comfortable with HLD, want to go deeper on OOP/LLD?** Jump straight to Part 5 (LLD framework/patterns) and Part 7 (coded LLD case studies).

- Every HLD case study follows the **same repeatable structure**: Problem → Requirements (functional/non-functional) → Capacity Estimation → API Design → High-Level Design (data flow, components) → Database Schema → Deep Dive (the 2-3 hardest sub-problems) → Bottlenecks & Scaling → Trade-offs → Follow-up interview questions. Once you've internalized this structure you can build confidence on *any* system, not just the ones listed here — that repeatable structure **is** the actual skill being tested in interviews.
- Every LLD case study follows: Problem → Requirements → Identify Entities/Actors → Class Diagram (in text) → Design Patterns Used → Full Java Code → Extensibility Discussion.

📚 Master Table of Contents

Part 1 — Foundations

1. What Is System Design? HLD vs LLD
2. Functional vs Non-Functional Requirements
3. Back-of-the-Envelope Estimation

Part 2 — Core Building Blocks 4. Scalability 5. Load Balancing 6. Availability & Reliability 7. Consistency, CAP & PACELC 8. Databases Deep Dive 9. Caching 10. Message Queues & Event-Driven Architecture 11. Networking Essentials 12. API Design 13. Microservices vs Monolith 14. Consistent Hashing 15. Distributed Systems Theory 16. Security in System Design 17. Observability 18. Storage & Blob Systems

Part 3 — The Interview Framework 19. How To Solve Any HLD Question in 45 Minutes

Part 4 — LLD Framework 20. OOP Fundamentals & SOLID Principles 21. Design Patterns for Interviews

Part 5 — HLD Case Studies 22. Design a URL Shortener 23. Design Pastebin 24. Design a Rate Limiter 25. Design a Web Crawler 26. Design a Notification System 27. Design a News Feed (Twitter/Facebook) 28. Design a Chat/Messaging App (WhatsApp) 29. Design Video Streaming (YouTube/Netflix) 30. Design Ride Sharing (Uber) 31. Design Ticket Booking (BookMyShow/Ticketmaster) 32. Design E-commerce Checkout (Amazon) 33. Design Search Autocomplete/Typeahead 34. Design a Distributed Cache (Redis) 35. Design Distributed File Storage (Dropbox/Drive) 36. Design a Payment/Wallet System 37. Design a Distributed Unique

ID Generator 38. Design a Proximity Service (Yelp/Nearby) 39. Design Instagram/Photo Sharing 40. Design a Distributed Task Scheduler/Cron 41. Design Ad Click Aggregation/Analytics 42. Design a Leaderboard System 43. Design a Log Aggregation System (Splunk-like)

Part 6 — LLD Case Studies (Full Java Code) 44. LLD: Parking Lot 45. LLD: Elevator System 46. LLD: Splitwise (Expense Sharing) 47. LLD: LRU Cache 48. LLD: Rate Limiter (Code) 49. LLD: Vending Machine 50. LLD: Movie Ticket Booking 51. LLD: Snake and Ladder 52. LLD: Library Management System 53. LLD: Tic-Tac-Toe & Chess 54. LLD: Logging Framework 55. LLD: Food Delivery (Swiggy/Zomato)

Part 7 — Interview Q&A Bank 56. System Design Interview Q&A Bank

Part 8 — Reference 57. Cheat Sheets 58. A–Z Glossary 59. Study Roadmap & Closing Notes

---

Part 1 — Foundations

1. What Is System Design? HLD vs LLD

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy a set of requirements — while explicitly navigating trade-offs like cost, latency, consistency, availability, and complexity. In an interview, it is a **structured conversation**, not a test of memorized diagrams. The interviewer wants to see how you think under ambiguity, how you scope a huge open-ended problem down to something buildable in the time given, and how you justify decisions with numbers and trade-offs rather than buzzwords.

There are two distinct flavors, and interviews test them very differently:

High-Level Design (HLD)

HLD answers: **"What are the major components of the system, how do they talk to each other, and how does the system behave at scale?"** It operates at the level of services, databases, caches, queues, load balancers, and the network calls between them. You are not writing classes or functions — you are drawing boxes and arrows (in text/ASCII in this guide) and reasoning about:

- How many servers/replicas do I need?
- Where does state live, and how is it replicated?
- What happens when a component fails?
- How do I partition data across machines?
- What's the read path vs write path, and which one is hotter?

HLD interviews are common for SDE-2/senior/staff roles, and for any role where the company cares about how you'd architect a feature that needs to survive real traffic (e.g. "design Twitter's news feed", "design a URL shortener that handles 100M requests/day").

Low-Level Design (LLD)

LLD answers: **"Given one component/service, how do you actually design its internal classes, interfaces, and objects so the code is correct, extensible, and maintainable?"** This is where object-oriented design, SOLID principles, and design patterns live. You *are* writing classes, interfaces, enums, and method signatures — sometimes actual working code. Typical prompts: "design a parking lot", "design an elevator system", "design Splitwise".

LLD interviews are common earlier in the pipeline (SDE-1/SDE-2) and at companies that value clean OOP code (many product companies, fintechs, and companies using Java/C++ heavily).

How they relate

Think of HLD as **the city plan** (where are the roads, water lines, power grid, zoning) and LLD as **the blueprint of one building** on that city plan (room layout, plumbing, wiring). A real system needs both: HLD without LLD is a system that can't actually be built correctly; LLD without HLD is a beautifully engineered component that falls over the moment real traffic hits it. Big Tech interview loops often test **both** — one HLD round (e.g. "design a ride-sharing app") and one LLD round (e.g. "design the trip-matching class model") for the same overall system.

Aspect	HLD	LLD
Focus	Architecture, components, data flow at scale	Classes, interfaces, objects, algorithms inside one component

Aspect	HLD	LLD
Output	Component diagram, API contracts, DB choice, scaling strategy	Class diagram, method signatures, working/pseudo code
Typical duration	45 min	45–60 min (sometimes requires actual runnable code)
Concerns	Availability, partition tolerance, replication, sharding, caching, queues	Encapsulation, extensibility, SOLID, design patterns, edge cases
Example prompt	"Design Instagram"	"Design the class model for a parking lot"
Failure mode if skipped	System can't handle real traffic/scale	Code becomes unmaintainable spaghetti as requirements grow

Why interviewers use system design rounds at all

1. **Ambiguity handling** — real requirements are never fully specified; can you ask the right clarifying questions instead of guessing or over-engineering?
2. **Trade-off reasoning** — there is no single "correct" answer; can you articulate *why* you chose SQL over NoSQL, or push vs pull for a feed, with actual justification?
3. **Communication** — can you drive a 45-minute technical conversation, think out loud, and adapt when the interviewer pushes back or changes constraints mid-interview?
4. **Depth on demand** — can you go deep into any one area (e.g. "how exactly does your rate limiter behave under a burst of 10,000 requests in 1 second?") when probed?
5. **Signal about seniority** — junior candidates describe *a* solution; senior candidates describe *the space of solutions* and justify the one they'd pick for the stated constraints, while flagging what they'd revisit if constraints changed.

---

2. Functional vs Non-Functional

Requirements

The single most common mistake in a system design interview is **skipping requirements gathering and jumping straight to drawing boxes.** Interviewers actively penalize this because in the real world, building the wrong thing perfectly is worse than not building at all. Spend the **first 5-7 minutes** of any HLD interview here. It is not throat-clearing — it directly determines every downstream decision (which database, whether you need strong consistency, whether you need a queue at all).

Functional Requirements (FRs) — What the system must *do*

These are the concrete features/use-cases. Elicit them by asking the interviewer questions like:

- "Who are the users/actors of this system?" (end users, admins, internal services?)
- "What are the 3-5 core actions a user takes?" (e.g., for a URL shortener: shorten a URL, redirect a short URL to the original, optionally track click analytics)
- "Is there anything explicitly out of scope?" (e.g., "we don't need custom aliases in v1")

Rule of thumb: narrow FRs down to **3-5 core features**. If you list 15 features you will run out of time doing a shallow pass over everything instead of a deep pass over what matters. Explicitly say out loud: *"Let's scope this to these 4 features and treat X, Y as out of scope for this discussion — I'm happy to circle back if we have time."* This single sentence signals strong seniority.

Non-Functional Requirements (NFRs) — How the system must *behave*

These are the quality attributes that actually drive your architecture decisions. The big ones, and the questions that surface them:

NFR	Question to ask	Why it matters
Scale	"How many users? DAU/MAU? Requests/sec at	Determines if you need sharding, caching, CDNs at

NFR	Question to ask	Why it matters
	peak?"	all
Read vs Write ratio	"Is this read-heavy or write-heavy?"	Determines caching strategy, replication topology
Latency	"What's an acceptable p99 latency for the core action?"	Determines whether you can afford synchronous cross-region calls
Availability	"What uptime do we need — 99.9%? 99.99%?"	Determines redundancy, failover, multi-region needs
Consistency	"Is stale data acceptable, or must every read see the latest write?"	Determines SQL vs NoSQL, sync vs async replication
Durability	"Can we ever lose data?"	Determines write-ahead logs, replication factor, backup strategy
Security/Compliance	"Any regulatory constraints (PCI, HIPAA, GDPR)?"	Determines encryption, data residency, audit logging needs
Cost	"Are we optimizing for cost or for raw performance?"	Determines whether to over-provision or scale precisely

A great habit: after gathering FRs and NFRs, **restate them back** in one or two sentences before moving on: *"So to confirm — we're building a URL shortener supporting shorten + redirect, at 100M shortens/month and a 100:1 read:write ratio, needing low-latency redirects (<100ms p99) and 99.9% availability, eventual consistency is fine. Sound right?"* This single move de-risks the entire rest of the interview because you're now building against an agreed contract, not a guess.

Common NFR trade-offs interviewers love to probe

- **Consistency vs Availability** (CAP theorem — see Part 2 §7): you cannot have both during a network partition. Which do you pick, and why, for *this specific*

system?

- **Latency vs Consistency**: strongly consistent reads across regions are slow (must wait for quorum/leader); eventually consistent reads are fast but can be stale.
- **Cost vs Redundancy**: 3x replication for durability costs 3x storage. Is that justified for this data?
- **Simplicity vs Scalability**: do you really need Kafka for a system with 100 req/sec? (No — don't over-engineer. Right-size the solution to the stated scale.)

---

3. Back-of-the-Envelope Estimation

Estimation math is where many candidates freeze, but it's mechanical once you memorize a handful of numbers and a repeatable process. The *goal* of estimation is not to be precisely correct — it's to know whether you're building something that fits on one server, needs a dozen servers, or needs a globally distributed system with sharding. Order of magnitude is what matters.

3.1 Numbers Every Engineer Should Know (Latency)

Operation	Approx Latency
L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference (RAM)	100 ns
Compress 1KB with a fast algorithm	3,000 ns (3 µs)
Send 1KB over 1 Gbps network	10,000 ns (10 µs)
Read 4KB randomly from SSD	150,000 ns (150 µs)
Read 1MB sequentially from RAM	250,000 ns (250 µs)
Round trip within same data center	500,000 ns (500 µs)

Operation	Approx Latency
Read 1MB sequentially from SSD	1,000,000 ns (1 ms)
Disk seek (HDD)	10,000,000 ns (10 ms)
Read 1MB sequentially from HDD	20,000,000 ns (20 ms)
Round trip between US and Europe	150,000,000 ns (150 ms)

Takeaways to actually use in an interview:

- Memory is ~100x faster than SSD, and SSD is ~10-20x faster than spinning disk.
- A cross-region round trip (150ms) can dominate your entire latency budget — this is why systems replicate data close to users (CDN, regional read replicas) instead of always hitting a single central database.
- Network within a data center (~0.5ms) is cheap; treat cross-DC/cross-region calls as "expensive" and minimize them in your critical path.

3.2 Availability — The "Nines" Table

Availability	Downtime/year	Downtime/month	Downtime/day
99% ("two nines")	3.65 days	7.2 hours	14.4 minutes
99.9% ("three nines")	8.76 hours	43.2 minutes	1.44 minutes
99.99% ("four nines")	52.56 minutes	4.32 minutes	8.64 seconds
99.999% ("five nines")	5.26 minutes	25.9 seconds	864 milliseconds

3.3 Storage Size Reference

Unit	Size
1 character (ASCII)	1 byte
1 KB	1,000 bytes (or 1,024 for KiB)
1 MB	1,000 KB
1 GB	1,000 MB
1 TB	1,000 GB
1 PB	1,000 TB

Unit	Size
A typical UUID	16 bytes (36 chars as string)
A typical tweet (280 chars + metadata)	~300 bytes text, but ~1KB with metadata (IDs, timestamps)
A compressed JPEG photo (phone camera)	2-5 MB
A 3-minute song (MP3, 128kbps)	~3 MB
A 10-minute 1080p video (H.264)	~130 MB

3.4 The Estimation Process (Step by Step)

Step 1 — Traffic estimation (QPS). Start from DAU (daily active users) and actions/user/day.

```
QPS (average) = (DAU × actions per user per day) / 86,400 seconds
Peak QPS = Average QPS × peak factor (commonly 2x-5x; ask or assume 3x if not
given)
```

Worked example: A URL shortener with 100M DAU where each user shortens 0.01 URLs/day (i.e., 1M new URLs/day) and reads (redirects) 100x more than writes:

```
Write QPS (avg)  = 1,000,000 / 86,400 ≈ 11.5 writes/sec
Read QPS (avg)   = 11.5 × 100 ≈ 1,150 reads/sec
Peak (×3)        ≈ 35 writes/sec, 3,450 reads/sec
```

This immediately tells you: this is a **read-heavy** system (100:1) — so caching redirects aggressively and having many read replicas matters far more than write throughput.

Step 2 — Storage estimation.

```
Storage/day = writes/day × avg size per record
Storage/year = Storage/day × 365
Total storage (N years) = Storage/year × N (+ replication factor)
```

Worked example (continuing URL shortener): Each record (short code + long URL + metadata) ≈ 500 bytes. At 1M new URLs/day:

```
Storage/day  = 1,000,000 × 500 bytes = 500 MB/day
Storage/year = 500 MB × 365 ≈ 182.5 GB/year
5-year total ≈ 913 GB ≈ ~1 TB (before replication)
With 3x replication for durability ≈ 3 TB
```

This tells you: this dataset **easily fits on a single beefy database server** (even un-sharded) for years — so don't over-engineer a 50-shard cluster for this; that's a red flag to interviewers, not a strength.

Step 3 — Bandwidth estimation.

```
Incoming bandwidth = write QPS × avg request size
Outgoing bandwidth = read QPS × avg response size
```

Worked example: 3,450 reads/sec × ~2KB average redirect response ≈ 6.9 MB/sec outgoing — trivially handled by a small fleet of app servers behind a CDN/cache.

Step 4 — Memory/cache estimation (for caching layers). A common heuristic is the **80/20 rule**: cache the ~20% of data responsible for ~80% of traffic (hot data). If daily reads are 100M and unique URLs read are much fewer (say 20M unique active short codes), and each cached entry is ~500 bytes:

```
Cache size to hold "hot" 20% ≈ 20,000,000 × 0.2 × 500 bytes = 2 GB
```

2GB comfortably fits in a single Redis instance's memory, telling you a single-node (or small cluster) cache in front of your DB removes the vast majority of DB read load.

Step 5 — Server count estimation.

```
Servers needed = Peak QPS / QPS a single server can handle
```

If a single app server instance handles ~1,000 req/sec (typical assumption for a lightweight stateless service, adjust based on stated workload complexity), and peak QPS is 3,450 + 35 ≈ 3,500, you need **~4 servers** at peak (round up, add 1-2 for redundancy/ headroom → say 5-6 servers behind a load balancer).

3.5 A Reusable Estimation Script (say this out loud in interviews)

1. "Let's estimate DAU and the core action's frequency to get average and peak QPS."
2. "Let's estimate the read:write ratio — this tells us whether to optimize for caching."
3. "Let's estimate storage per record and growth over N years — this tells us if we need sharding or a single database is fine."
4. "Let's estimate bandwidth for the hottest endpoint."
5. "Let's sanity check: does this fit on one machine, needs a handful of machines, or needs a genuinely distributed/sharded architecture?" — **this single conclusion should drive every subsequent design decision**, and stating it explicitly shows the interviewer you understand *why* you're about to propose the architecture you're about to propose.

3.6 Common Estimation Mistakes

- **Over-precision**: spending 10 minutes computing exact numbers. Round aggressively — "~1,000 QPS" not "987.43 QPS". The interviewer cares about order of magnitude.
- **Ignoring peak vs average**: systems must be provisioned for peak, not average — always apply a peak multiplier (2x-10x depending on domain; e.g. ticket sales for a popular event can spike 50-100x above average).
- **Forgetting replication factor** in storage math (3x is a common default for durability).
- **Not connecting the number back to a decision.** Estimation is worthless if you don't say "...and therefore we need N servers / need sharding / don't need sharding / need a cache / a single Postgres instance is enough." Always close the loop.

Part 2 — Core Building Blocks

4. Scalability

Scalability is a system's ability to handle growing load (more users, more data, more requests) by adding resources, ideally with graceful, near-linear improvement. There are two fundamental strategies, and almost every design decision downstream depends on which one (or mix) you pick.

4.1 Vertical Scaling (Scale Up)

Add more power (CPU, RAM, faster disk) to a **single machine**.

- **Pros:** simple — no code changes, no distributed systems complexity, no data partitioning needed, strong consistency is trivial (it's one machine).
- **Cons:** hard physical ceiling (there's a biggest machine money can buy), single point of failure, expensive at the high end (cost grows faster than linearly), downtime during upgrade (usually needs a restart).
- **When to use:** early-stage systems, systems with strict consistency needs that are hard to distribute (e.g. some monolithic relational DB workloads), or as a *stopgap* before a proper horizontal redesign.

4.2 Horizontal Scaling (Scale Out)

Add **more machines** and distribute load/data across them.

- **Pros:** near-unlimited ceiling, no single point of failure (if designed with redundancy), incremental cost (add one more commodity box at a time), can scale down too.
- **Cons:** real complexity — now you need load balancing, data partitioning/sharding, distributed consistency, network calls between nodes, and observability across many machines. "Fallacies of distributed computing" apply (network is not reliable, latency isn't zero, bandwidth isn't infinite).
- **When to use:** the default choice for any system expected to scale beyond one box — essentially all modern large-scale systems (web services, most databases used at scale, caches, queues).

4.3 Statelessness — The Enabler of Horizontal Scaling

The single biggest architectural decision that makes horizontal scaling easy is: **keep your application servers stateless.** Any request should be servable by *any* instance — session data, user state, and uploaded files should live in a shared store (DB, distributed cache like Redis, or object storage), never in an individual server's local memory/disk.

- Stateless servers can be freely added/removed behind a load balancer (auto-scaling groups scale trivially).
- If a server crashes, no user-visible state is lost (beyond the in-flight request).
- Session data, when needed, is stored in a shared session store (Redis) or encoded in a signed client-side token (JWT) so *any* server can validate it.

4.4 Scaling the Data Tier

Scaling app servers is "easy" once stateless; the hard part is almost always the **database** because data has gravity — it must live somewhere consistent. Strategies (each detailed later in §8 Databases):

1. **Read replicas** — scale reads horizontally (most systems are read-heavy).
2. **Caching** — remove repeat reads from hitting the DB at all (§9).
3. **Sharding/partitioning** — scale writes and total data volume by splitting data across multiple database instances by some partition key.
4. **Denormalization / CQRS** — separate the write model from heavily-read-optimized views.

4.5 Scaling Patterns Summary Table

Pattern	Scales	Complexity added	Typical use
Vertical scaling	Single node capacity	Low	Early stage, simple relational DB
Stateless app servers + LB	Compute/request throughput	Low-Medium	Nearly every web service
Read replicas	Read throughput	Medium	Read-heavy services (most systems)

Pattern	Scales	Complexity added	Typical use
Caching (CDN/Redis)	Read throughput, latency	Medium	Hot-data-heavy systems
Sharding	Write throughput, storage volume	High	Very large datasets/write volume
Async processing (queues)	Decouples spiky load from processing	Medium	Anything with bursty/expensive work
Microservices	Independent scaling per domain	High (operational)	Large orgs, independently-scaling domains

4.6 Interview Angle

When asked to "make this scale to 10x/100x traffic," the systematic answer is:

1. Identify the bottleneck first (CPU-bound app servers? DB reads? DB writes? network bandwidth? a single hot partition/celebrity user?) — **don't scale blindly.**
2. Apply the cheapest fix first: add stateless app server replicas behind a LB, add a cache in front of the DB, add read replicas.
3. Only reach for sharding/microservices decomposition when read replicas + caching are provably insufficient (e.g. write volume itself, not just read volume, exceeds a single primary's capacity) — this shows engineering judgment, not just "throw NoSQL at it."

---

5. Load Balancing

A **load balancer (LB)** distributes incoming traffic across a pool of backend servers so no single server is overwhelmed, while also detecting and routing around unhealthy servers.

5.1 Layer 4 vs Layer 7 Load Balancing

	Layer 4 (Transport)	Layer 7 (Application)
Operates on	IP + TCP/UDP port	HTTP headers, URL path, cookies, payload
Speed	Faster (no payload inspection)	Slower (parses request content)
Routing intelligence	None — pure IP/port based	Can route by path (`/api/*` vs `/static/*`), header, cookie (sticky sessions)
Can terminate TLS	Sometimes (passthrough is common)	Yes, typically terminates TLS here
Examples	AWS NLB, IPVS, basic hardware LBs	Nginx, HAProxy, AWS ALB, Envoy
Use case	Very high throughput, protocol-agnostic (e.g. raw TCP for a game server)	Web traffic needing smart routing (microservices API gateway, A/B testing by header)

5.2 Load Balancing Algorithms

- **Round Robin** — requests distributed sequentially across servers. Simple, works well when servers are homogeneous and requests are roughly uniform cost.
- **Weighted Round Robin** — like round robin but some servers get proportionally more traffic (useful for heterogeneous hardware).
- **Least Connections** — route to the server with the fewest active connections. Better than round robin when request processing time varies significantly.
- **Least Response Time** — route to the server with fewest active connections *and* lowest recent response time.
- **IP Hash / Consistent Hash** — hash the client IP (or another key) to consistently route the same client to the same server — useful for sticky sessions or cache locality (see §14 Consistent Hashing for the deeper distributed-systems version of this idea).
- **Random (with 2 choices)** — pick 2 random servers, route to the less loaded of the two; surprisingly effective and simple ("power of two choices").

5.3 Health Checks

LBs periodically probe backend servers (HTTP GET `/health`, TCP connect, or a custom check) and stop routing to instances that fail checks. Two flavors:

- **Active health checks** — LB proactively pings instances.
- **Passive health checks** — LB observes real traffic failures/timeouts and ejects a server after N consecutive failures ("circuit breaking" at the LB level).

5.4 High Availability of the LB Itself

A load balancer is itself a potential single point of failure — solved via:

- **DNS round robin** across multiple LB IPs, or
- **Active-passive LB pair** with a floating/virtual IP (VRRP/keepalived) that fails over, or
- **Anycast IP** where the same IP is announced from multiple locations and routers direct traffic to the nearest healthy one (used by large CDNs/DNS providers).

5.5 Global vs Local Load Balancing

- **Local LB** — distributes traffic across servers within one data center/region.
- **Global Server Load Balancing (GSLB)** — distributes traffic across *regions* (e.g. via GeoDNS routing users to their nearest region, combined with health checks so an entire down region can be failed over).

5.6 Interview Angle

Common follow-ups: *"What happens if the LB itself fails?"* (see 5.4), *"How would you do sticky sessions without breaking horizontal scaling?"* (prefer NOT to need sticky sessions at all — keep servers stateless; if you must, use consistent hashing or a shared session store instead of pinning), *"L4 or L7 for this system?"* (L7 if you need path/header-based routing or TLS termination + smart retries; L4 if you need max throughput/protocol flexibility).

---

6. Availability & Reliability

Availability = the fraction of time a system is operational and able to serve requests correctly. **Reliability** = the probability a system performs correctly over a given period without failure. They're related but distinct: a system can fail often but recover instantly (high reliability issues, but if recovery is fast, availability numbers can still look OK) — in practice interviews mostly care about the combined, user-visible "was it up and correct."

6.1 SLA, SLO, SLI

- **SLI (Service Level Indicator)** — an actual measured metric, e.g. "p99 latency of the `/checkout` endpoint" or "% of requests returning 2xx."
- **SLO (Service Level Objective)** — the internal target for an SLI, e.g. "p99 latency < 200ms" or "99.95% success rate," used to drive engineering priorities and alerting.
- **SLA (Service Level Agreement)** — an external, often contractual, commitment to customers (usually a looser version of the SLO, with penalties/credits if breached).

6.2 Redundancy & Replication

Eliminating single points of failure (SPOFs) is the core lever for availability:

- **N+1 redundancy** — run one more instance than the minimum needed, so any single failure doesn't cause an outage.
- **Active-active** — multiple instances all serve live traffic simultaneously (higher resource utilization, but requires handling concurrent writes/conflicts if stateful).
- **Active-passive** — a standby instance is kept in sync (via replication) but doesn't serve traffic until a failover is triggered. Simpler consistency story, slightly slower failover (has to detect failure + promote standby).
- **Data replication** — keep multiple copies of data across nodes/AZs/regions so a node failure doesn't lose data (see §8.4 for replication topologies).

6.3 Failover

The process of automatically switching to a redundant/standby component when the primary fails. Key aspects interviewers probe:

- **Detection time** — how fast do you know the primary is down? (health checks, heartbeat timeouts — usually a few seconds).
- **Promotion time** — how fast can a replica become the new primary? (instant if already warm/in-sync; slower if it needs to replay logs).
- **Split-brain risk** — if the network partitions such that both old-primary and new-primary think they're the leader, you can get data corruption. Solved via consensus protocols (Raft/Paxos, see §15) or a quorum-based leader election, or an external coordinator (e.g. ZooKeeper/etcd).

6.4 Multi-AZ and Multi-Region

- **Multi-AZ (Availability Zone)** — deploy across physically separate data centers within one region (separate power/network/cooling, low latency between them, typically <2ms). Protects against a single data center failure. This is the **baseline** most production systems should have.
- **Multi-Region** — deploy across geographically distant regions (protects against a regional disaster, reduces latency for global users via serving from the nearest region). Adds real complexity: cross-region data replication is slow (tens-hundreds of ms), so you must choose between synchronous replication (safe but slow writes) or asynchronous replication (fast writes, risk of data loss on region failure — RPO > 0).

6.5 Disaster Recovery Metrics

- **RTO (Recovery Time Objective)** — how long can the system be down after a disaster before it must be back up?
- **RPO (Recovery Point Objective)** — how much data (in time) can you afford to lose? (e.g. RPO of 5 minutes means backups/replication must be no more than 5 min stale.)

6.6 Graceful Degradation

When a dependency fails, a well-designed system degrades gracefully instead of failing entirely:

- Serve stale cached data instead of an error if the DB is slow/down.

- Disable a non-critical feature (e.g. "recommended for you" widget) if its service is down, while keeping the core flow (e.g. checkout) working.
- **Circuit breakers** — stop calling a failing downstream dependency for a cooldown period instead of piling up timeouts and exhausting your own resources (thread pool exhaustion, cascading failure).
- **Bulkheads** — isolate resource pools (e.g. separate thread pools/connection pools per downstream dependency) so one slow dependency can't starve resources needed by others.

6.7 Interview Angle

Availability questions are almost always paired with a trade-off: *"How would you make this 99.99% available, and what does that cost you?"* Good answers explicitly connect a nines target (§3.2 table) to concrete mechanisms (replicas count, multi-AZ, health checks + auto-failover, circuit breakers) and acknowledge the cost (infra $, consistency trade-offs, operational complexity).

---

7. Consistency, CAP & PACELC

7.1 The CAP Theorem

For a distributed system, during a **network partition**, you must choose between:

- **C — Consistency**: every read receives the most recent write (or an error).
- **A — Availability**: every request receives a (non-error) response, without guarantee it's the most recent write.
- **P — Partition Tolerance**: the system continues to operate despite network partitions (message loss/delay between nodes).

The actual theorem: partitions *will* happen in any real distributed system (network is unreliable) — so **P is not really optional**, it's a given. The real choice CAP forces on you is: **when a partition happens, do you sacrifice C or A?**

- **CP systems** (choose consistency): refuse to serve a request if they can't guarantee it's consistent (e.g. return an error/timeout rather than stale data). Examples: HBase, MongoDB (in default config with majority reads/writes), ZooKeeper, etcd, most systems built on Raft/Paxos consensus.

- **AP systems** (choose availability): always respond, potentially with stale/conflicting data that gets reconciled later. Examples: Cassandra (tunable, but classically AP), DynamoDB (classic Dynamo model), Riak, CouchDB.

Common misconception to avoid in interviews: CAP is *only* about behavior **during a partition**. When there's no partition (the common case!), a well-designed system can be both consistent and available. CAP does not mean "pick 2 of 3 permanently" — it means "pick which one you sacrifice during the rare-but-inevitable partition window."

7.2 PACELC — The More Complete Model

PACELC extends CAP to also describe behavior when there is **no** partition, which is most of the time and arguably more practically important:

> **If Partitioned:** choose **A**vailability or **C**onsistency (as in CAP). **Else (no partition):** choose **L**atency or **C**onsistency.

This captures a crucial real-world trade-off CAP alone misses: even with a perfectly healthy network, achieving strong consistency (e.g. synchronously replicating to a quorum before acknowledging a write) costs you latency, versus acknowledging immediately and replicating asynchronously (lower latency, weaker consistency guarantee).

- **PA/EL systems** (e.g. Cassandra, DynamoDB default): prioritize availability during partition, and low latency (over strict consistency) normally. Best for: user-facing, latency-sensitive, tolerant-of-slight-staleness data (social media feeds, product catalogs, view counts).
- **PC/EC systems** (e.g. traditional RDBMS with synchronous replication, HBase, most Raft-based stores): prioritize consistency during partition, and consistency (over latency) normally. Best for: financial transactions, inventory counts, anything where a wrong/stale read causes real-world harm (overselling, double-spending).

7.3 Consistency Models (Spectrum, Strongest to Weakest)

- **Strict/Linearizable Consistency** — every operation appears to take effect instantly at some point between its invocation and response, and all nodes agree on a single global order. Strongest, most expensive (requires consensus/coordination on every write).

- **Sequential Consistency** — all nodes see operations in the *same* order, but not necessarily in real-time order.
- **Causal Consistency** — operations that are causally related (B happens after reading A) are seen by everyone in that order; unrelated (concurrent) operations may be seen in different orders by different nodes. (E.g. a reply to a comment always appears after the comment, but two unrelated comments may appear in different orders to different viewers.)
- **Read-Your-Writes Consistency** — a user always sees their own writes immediately, even if other users see them later (common pattern: route a user's reads to the primary/same replica right after they write, or to a replica guaranteed caught-up).
- **Eventual Consistency** — given no new writes, all replicas *will* eventually converge to the same value, with no bound on how long "eventually" takes (usually milliseconds-seconds in practice, but not guaranteed). Cheapest, most available, most scalable.

7.4 Strong vs Eventual Consistency — Practical Interview Framing

	Strong Consistency	**Eventual Consistency**
Read always sees latest write?	Yes	No (may be stale briefly)
Latency	Higher (coordination cost)	Lower
Availability during partition	Lower (may reject requests)	Higher
Typical use cases	Bank balance, inventory count, seat booking, leader election	Social media likes/views, DNS, CDN cache, product recommendations, feed ranking
How achieved	Synchronous replication, quorum reads/writes, single-leader with reads from leader	Asynchronous replication, read from any replica, gossip protocols

7.5 Interview Angle

Whenever a system has a "money" or "inventory" or "unique resource allocation" element (payments, seat booking, coupon redemption), immediately flag: *"This needs strong consistency for the [specific operation] — I'll use [synchronous replication / a transaction / a distributed lock / quorum writes] here, but the rest of the system (e.g. browsing, search) can be eventually consistent for better latency and availability."* This shows you don't apply one consistency model uniformly — real systems mix them per data type.

8. Databases Deep Dive

Choosing and correctly using a database is the single highest-leverage decision in most system designs. This section covers the taxonomy, internals, and scaling techniques you're expected to reason about.

8.1 SQL vs NoSQL — The Real Decision Framework

Stop thinking "SQL = old/slow, NoSQL = modern/fast" — that's not how the decision works.

	SQL (Relational)	**NoSQL**
Schema	Fixed, strongly typed schema (migrations needed to change)	Flexible/schema-less (or schema-on-read)
Relationships	First-class (JOINs, foreign keys)	Usually denormalized; app handles relationships
Transactions	Full ACID across multiple rows/tables	Often limited to single document/partition-key scope
Scaling	Vertical easier; horizontal (sharding) is manual/harder	Built for horizontal scaling from day one
Query flexibility	Very high (arbitrary joins/aggregations via SQL)	Lower — usually optimized for known access patterns
Consistency	Strong by default	Often tunable, frequently eventual by default

	SQL (Relational)	NoSQL
Best for	Data with complex relationships, need for ad-hoc queries, strong consistency (orders, payments, inventory)	Massive scale, simple access patterns known in advance, flexible/evolving schema, high write throughput

Rule of thumb for interviews: default to a relational DB (Postgres/MySQL) unless you have a *specific* reason not to (e.g. proven need for horizontal write scaling beyond what sharding a relational DB reasonably supports, or a genuinely schema-flexible domain, or an access pattern that's a perfect fit for a specialized NoSQL model like a graph or wide column store). Saying "I'd pick NoSQL because it's more scalable" without justification is a red flag — many NoSQL choices at scale are wrong when a well-indexed, well-sharded SQL DB would have worked and given you transactions/joins for free.

8.2 NoSQL Types (know when to use each)

- **Key-Value stores** (Redis, DynamoDB, Riak) — simplest model, O(1) get/put by key. Best for: caching, session storage, simple lookups (e.g. shortcode → URL).
- **Document stores** (MongoDB, Couchbase) — store semi-structured JSON/BSON documents, queryable on any field. Best for: content with nested/variable structure (product catalogs, user profiles, CMS content).
- **Wide-column / column-family stores** (Cassandra, HBase, Bigtable) — rows can have different columns, optimized for huge write throughput and range scans over a partition key + clustering key. Best for: time-series data, event logging, IoT data, messaging (e.g. per-user message history).
- **Graph databases** (Neo4j, Amazon Neptune) — nodes + edges, optimized for traversing relationships (friend-of-friend, recommendation graphs, fraud rings). Best for: social graphs, recommendation engines, fraud detection.
- **Search engines** (Elasticsearch, Solr) — inverted-index based, optimized for full-text search and complex filtering/faceting. Best for: search bars, log search, analytics dashboards. (Often used *alongside* a primary DB, not as the primary store.)
- **Time-series databases** (InfluxDB, TimescaleDB, Prometheus) — optimized for timestamp-indexed data with high write rates and range/aggregation queries. Best for: metrics, monitoring, IoT sensor data.

8.3 Indexing

An index is a data structure (usually a **B-Tree** or **LSM-Tree**) that trades write cost and storage for dramatically faster reads on a specific column/key.

- **B-Tree indexes** (used by most RDBMS like Postgres/MySQL InnoDB): balanced tree, O(log n) reads/writes, good for range queries, but each write requires updating the tree in place (random I/O).
- **LSM-Trees (Log-Structured Merge Trees)** (used by Cassandra, RocksDB, LevelDB, HBase): writes go to an in-memory structure (memtable) + append-only log, later flushed and merged ("compacted") into sorted files on disk. Writes are sequential (fast!), but reads may need to check multiple files (mitigated with bloom filters). Great for **write-heavy** workloads.
- **Hash indexes** — O(1) exact-match lookup, but no range query support.
- **Composite indexes** — index on multiple columns; column order matters (can serve queries filtering on a prefix of the columns, e.g. index on `(user_id, created_at)` serves "all posts by user X" and "all posts by user X after date Y" but not "all posts after date Y" alone).
- **Trade-off to always mention:** every index speeds up reads on that column but slows down writes (every insert/update must also update the index) and adds storage. Don't over-index; index based on actual query patterns.

8.4 Replication

Copying data across multiple nodes for durability and read scalability.

- **Single-leader (master-slave) replication** — one primary accepts all writes, propagates to N replicas (read-only). Reads can be served by any replica (scales reads), but writes are bottlenecked by the single primary. Simple, most common. Failover requires promoting a replica (brief unavailability + risk of losing the last few un-replicated writes if async).
- **Multi-leader (master-master) replication** — multiple nodes accept writes, replicate to each other. Enables writes in multiple regions (lower write latency globally), but introduces **write conflicts** (two regions modify the same record concurrently) requiring conflict resolution (last-write-wins, CRDTs, application-level merge logic).
- **Leaderless replication** (Dynamo-style, e.g. Cassandra) — any node can accept a read/write; client (or coordinator) writes to N nodes and reads from N nodes,

using **quorums** to guarantee overlap: with replication factor N, write quorum W, read quorum R, if `W + R > N` you're guaranteed at least one node in the read set has the latest write (tunable consistency).
- **Sync vs Async replication:** synchronous = primary waits for replica ack before confirming the write to the client (strong durability, higher latency, replica failure can block writes); asynchronous = primary confirms immediately, replicates in background (lower latency, risk of data loss if primary fails before replicating).

8.5 Partitioning / Sharding

Splitting a single logical dataset across multiple physical database nodes so no single node holds (or must serve) all the data/traffic.

Partitioning strategies:

- **Range-based sharding** — partition by a key range (e.g. user IDs 1-1M on shard 1, 1M-2M on shard 2). Simple, supports range queries within a shard, but risks "hot shards" if data/ traffic isn't uniform (e.g. all new users go to the last shard).
- **Hash-based sharding** — hash the partition key, mod by number of shards (or use consistent hashing, §14) to pick a shard. Spreads load evenly, but loses range-query locality (a range scan now hits every shard) and resharding is painful without consistent hashing.
- **Directory-based sharding** — a lookup service maps each key (or key range) to a shard, giving full flexibility to rebalance, at the cost of an extra lookup hop and the lookup service itself becoming critical infrastructure.
- **Geo-based sharding** — partition by user's region/geography — naturally reduces latency (data lives close to the user) and can help with data residency/compliance requirements.

Choosing a partition/shard key — the most important decision:

- Must have **high cardinality** (many distinct values) to spread data evenly.
- Must match your **dominant access pattern** (most queries should be satisfiable by hitting one shard, not fanning out to all of them — fan-out queries are expensive and don't scale).
- Avoid **hot keys/hot shards** — e.g. sharding a "posts" table by `created_date` means *all* writes today go to one shard (bad); sharding by `user_id` spreads writes across shards evenly (assuming no single mega-celebrity user, which needs special-casing).

Challenges sharding introduces (always mention these — this is what makes sharding "the big hammer," not the default choice):

- **Cross-shard joins/transactions** become expensive or impossible without a distributed transaction protocol (2PC/Saga, see §13.6).
- **Resharding** (adding/removing shards) requires moving data — consistent hashing (§14) minimizes how much data moves.
- **Celebrity/hot-key problem** — one key (e.g. a viral post, a huge account) gets disproportionate traffic that overwhelms its single shard; mitigated by further sub-partitioning that key's data or adding a cache layer in front.

8.6 ACID vs BASE

ACID (traditional RDBMS transactions)	BASE (many distributed/NoSQL systems)
Atomicity — all-or-nothing	**B**asically **A**vailable — system guarantees availability over consistency
Consistency — DB moves between valid states only	**S**oft state — state may change over time even without new input (as replicas converge)
Isolation — concurrent transactions don't interfere	**E**ventual consistency — replicas converge given enough time
Durability — committed data survives crashes	

ACID is what you want for money/inventory/anything where partial failure is unacceptable. BASE is what you accept in exchange for the massive availability/scalability of leaderless, eventually-consistent systems. Modern systems often mix both: ACID for the "core of truth" transactional data, BASE for derived/denormalized read-optimized views.

8.7 Normalization vs Denormalization

- **Normalization** (3NF etc.) — eliminate redundant data, each fact stored once, related via foreign keys. Pros: no update anomalies, smaller storage, strong consistency. Cons: reads require JOINs (slower, harder to shard/scale).

- **Denormalization** — duplicate data to avoid JOINs at read time (e.g. store the author's display name directly on each post instead of joining to a users table every read). Pros: much faster reads, easier to shard (each denormalized row is self-contained). Cons: update anomalies (must update every duplicate on change, or accept staleness), more storage.
- **Interview framing:** "This system is read-heavy with a well-known query pattern, so I'll denormalize [specific field] to avoid a join on the hot path, accepting that [field] updates need to fan out (via an async job/event) to duplicated copies."

8.8 Transactions & Isolation Levels

- **Isolation levels** (weakest to strongest): Read Uncommitted → Read Committed → Repeatable Read → Serializable. Higher isolation = fewer anomalies (dirty reads, non-repeatable reads, phantom reads) but more locking/lower concurrency.
- **Optimistic vs Pessimistic concurrency control:** pessimistic locks the row upfront (safe, but reduces concurrency, risk of deadlocks); optimistic checks a version number/ timestamp at commit time and retries on conflict (higher concurrency, better for low-contention workloads, needs retry logic).
- **Distributed transactions** across shards/services: **Two-Phase Commit (2PC)** — coordinator asks all participants to "prepare" (lock + vote), then "commit" if all voted yes (strong consistency, but blocking — a crashed coordinator/participant can hold locks indefinitely). **Saga pattern** — break a distributed transaction into a sequence of local transactions, each with a compensating action to undo it if a later step fails (eventual consistency, non-blocking, more application complexity to write compensations).

8.9 Interview Angle

Database questions are rarely "SQL or NoSQL" in isolation — they're "given these specific read/write patterns and consistency needs, what do you pick and how do you scale it?" Always walk through: **choice of DB type → schema/access pattern → indexing strategy → replication (for availability/read-scaling) → partitioning (only if genuinely needed for write/volume scale) → consistency requirements per data type.**

---

9. Caching

Caching stores a copy of frequently-accessed data in a faster-access layer to reduce latency and offload the primary data store. It is usually **the single highest-leverage, lowest-complexity lever** for making a read-heavy system fast and scalable — reach for caching before reaching for sharding.

9.1 Where to Cache (Layers, Client to Server)

1. **Client-side cache** (browser cache, mobile app local cache) — zero network hop, but you don't control invalidation across all clients.
2. **CDN (Content Delivery Network)** — caches static/semi-static content (images, JS/CSS, even API responses) at edge locations near users. Massively reduces latency for geographically distributed users and offloads origin servers entirely for cache hits.
3. **Application-level cache (local, in-process)** — e.g. an in-memory `HashMap`/Caffeine cache inside each app server. Fastest (no network hop), but not shared across server instances (each instance has its own copy — can waste memory and cause inconsistency across instances).
4. **Distributed cache** (Redis, Memcached) — shared cache tier accessible by all app servers. Slightly slower than in-process (network hop) but consistent across the fleet and much larger capacity than per-instance memory.
5. **Database-level cache** (query result cache, buffer pool/page cache built into the DB engine itself) — often invisible to the application but still worth knowing exists.

9.2 Caching Strategies (Read Path)

- **Cache-Aside (Lazy Loading)** — application checks cache first; on miss, reads from DB, then populates cache. Most common pattern. Pro: only requested data is cached (no wasted space). Con: first request for any key is always a slow "cache miss," and there's a small window where cache/DB can be inconsistent (mitigated with a reasonable TTL).
- **Read-Through** — the cache itself sits in front of the DB and handles the miss-then-load logic transparently (application only ever talks to the cache). Similar semantics to cache-aside but the loading logic lives in the cache layer, not the app.

9.3 Caching Strategies (Write Path)

- **Write-Through** — write goes to the cache, which synchronously writes through to the DB before acknowledging. Cache always consistent with DB, but write latency = cache + DB write latency combined.
- **Write-Behind (Write-Back)** — write goes to the cache, acknowledged immediately, and the cache asynchronously flushes to the DB later (often batched). Very low write latency, but risk of data loss if the cache crashes before flushing, and DB is temporarily "behind" the cache.
- **Write-Around** — write goes directly to the DB, bypassing the cache; cache is only populated on a subsequent read (cache-aside style). Good when written data isn't immediately re-read (avoids polluting cache with write-once-read-rarely data).

9.4 Cache Eviction Policies

When the cache is full, which entry gets removed to make room for a new one?

- **LRU (Least Recently Used)** — evict the entry not accessed for the longest time. Most common default; good general-purpose choice, implemented with a hashmap + doubly linked list for O(1) get/put (see LLD case study §47).
- **LFU (Least Frequently Used)** — evict the entry accessed the fewest times. Better than LRU when access frequency is a better popularity signal than recency (e.g. some items are rarely-but-consistently accessed and shouldn't be evicted just because of a recent lull).
- **FIFO** — evict the oldest-inserted entry regardless of access pattern. Simple, rarely optimal, but sometimes used for its predictability/simplicity.
- **TTL (Time-To-Live)** — entries expire after a fixed duration regardless of access — used *alongside* LRU/LFU (not a full replacement) to bound staleness.

9.5 Cache Invalidation ("There are only two hard things in Computer Science: cache

invalidation and naming things.")

- **TTL-based expiry** — simplest; accept some staleness up to TTL duration.
- **Write-invalidate** — on write to the DB, explicitly delete (or update) the corresponding cache key so the next read repopulates it fresh.
- **Event-driven invalidation** — DB change triggers an event (via CDC/Change Data Capture or an app-level event) that invalidates relevant cache keys across the fleet — needed when multiple services/paths can write the underlying data.

- **Versioned/keyed cache** — include a version number or timestamp in the cache key itself so old versions simply become unreachable (no explicit deletion needed) — trades memory for invalidation simplicity.

9.6 Cache Stampede / Thundering Herd

When a hot key expires (or the cache restarts cold), a flood of concurrent requests all miss simultaneously and hammer the DB at once, potentially taking it down. Mitigations:

- **Request coalescing / locking** — only let one request per key actually query the DB; others wait for that result (many cache client libraries support this natively).
- **Early/probabilistic refresh** — refresh a hot key slightly *before* it expires (proportionally more likely as it nears expiry) so it rarely fully expires under load.
- **Staggered TTLs (jitter)** — add random jitter to TTLs so many keys inserted at the same time don't all expire in the same instant.
- **Serve stale while revalidating** — return the (slightly) stale cached value immediately while asynchronously refreshing it in the background.

9.7 CDN Deep Dive

A CDN is a globally distributed network of edge servers (Points of Presence, PoPs) that cache content close to end users.

- **Push CDN** — origin proactively pushes content to CDN edges (good for content you know in advance, e.g. a video release, static assets from a build pipeline).
- **Pull CDN** — CDN edge fetches from origin on first request (cache miss) and caches it for subsequent requests (good for long-tail content where you can't predict what will be popular). Most CDNs default to pull-based for general web content.
- CDNs are also used for **dynamic content acceleration** (not just static files) via techniques like edge-side includes, and for absorbing/mitigating DDoS traffic before it reaches your origin.

9.8 Interview Angle

Whenever a system is read-heavy (which most are), explicitly propose: *"I'll put a [CDN/distributed cache] in front of [specific hot endpoint/data], using [cache-aside /* ☑

and [jitter/stampede protection] because [specific justification tied to the numbers from estimation]." Always name the specific data being cached, the strategy, the eviction policy, and the invalidation approach — vague "we'll add a cache" answers score far lower than specific ones.

10. Message Queues & Event-Driven Architecture

Message queues **decouple** producers of work from consumers of work in time and pace, enabling asynchronous processing, load leveling, and independent scaling.

10.1 Why Use a Queue At All

- **Decoupling** — producer doesn't need to know who consumes the message or wait for it to be processed (fire-and-forget from the producer's perspective).
- **Load leveling / buffering** — absorb bursty traffic (e.g. flash sale) without overwhelming downstream consumers; consumers drain the queue at their own sustainable rate.
- **Reliability** — if a consumer crashes mid-processing, the message isn't lost — it stays in the queue (or gets redelivered) until acknowledged.
- **Async/long-running work off the request path** — e.g. sending an email, generating a report, processing a video upload — don't make the user's HTTP request wait for this.

10.2 Queue vs Pub-Sub vs Log

- **Point-to-point queue** (e.g. SQS, RabbitMQ default) — each message is delivered to (and consumed/acknowledged by) exactly **one** consumer, even with multiple consumers competing for messages (this is how you scale consumers horizontally — they act as a competing consumer pool).
- **Publish-Subscribe (Pub-Sub)** (e.g. SNS, Redis Pub/Sub) — each message is delivered to **all** subscribers of a topic — used for fan-out (e.g. notify multiple services when an order is placed).
- **Distributed Log** (Kafka, Pulsar) — messages are appended to a durable, ordered, partitioned log and retained for a configurable period (not deleted on consumption). Multiple independent consumer groups can each read the whole

log at their own pace/offset. Combines pub-sub fan-out with replay-ability (a new consumer can start from the beginning) — this is why Kafka is the default choice for most modern event-driven architectures.

10.3 Kafka vs RabbitMQ vs SQS (Common Comparison Question)

	Kafka	**RabbitMQ**	**AWS SQS**
Model	Distributed commit log, partitioned	Traditional broker, exchange/queue routing	Managed point-to-point queue
Ordering	Guaranteed within a partition	Guaranteed within a queue (mostly)	FIFO queues only (standard queues don't guarantee order)
Throughput	Very high (millions/sec, built for streaming)	High but lower than Kafka at extreme scale	High, fully managed, auto-scales
Message retention	Configurable, can replay (log-based)	Deleted after consumption (typically)	Deleted after consumption (visibility timeout + delete)
Consumer model	Consumer groups pull at their own offset	Push or pull, broker tracks ack	Pull-based, visibility timeout
Routing flexibility	Basic (topic/partition)	Rich (exchanges: direct/topic/fanout/headers)	Basic (queue + optional FIFO/dedup)
Operational complexity	Higher (Zookeeper/KRaft, partition mgmt)	Medium	Zero (fully managed)

	Kafka	RabbitMQ	AWS SQS
Best for	High-throughput event streaming, event sourcing, log aggregation, multiple independent consumers replaying history	Complex routing needs, task queues, RPC-style patterns	Simple, fully-managed decoupling without wanting to run infra

10.4 Delivery Guarantees

- **At-most-once** — message may be lost, never redelivered/duplicated. Fastest, used when occasional loss is acceptable (e.g. non-critical metrics/logs).
- **At-least-once** — message is guaranteed delivered, but may be delivered more than once (e.g. consumer crashes after processing but before acknowledging). Requires consumers to be **idempotent** (processing the same message twice has the same effect as once — e.g. via a dedup/idempotency key). This is the most common guarantee used in practice.
- **Exactly-once** — message is processed exactly once, no loss, no duplication. Hardest and most expensive to guarantee end-to-end (usually achieved via idempotent consumers + transactional writes, e.g. Kafka's transactional producer/consumer API, rather than a "free" broker feature) — in interviews, prefer to say "effectively exactly-once via at-least-once delivery + idempotent processing" rather than claiming true exactly-once is free.

10.5 Backpressure & Dead-Letter Queues

- **Backpressure** — when consumers can't keep up with producers, the queue grows unbounded unless something gives: either the queue applies backpressure to producers (reject/slow down new messages), or you auto-scale consumers, or you accept unbounded growth up to a storage limit (risky).
- **Dead-Letter Queue (DLQ)** — after a message fails processing N times (poison message, e.g. malformed data causing a crash every time), move it to a separate DLQ instead of retrying forever — lets the main queue keep flowing while the DLQ is investigated/ reprocessed manually or by a separate process.

10.6 Event-Driven Architecture Patterns

- **Event notification** — a lightweight event ("OrderPlaced") is published; interested services fetch full details separately if needed. Keeps events small, but causes extra round trips.
- **Event-carried state transfer** — the event itself carries the full relevant state (e.g. the whole order payload), so subscribers don't need to call back — reduces coupling and round trips, at the cost of larger messages and potential staleness if the source changes after the event is sent.
- **Event sourcing** — instead of storing current state, store the full sequence of events that led to it; current state is derived by replaying events (or from a periodic snapshot + recent events). Gives a full audit trail and the ability to rebuild any past state, at the cost of query complexity (usually paired with CQRS, see below).
- **CQRS (Command Query Responsibility Segregation)** — separate the write model (optimized for handling commands/business logic correctness) from the read model (optimized/denormalized for fast queries), synced via events. Common in systems with very different read vs write access patterns.

10.7 Interview Angle

When a design has any "notify/fan-out/decouple/process asynchronously/handle bursty load" requirement, propose a specific queue technology *with justification tied to the ordering, throughput, and replay needs* (not just "we'll use Kafka" reflexively), and explicitly address idempotency for at-least-once delivery and what happens to a poison message (DLQ).

---

11. Networking Essentials

11.1 DNS (Domain Name System)

Translates human-readable domain names to IP addresses, via a hierarchical, heavily-cached lookup: browser/OS cache → recursive resolver (often ISP or 1.1.1.1/8.8.8.8) → root nameservers → TLD (.com) nameservers → authoritative nameserver for the domain. Key system design relevance: DNS is also used for **GeoDNS/traffic routing** (returning different IPs based on the requester's location, used for global load

balancing, see §5.5) and DNS responses are **cached with a TTL**, meaning DNS-based failover isn't instant (clients may keep using a stale/dead IP until their cached TTL expires).

11.2 TCP vs UDP

	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery, ordering, retransmission	No guarantees — "fire and forget"
Overhead	Higher (headers, ACKs, flow/congestion control)	Lower, minimal header
Use cases	Web (HTTP), file transfer, anything needing correctness	Video/voice streaming, gaming, DNS queries, anything latency-sensitive where occasional loss is acceptable

11.3 HTTP/1.1 vs HTTP/2 vs HTTP/3

- **HTTP/1.1** — text-based, one request per TCP connection at a time by default (head-of-line blocking without pipelining), commonly mitigated historically by opening multiple parallel connections per host.
- **HTTP/2** — binary framing, **multiplexing** (many requests/responses interleaved over a single TCP connection, no more head-of-line blocking at the HTTP layer), header compression (HPACK), server push. Still suffers head-of-line blocking **at the TCP layer** (one lost packet stalls all multiplexed streams since TCP guarantees in-order delivery).
- **HTTP/3** — runs over **QUIC** (built on UDP, not TCP), eliminates TCP-layer head-of-line blocking (each stream's lost packets don't block other streams), faster connection setup (combines transport + TLS handshake), better performance on lossy/mobile networks.

11.4 WebSockets vs Long Polling vs Server-Sent Events (SSE)

- **Polling** — client repeatedly asks "anything new?" on an interval. Simple but wasteful (many empty responses) and adds latency (up to the poll interval).
- **Long Polling** — client asks, server holds the request open until there's data (or a timeout), then responds; client immediately re-polls. Reduces empty responses vs plain polling, still has per-request overhead.
- **Server-Sent Events (SSE)** — server pushes a stream of events over a single long-lived HTTP connection; **one-directional** (server → client only), built on plain HTTP (works through most proxies/firewalls easily), auto-reconnect built into the spec. Good for live feeds/notifications where client doesn't need to send data back over the same channel.
- **WebSockets** — full-duplex, persistent TCP connection after an HTTP upgrade handshake; both client and server can push messages anytime. Best for: chat apps, collaborative editing, real-time gaming, trading platforms — anything needing low-latency bidirectional communication. Costs: servers must hold open connections (stateful — complicates horizontal scaling/load balancing, usually needs sticky routing or a shared pub-sub backplane like Redis so any server instance can push to a client connected to a *different* instance).

11.5 TLS Handshake (Brief)

TLS (used in HTTPS) establishes an encrypted channel: client and server exchange certificates/keys, agree on a shared symmetric session key (via asymmetric crypto during the handshake), then all subsequent traffic is encrypted with the fast symmetric key. TLS 1.3 reduced this to effectively 1 round trip (vs 2 in TLS 1.2), which matters for latency-sensitive systems doing lots of new connection setups.

11.6 Interview Angle

Networking questions usually surface as: *"How does the client get real-time updates?"* (pick WebSocket/SSE/polling based on directionality and how real-time it truly needs to be — don't reach for WebSockets if SSE or even polling would satisfy the actual requirement, over-engineering here is a common ding) or *"How do you route users to their nearest region?"* (GeoDNS/anycast, §5.5, §11.1).

---

12. API Design

12.1 REST Principles Recap

- Resources identified by URLs (nouns, not verbs: `/orders/123`, not `/getOrder?id=123`).
- Standard HTTP verbs map to actions: `GET` (read, safe+idempotent), `POST` (create, not idempotent), `PUT` (full replace, idempotent), `PATCH` (partial update, not necessarily idempotent), `DELETE` (remove, idempotent).
- Stateless — each request contains all info needed to process it (no server-side session required for the API contract itself).
- Use proper status codes (2xx success, 3xx redirect, 4xx client error, 5xx server error) — don't return 200 with an error message in the body.

12.2 REST vs GraphQL vs gRPC

	REST	GraphQL	gRPC
Data fetching	Fixed shape per endpoint (over/under-fetching common)	Client specifies exactly the fields needed (no over/under-fetching)	Fixed shape (protobuf schema), like REST
Transport	HTTP/JSON typically	HTTP (usually POST to single endpoint)	HTTP/2, binary protobuf
Performance	Good, JSON parsing overhead	Good, but a single complex query can be expensive server-side	Excellent (binary, HTTP/2 multiplexing, code-gen)
Best for	Public APIs, simple CRUD, wide client compatibility	Mobile/complex UIs needing flexible queries across many resources	Internal service-to-service (microservices), low-latency/high-throughput needs
Tooling/discoverability	Mature, widely understood,	Strong typing via schema,	Strong typing via .proto, codegen

	REST	GraphQL	gRPC
	OpenAPI/Swagger	introspection	for many languages
Caching	Easy (HTTP caching semantics, GET is cacheable)	Harder (usually POST, needs custom caching)	N/A (not typically browser-facing)

Interview default: REST for public/external APIs (ubiquitous tooling, HTTP caching), gRPC for internal service-to-service calls in a microservices architecture (performance + strong contracts), GraphQL when the client (especially mobile, where over-fetching wastes bandwidth/battery) needs flexible querying across many related resources.

12.3 Pagination

- **Offset-based** (`?page=3&size=20` or `?offset=40&limit=20`) — simple, but breaks if items are inserted/deleted between pages (items shift, causing skips/duplicates), and gets slow for deep pages in some DBs (must scan/skip all prior rows).
- **Cursor-based (keyset pagination)** (`?after=<opaque_cursor>&limit=20`, cursor usually encodes the last-seen sort key, e.g. an ID or timestamp) — stable under insertions/deletions, efficient (`WHERE id > last_id ORDER BY id LIMIT 20`, uses an index, no scanning skipped rows). Preferred for infinite-scroll feeds and large datasets.

12.4 Rate Limiting Algorithms (also see full HLD case study §24)

- **Token Bucket** — bucket holds up to N tokens, refills at rate R/sec; each request consumes a token, rejected if bucket empty. Allows bursts up to bucket size while enforcing a long-term average rate. Most commonly used in practice.
- **Leaky Bucket** — requests enter a queue (bucket) and are processed ("leak out") at a fixed rate; smooths bursts into a constant output rate, but a burst can still fill the queue and cause requests to be dropped/delayed.
- **Fixed Window Counter** — count requests in a fixed time window (e.g. per-minute boundary); simple, but allows up to 2x the limit at window boundaries (burst at end of one window + burst at start of next).

- **Sliding Window Log** — keep a timestamp log of every request in the last window; precise, but memory-heavy at high volume.
- **Sliding Window Counter** — approximate sliding window using a weighted average of the current and previous fixed window counts; good accuracy/memory trade-off, commonly used at scale (e.g. Cloudflare's approach).

12.5 Idempotency

An operation is idempotent if performing it multiple times has the same effect as performing it once. Critical for APIs that might be retried (client timeout + retry, at-least-once queue delivery, network blips). Achieved via an **idempotency key**: client generates a unique key per logical operation (e.g. a UUID for "this specific checkout attempt"), sends it with the request; server stores (key → result) and, on seeing a repeated key, returns the cached result instead of re-executing the operation (e.g. charging a card twice).

12.6 API Versioning

- **URI versioning** (`/v1/orders`, `/v2/orders`) — simplest, most visible, easiest to route/cache differently per version.
- **Header versioning** (`Accept: application/vnd.myapi.v2+json`) — cleaner URLs, less visible/discoverable, harder to test casually (e.g. in a browser).
- **Query param versioning** (`/orders?version=2`) — simple but easy to forget/omit.
- Best practice regardless of scheme: support **backward-compatible** additive changes (new optional fields) without bumping version at all; only bump version for breaking changes, and support N-1 (or N-2) versions concurrently with a clear deprecation policy/ sunset timeline communicated to API consumers.

12.7 Interview Angle

When designing any HLD's API surface, keep it to the **3-5 core endpoints** matching your scoped functional requirements (§2), specify the verb, path, request/response shape, and call out pagination/rate-limiting/idempotency explicitly for the endpoints where they matter (e.g. "the `POST /orders` endpoint needs an idempotency key because network retries must not double-charge").

---

13. Microservices vs Monolith

13.1 Monolith

A single deployable unit containing all application logic. **Pros:** simple to develop, test, deploy, and debug early on; no network calls between modules (in-process function calls are fast and transactionally consistent); easier to reason about end-to-end. **Cons:** the entire app scales as one unit (can't scale just the hot module), a bug in one module can crash the whole process, large codebases become harder to onboard onto/build/deploy as team size grows, a single tech stack for everything.

13.2 Microservices

Independently deployable services, each owning a bounded piece of business capability and (ideally) its own datastore. **Pros:** independent scaling (scale only the hot service), independent deployment (teams ship on their own cadence), fault isolation (one service's crash doesn't take down unrelated services if well-isolated), tech stack freedom per service. **Cons:** massive operational complexity (service discovery, distributed tracing, network calls replace function calls — introducing latency + partial failure modes), distributed transactions are hard (§8.8 Saga/2PC), data consistency across service boundaries requires careful design, testing end-to-end flows is harder.

13.3 When to Choose Which (Interview Framing)

Default to a **monolith** for new products/small teams/unclear domain boundaries — you can always split out microservices later once boundaries stabilize ("monolith first" is a widely respected pattern, e.g. from Martin Fowler). Reach for microservices when: the org has multiple independent teams that need to deploy without blocking each other, different parts of the system have wildly different scaling needs (e.g. an image-processing service needing GPUs vs a simple CRUD service), or you need strong fault isolation between domains. Saying "microservices are just better/more modern" without this context is a red flag in interviews.

13.4 Service Discovery

In a dynamic environment (auto-scaling, containers with ephemeral IPs), services need a way to find each other's current network location.

- **Client-side discovery** — client queries a registry (e.g. Netflix Eureka, Consul, Zookeeper) directly and load-balances between returned instances itself.
- **Server-side discovery** — client calls a well-known LB/router (e.g. Kubernetes Service, AWS ALB), which itself queries the registry and forwards the request — client stays simple, discovery logic centralized.

13.5 API Gateway

A single entry point that sits in front of all microservices, handling cross-cutting concerns so individual services don't each reimplement them: routing, authentication, rate limiting, request/response transformation, aggregation (combining multiple backend calls into one client-facing response), TLS termination, and API versioning.

13.6 Service Mesh & Sidecar Pattern

A **service mesh** (e.g. Istio, Linkerd) handles service-to-service concerns (mTLS, retries, timeouts, circuit breaking, load balancing, observability) via a **sidecar** proxy (e.g. Envoy) deployed alongside each service instance, intercepting all its network traffic — so this logic lives in infrastructure, not duplicated in every service's application code (in every language/framework used across the org).

13.7 Distributed Transactions Across Services (recap + expansion of §8.8)

- **2PC** — strongly consistent but blocking; rarely used across microservices in practice because it requires all participants to be available and hold locks during the multi-step protocol (a crashed participant can block the whole transaction).
- **Saga Pattern** — sequence of local transactions with compensating actions; two coordination styles:
  - **Choreography** — each service publishes an event after its local transaction; the next service reacts to that event. No central coordinator (fully decentralized), but can be hard to trace/debug the overall flow across many services.

- **Orchestration** — a central orchestrator explicitly tells each service what to do next and handles failure/compensation logic. Easier to understand/debug the overall flow (one place to look), but the orchestrator itself becomes a critical component.

13.8 Interview Angle

When asked to break a monolith into microservices (or design a system as microservices from scratch), justify the service boundaries using **Domain-Driven Design** thinking (bounded contexts — e.g. Order Service, Payment Service, Inventory Service, Notification Service — each owns its own data and a clear API contract), then immediately address: how do they discover each other (13.4), what enforces cross-cutting concerns (13.5 gateway, 13.6 mesh), and how do multi-service operations stay consistent (13.7 Saga).

14. Consistent Hashing

14.1 The Problem It Solves

With simple `hash(key) % N` sharding, changing the number of nodes N (adding/removing a server) reshuffles **almost every key's assigned node** (because the modulus changes for nearly every key) — meaning a huge, disruptive data migration just to add one more cache/DB node. Consistent hashing minimizes this: adding/removing one node only remaps **~1/N of the keys** on average.

14.2 How It Works

1. Imagine a **hash ring**: hash values from `0` to `2^32 - 1` (or any fixed range), arranged in a circle.
2. Hash each server (node) onto the ring (e.g. `hash(server_id)`).
3. Hash each key onto the same ring.
4. A key is owned by the **first server found walking clockwise** from the key's position on the ring.
5. **Adding a node**: it takes over a contiguous arc of the ring (only keys that fall between the new node and its counter-clockwise neighbor move) — everything else is untouched.

6. **Removing a node**: its keys simply move to the next server clockwise — again, only that arc's keys are affected, not the whole ring.

14.3 Virtual Nodes (the practical refinement everyone actually uses)

With only one point per physical server on the ring, load can be very uneven (some servers own a much bigger arc than others, just by hash luck, and the arcs are highly sensitive to exactly where nodes land). Solution: hash each physical server to **many points** on the ring (e.g. 100-200 virtual nodes per physical server, e.g. `hash(server_id + "-0")`, `hash(server_id + "-1")`, ...). This spreads each physical server's ownership across many small arcs distributed around the ring, giving much more even load distribution and making rebalancing after a node join/leave smoother and fairer across the remaining nodes.

14.4 Where It's Used

- Distributed caches (Memcached client-side sharding, Redis Cluster's hash slots are a related but distinct idea using a fixed 16384 hash slot space).
- Distributed databases: Cassandra, DynamoDB (this is literally what "Dynamo" — the paper Cassandra/DynamoDB are based on — introduced consistent hashing for).
- CDN request routing (routing a URL/object consistently to the same edge/origin shard).
- Load balancers doing session affinity without a shared session store.

14.5 Interview Angle

Whenever you propose sharding (§8.5) or a distributed cache cluster, proactively mention: *"I'll use consistent hashing with virtual nodes (~100-200 per physical node) so that adding/removing a node only reshuffles a small fraction of keys, instead of a naive modulus-based scheme that would reshuffle almost everything."* This is one of the highest-value one-liners in system design interviews — it directly answers "how do you handle scaling the cache/shard cluster up or down?" before it's even asked.

---

15. Distributed Systems Theory

15.1 Consensus (Why It's Needed)

Multiple nodes need to agree on a single value/decision (who is the leader, what is the next entry in a replicated log, was this transaction committed) **even in the presence of node failures and message delays/loss.** This is provably impossible to solve perfectly in theory (FLP impossibility result — no fully asynchronous consensus algorithm can guarantee both safety and liveness if even one node can fail), but practical algorithms make reasonable real-world assumptions (partial synchrony, timeouts) and work well in practice.

15.2 Paxos (Conceptual)

The original widely-cited consensus protocol; notoriously difficult to understand and implement correctly (hence why Raft was created as a more understandable alternative). Roughly: proposers propose values with a mostly-increasing proposal number, acceptors promise not to accept older-numbered proposals and vote on the highest they've seen, and a value is chosen once a majority (quorum) of acceptors accept it. Interview-level takeaway: **know that it exists, it's the theoretical foundation, and it's hard** — you won't be asked to derive it, but you may be asked "why does this need a quorum?" (because any two majority quorums out of N nodes must overlap by at least one node, which is what guarantees agreement is preserved even if some nodes fail/partition).

15.3 Raft (What You Should Actually Know)

Raft was designed to be *understandable* while providing equivalent guarantees to Paxos. Key ideas:

- **Leader election** — nodes are in one of three states: Follower, Candidate, Leader. If a follower doesn't hear from a leader within a randomized election timeout, it becomes a candidate, increments a "term" number, and requests votes; whoever gets a majority becomes leader for that term. Randomized timeouts reduce the chance of split votes. Terms act as a logical clock, letting nodes detect and ignore messages from an outdated ("stale") leader.

- **Log replication** — the leader accepts writes, appends them to its local log, and replicates to followers; an entry is considered **committed** once a majority of nodes have it in their logs — only then is it applied to the state machine and acknowledged to the client.
- Used in practice by: etcd, Consul, CockroachDB, TiKV, and countless other systems needing strongly-consistent replicated state (this is the mechanism behind "leader election" and "distributed configuration store" building blocks referenced elsewhere in this guide).

15.4 Leader Election (Practical Framing)

Many systems need exactly one active coordinator/primary at a time (a DB primary, a job scheduler, a lock holder). Rather than reimplementing Raft/Paxos yourself, real systems typically **delegate** this to a battle-tested coordination service: **ZooKeeper**, **etcd**, or **Consul** — all of which are themselves built on a Raft/Paxos-family consensus algorithm internally and expose primitives (ephemeral nodes, watches, distributed locks) that application code uses to implement leader election. **Interview answer:** "I'd use etcd/ZooKeeper for leader election rather than hand-rolling consensus" is the correct, senior-level answer for almost every practical scenario.

15.5 Gossip Protocol

Nodes periodically exchange state information with a few random peers (not everyone), and information **epidemically spreads** through the cluster over multiple rounds (similar to how a rumor spreads through a social network) — reaching all nodes in O(log N) rounds with high probability, without needing a central coordinator or every node talking to every other node (which wouldn't scale). Used for: cluster membership and failure detection (Cassandra, Consul use gossip to know which nodes are alive), because it's highly resilient (no single point of failure, tolerates significant churn/partial failure) even though it gives only **eventually consistent** view of cluster state (not instant).

15.6 Vector Clocks

A mechanism to detect whether two events in a distributed system are **causally ordered** (one happened-before the other) or **concurrent** (neither caused the other — a potential conflict). Each node keeps a vector of counters (one per node); every event

increments the node's own counter and the vector is attached to messages/writes. Comparing two vectors: if one is component-wise ≥ the other, it happened after; if neither dominates, they're concurrent (a genuine conflict needing resolution, e.g. via last-write-wins or application-level merge — this is the mechanism classic Dynamo used to detect conflicting concurrent writes to the same key across replicas).

15.7 Quorum Reads/Writes (recap + formalization from §8.4)

With replication factor **N**, define write quorum **W** and read quorum **R**. If `W + R > N`, every read is guaranteed to overlap with the most recent write's replica set (strong consistency guarantee), because any two sets of size W and R out of N must share at least one node. Common tunings: `W=N, R=1` (fast reads, slow/less available writes), `W=1, R=N` (fast writes, slow/less available reads), `W=R=(N/2)+1` (balanced quorum — the most common default, e.g. Cassandra's `QUORUM` consistency level).

15.8 CRDTs (Conflict-Free Replicated Data Types)

Data structures specifically designed so that concurrent updates from different replicas can **always be merged automatically** into a consistent result, without needing coordination/locking, and without losing updates — because the merge function is mathematically commutative, associative, and idempotent. Examples: a G-Counter (grow-only counter, merge = take max per replica then sum), an OR-Set (add/remove set with tombstones to handle concurrent add/remove correctly). Used in: collaborative editing tools, distributed counters/likes at massive scale, offline-first mobile apps that sync later. Interview takeaway: mention CRDTs when asked "how do you handle concurrent updates to the same counter/set across regions without a central coordinator?"

15.9 Distributed Locks

Needed when multiple nodes/processes must ensure only one of them does something at a time (e.g. only one worker processes a given job). Common implementation: **Redlock** (using Redis) or a lock built on **etcd/ZooKeeper** (using their consensus-backed primitives — generally the safer choice, since Redlock's safety under certain

failure/clock-skew scenarios has been debated in the community). Always pair a distributed lock with a **lease/ TTL** (auto-expire if the lock holder crashes without releasing) and be aware of the "stop-the-world GC pause" problem: a lock holder that pauses past its lease can have its lock stolen by another node, then wake up and still think it holds the lock — mitigated with **fencing tokens** (a monotonically increasing number issued with each lock acquisition; downstream resources reject any request with an old, since-superseded token).

15.10 Interview Angle

Distributed systems theory questions are usually not asked in isolation — they surface as follow-ups to your HLD ("how do you elect a new primary if it fails?" → 15.4; "how do two regions reconcile conflicting writes?" → 15.6/15.8; "how do you guarantee only one worker picks up this job?" → 15.9). Knowing the *names and one-sentence mechanism* of each tool here, and which practical system to reach for (etcd/ZooKeeper for coordination, not hand-rolled Paxos) is the actual tested skill.

---

16. Security in System Design

16.1 Authentication vs Authorization

- **Authentication (AuthN)** — verifying *who* the caller is (login, token validation).
- **Authorization (AuthZ)** — verifying *what* the authenticated caller is allowed to do (permissions/roles/scopes).

16.2 Common AuthN Mechanisms

- **Session-based** — server creates a session on login, stores it server-side (DB/Redis), gives the client an opaque session ID cookie. Server must look up the session on every request (extra DB/cache hit), but sessions can be revoked instantly (delete from store).
- **Token-based (JWT)** — server issues a signed token (JSON Web Token) containing claims (user ID, roles, expiry); client sends it on every request; server verifies the **signature** (no DB lookup needed — stateless!). Trade-off: **can't be revoked before expiry** without extra infrastructure (a blocklist/short expiry +

refresh tokens are common mitigations). Great fit for stateless, horizontally-scaled services (§4.3).
- **OAuth2 / OIDC** — delegated authorization framework letting a user grant a third-party app limited access without sharing their password (e.g. "Sign in with Google"). OIDC layers identity (who the user is) on top of OAuth2 (which is fundamentally about authorization/access delegation, not identity).

16.3 Encryption

- **In transit** — TLS/HTTPS for all client-server and (ideally) service-to-service traffic (mTLS in a service mesh, §13.6) so data can't be sniffed on the network.
- **At rest** — encrypt sensitive data in the database/disk/object storage (transparent DB encryption, or application-level field encryption for especially sensitive fields like SSNs/card numbers) so a storage/backup leak doesn't expose plaintext data.
- **Key management** — use a dedicated secrets/key management service (AWS KMS, HashiCorp Vault, Azure Key Vault) rather than hardcoding keys in config/code — supports key rotation and audit logging of key access.

16.4 Rate Limiting & DDoS Mitigation

Beyond the per-user API rate limiting covered in §12.4, system-wide DDoS protection typically involves: a CDN/edge layer absorbing and filtering volumetric attacks before they reach origin servers, IP reputation/blocklisting, challenge mechanisms (CAPTCHA, proof-of- work) for suspicious traffic, and auto-scaling to absorb legitimate traffic spikes while rate-limiting abuse.

16.5 OWASP-Relevant Design Considerations

Even at the architecture level (not just code level), design choices affect security posture: **input validation at every trust boundary** (never trust client-supplied data, including internal service-to-service calls in a zero-trust architecture), **least privilege** (each service/DB user has only the permissions it needs — a compromised read-only reporting service shouldn't be able to write to the orders table), **defense in depth** (WAF + input validation + parameterized queries, not relying on just one layer to prevent injection), and **audit logging** of sensitive actions (who accessed/changed what, when) for detection and compliance.

16.6 Interview Angle

Security rarely dominates a system design interview but showing awareness scores well: mention AuthN/AuthZ approach for user-facing systems, flag PII/sensitive data needing encryption at rest, and mention rate limiting at the edge for public APIs. Don't over-invest time here relative to the core scaling/data questions unless the interviewer signals security is the focus (e.g. explicitly a "design a secure X" prompt).

---

17. Observability

17.1 The Three Pillars

- **Logging** — discrete, timestamped records of events ("what happened"). Best for detailed forensic debugging of a specific request/error. At scale, needs centralized aggregation (see HLD case study §43 Log Aggregation) since individual server disks aren't practically searchable.
- **Metrics** — numeric, aggregatable time-series measurements (request rate, error rate, latency percentiles, CPU/memory usage). Cheap to store/query at scale (pre-aggregated), ideal for dashboards and alerting thresholds. Tools: Prometheus, Datadog, CloudWatch.
- **Tracing (Distributed Tracing)** — follows a single request's journey across multiple services, recording timing for each hop ("span"), stitched together by a shared **trace ID** propagated through headers across service calls. Essential in microservices to answer "which of these 8 services in the call chain is causing the slowdown?" Tools: Jaeger, Zipkin, OpenTelemetry (the emerging standard instrumentation API/SDK).

17.2 The RED and USE Methods (Quick Frameworks)

- **RED** (for request-driven services): **R**ate (requests/sec), **E**rrors (error rate/count), **D**uration (latency distribution, especially p50/p95/p99).
- **USE** (for resources): **U**tilization (% busy), **S**aturation (queue depth/backlog), **E**rrors (error count) — useful for infra resources (CPU, disk, network).

17.3 Alerting

Alert on **symptoms that affect users** (elevated error rate, p99 latency breach, SLO burn rate) rather than every possible internal cause — this avoids alert fatigue. A common best practice is **SLO-based alerting** (alert when you're burning your error budget faster than sustainable, e.g. "we'll exhaust this month's 99.9% SLO error budget within 4 hours at the current error rate" — this catches real problems early without noisy, low-value alerts).

17.4 Health Checks (recap, tied to §5.3)

- **Liveness probe** — "is the process alive/not deadlocked?" A failing liveness check usually triggers a restart of that instance.
- **Readiness probe** — "is the instance ready to serve traffic right now?" (e.g. still warming up a cache, or a downstream dependency is temporarily unavailable). A failing readiness check removes the instance from the load balancer's rotation *without* restarting it.

17.5 Interview Angle

Mentioning "I'd add metrics (RED method) on this service, structured logs correlated by request ID, and distributed tracing across the service calls in the critical path, with SLO-based alerting on p99 latency and error rate" is a strong, concise way to show production-readiness thinking without derailing the interview into a full observability deep dive (save depth for if the interviewer asks).

---

18. Storage & Blob Systems

18.1 Object Storage (S3-style)

Stores arbitrary binary blobs (images, videos, backups, logs) as objects with a key, value (the bytes), and metadata — accessed via a simple HTTP API (`PUT`/`GET`/`DELETE` by key), **not** a filesystem or block device. Key properties: virtually unlimited scale, very high durability (data replicated across multiple devices/AZs, e.g. "11 nines" of durability commonly advertised), cheaper than block/file storage for large blobs, but higher

latency per-object and no partial in-place edits (must rewrite the whole object, or use multipart upload for large objects). This is *the* default choice for storing user uploads, media, backups, and static assets in almost every modern system design answer.

18.2 Object Storage Internals (What's Actually Happening)

Under the hood, systems like S3 typically: split large objects into chunks distributed across many disks/nodes for parallel throughput and fault tolerance, use **erasure coding** (more storage-efficient than full replication — e.g. rather than 3 full copies (3x storage), split data into k data shards + m parity shards across k+m nodes, tolerating up to m node failures while using much less than 3x the storage) or replication depending on the durability/cost trade-off desired, and maintain a separate **metadata service** (mapping object key → physical chunk locations) that must itself be highly available.

18.3 CDN Integration (recap of §9.7)

Object storage is almost always paired with a CDN in front of it for anything served to end users (serve the object once from origin storage, cache at edge for all subsequent requests globally) — this is the standard pattern for image/video/static asset delivery referenced throughout the HLD case studies later in this guide.

18.4 File Chunking (for large files/videos)

Large files (videos, big uploads) are typically split into fixed-size chunks (e.g. 5-10MB) for: **parallel upload/download** (fetch multiple chunks concurrently), **resumability** (a failed transfer only needs to retry the failed chunk, not the whole file — critical for mobile clients on flaky networks), **deduplication** (identical chunks across different files/versions can be stored once, referenced multiple times — this is the core trick behind Dropbox-style file sync efficiency, see HLD case study §35), and **adaptive streaming** (video specifically is chunked into short segments at multiple quality levels, so a video player can request different quality chunks based on current network conditions — this is how adaptive bitrate streaming/HLS/DASH work, see HLD case study §29).

18.5 Block Storage vs File Storage vs Object Storage (Quick Comparison)

	Block Storage	File Storage	Object Storage
Example	AWS EBS	AWS EFS, NFS	AWS S3
Access unit	Raw disk blocks	Files in a directory hierarchy	Objects by key (flat namespace)
Mutability	In-place edits, low latency	In-place edits	Usually write-whole-object (no partial edit)
Best for	Databases, boot volumes (needs low-latency random I/O)	Shared file access across multiple servers (legacy apps expecting a filesystem)	Media, backups, logs, static assets, huge scale
Scalability	Limited (attached to one instance typically)	Medium	Effectively unlimited

18.6 Interview Angle

For any HLD involving user-uploaded media (photos, videos, documents), the correct default answer is: "store the actual bytes in object storage (S3-style), store only the metadata (owner, filename, size, S3 key/URL, timestamps) in the primary database, and serve reads through a CDN" — **never** store binary blobs directly in a relational database (bloats the DB, hurts backup/replication performance, wastes an expensive resource on cheap storage needs).

Part 3 — The Interview Framework

19. How To Solve Any HLD Question in 45 Minutes

This is the single most reusable asset in this guide. Internalize this framework and you can walk into literally any "design X" prompt — including ones you've never seen — with a repeatable process instead of blind panic. Every case study in Part 5 follows this exact structure.

19.1 The Timeline (45-Minute Interview)

Time	Phase	Goal
0-5 min	Requirements gathering	Scope FRs/NFRs, agree what's in/out of scope
5-10 min	Capacity estimation	QPS, storage, bandwidth — sanity check scale
10-15 min	API design	Define the 3-5 core endpoints/contracts
15-25 min	High-level design	Draw components, data flow, pick DB/cache/queue
25-40 min	Deep dive	Interviewer-driven — go deep on 1-3 hard sub-problems
40-45 min	Wrap up	Bottlenecks, trade-offs, what you'd do differently at 10x scale

Critical mindset: this is a *conversation*, not a monologue. Check in with the interviewer at each phase transition ("Does this scope sound right? Should I move on to estimation?"). Interviewers actively want to steer you toward the areas they care about — let them.

19.2 Phase 1 — Requirements Gathering (see also Part 1 §2)

Ask targeted questions, don't guess:

- Who are the actors/users? What are the 3-5 core use cases?
- What's explicitly out of scope for this discussion?
- Scale: DAU/MAU, read:write ratio, data growth rate?
- Any hard constraints: latency SLA, consistency requirements, compliance?

Output: a short, explicit list like: *"FRs: [A, B, C]. NFRs: handle 10M DAU, read-heavy 100:1, p99 latency <200ms for reads, eventual consistency acceptable except for [specific operation]."* Write this down (on the whiteboard/doc) — you'll refer back to it constantly.

19.3 Phase 2 — Capacity Estimation (see also Part 1 §3)

Run through QPS → storage → bandwidth → "does this fit on one box or do we need to distribute?" Keep it fast (5 minutes) and always **close the loop** — state the architectural conclusion the numbers imply.

19.4 Phase 3 — API Design

Define concrete endpoints for your core FRs only. For each: method, path, request body, response, and any special semantics (idempotency, pagination, rate limits). Example:

```
POST /urls              { "long_url": "https://..." } -> { "short_url":
"https://sho.rt/abc123" }
GET  /{shortCode}        -> 302 redirect to long_url
```

19.5 Phase 4 — High-Level Design

Draw (describe in words/ASCII) the components and how a request flows through them, typically: Client → CDN/LB → API Gateway → App Servers → Cache → Database (+ any async path: App Servers → Queue → Workers → separate stores). For each component, state *why* it's there and what it's chosen to be (e.g. "Redis in front of Postgres for the redirect read path, because reads are 100x writes and redirects need to be fast"). This is where you apply the Part 2 building blocks — don't introduce a component (queue, cache, sharding) without a stated reason tied back to your requirements/estimation.

19.6 Phase 5 — Deep Dive

The interviewer will usually steer here ("how exactly do you generate a unique short code at this scale without collisions?", "how do you handle a celebrity user's feed fan-out?"). This is where seniority is actually assessed — anyone can draw boxes, but reasoning through one hard sub-problem in real depth (with actual algorithms/data structures/numbers) separates strong performances from weak ones. Common deep-dive topics per system are called out explicitly in each Part 5 case study.

19.7 Phase 6 — Bottlenecks, Trade-offs, and Wrap-Up

Proactively (even if not asked): *"The current bottleneck as we scale further would be [X]; I'd address it by [Y]. If I had more time I'd also consider [Z — e.g. multi-region, a different consistency model for feature W]."* This demonstrates you know the design isn't "finished" — it's a snapshot appropriate for the stated scale, with known next steps.

19.8 The Reusable Checklist (say it, don't just think it)

1. ✅ Scoped FRs and NFRs explicitly, confirmed with interviewer.
2. ✅ Estimated QPS, storage, bandwidth — stated the "fits on one box / needs N servers / needs sharding" conclusion.
3. ✅ Defined 3-5 core API endpoints.
4. ✅ Drew high-level components with justification for each technology choice.
5. ✅ Identified the read path and write path separately (they often need different optimizations).
6. ✅ Called out consistency requirements per data type (not uniformly one model).
7. ✅ Went deep on at least one hard sub-problem with real detail (algorithm, data structure, numbers).
8. ✅ Named bottlenecks and next steps at 10x scale.

19.9 Common Pitfalls That Lose Points

- **Jumping straight to a solution** before requirements/estimation ("we'll use Kafka and Cassandra" in the first 2 minutes, with zero justification).
- **Silence** — not thinking out loud. Interviewers can't score what they can't hear.

- **Over-engineering** — adding sharding/Kafka/microservices for a system that estimation shows fits on a single server + a cache.
- **Under-engineering** — proposing a single Postgres instance for a system that estimation clearly shows needs to handle 500K writes/sec.
- **Ignoring interviewer hints** — if they say "let's focus on the notification delivery part," don't keep talking about the database schema.
- **Not defending trade-offs** — every choice should be paired with "...because..." and "...the trade-off is...". A design with zero acknowledged trade-offs is a red flag (it means you don't see the downsides of your own choices).
- **Running out of time on fundamentals** — spending 20 minutes on requirements/estimation leaves no time for the deep dive, which is usually where the most points are earned.

Part 4 — LLD Framework

20. OOP Fundamentals & SOLID Principles

LLD interviews test whether you can translate a real-world problem into a clean, extensible object model — not whether you can write a working program (any working program that's a tangled mess of static methods and god-classes will score poorly even if it "works").

20.1 Core OOP Concepts (Recap)

- **Encapsulation** — bundle data with the methods that operate on it, hide internal state behind a controlled interface (private fields + public methods) so invariants can't be violated from outside.
- **Abstraction** — expose only what's necessary through interfaces/abstract classes, hiding implementation details — callers depend on *what* an object does, not *how*.
- **Inheritance** — a subclass reuses/extends behavior from a parent. Use sparingly in interviews — prefer composition (see 20.3) unless there's a genuine "is-a" relationship and shared behavior that truly belongs in a common parent.
- **Polymorphism** — different objects respond to the same method call according to their own type (via method overriding or interface implementation) — this is the

mechanism that lets you add new types (e.g. a new vehicle type in a parking lot) without modifying existing code (ties directly into the Open/Closed Principle below).

20.2 SOLID Principles (The Actual Thing Being Tested)

- **S — Single Responsibility Principle**: a class should have one, and only one, reason to change. If a `ParkingLot` class both manages spot allocation *and* calculates pricing *and* sends notifications, split it into `SpotAllocator`, `PricingStrategy`, and `NotificationService`.
- **O — Open/Closed Principle**: classes should be open for extension, closed for modification. Adding a new vehicle type or payment method shouldn't require editing existing `if/else`/`switch` chains — instead, use polymorphism/interfaces so new types plug in via a new class, not by editing old ones.
- **L — Liskov Substitution Principle**: subtypes must be substitutable for their base types without breaking correctness. A classic violation: `Square extends Rectangle` where setting width/height independently breaks the invariant callers expect from a `Rectangle`.
- **I — Interface Segregation Principle**: many small, focused interfaces are better than one large, general-purpose interface — don't force a class to implement methods it doesn't need (e.g. don't make a `Robot` implement a `Worker` interface that has an `eat()` method it can't meaningfully support).
- **D — Dependency Inversion Principle**: depend on abstractions (interfaces), not concrete implementations. A `NotificationService` should depend on a `MessageSender` interface, not directly on a concrete `EmailSender` class — so you can swap in `SmsSender` or `PushNotificationSender` later without changing `NotificationService`.

20.3 Composition Over Inheritance

Prefer "has-a" (composition — a class holds a reference to another class it delegates to) over "is-a" (inheritance) in most LLD designs. Composition is more flexible: behavior can be swapped at runtime (e.g. inject a different `PricingStrategy` object), avoids the fragile base class problem (changes to a parent class unexpectedly breaking many subclasses), and avoids deep, hard-to-reason-about inheritance

hierarchies. Rule of thumb: use inheritance only for genuine "is-a" relationships with substantial shared behavior; otherwise, compose.

20.4 The LLD Interview Process (Step by Step)

1. **Clarify requirements/scope** — same discipline as HLD; ask what operations must be supported ("can a parking lot have multiple entry/exit points? multiple vehicle types? different pricing per vehicle type?").
2. **Identify the nouns (entities) and verbs (actions/behaviors)** from the problem statement — nouns often become classes, verbs often become methods.
3. **Identify relationships** between entities (is-a → inheritance/interface; has-a → composition; uses-a → dependency).
4. **Sketch the class diagram** (in text: class names, key fields, key methods, and relationships) before writing code.
5. **Identify which design patterns naturally fit** (see §21) — don't force a pattern that doesn't fit just to show it off; interviewers notice pattern-shoehorning.
6. **Write the code** — focus on the class structure, interfaces, and method signatures being correct and clean; you don't always need every method body fully implemented, but the core/tricky logic (e.g. spot allocation algorithm, LRU eviction) should be real, working code.
7. **Discuss extensibility** — "if we needed to add feature X, what would change?" A good design answers this with "just add a new class implementing interface Y" — not "we'd need to modify classes A, B, and C."

---

21. Design Patterns for Interviews

A curated set of the patterns that come up **repeatedly** in LLD interviews, each with the problem it solves, the core idea, and a compact Java example.

21.1 Singleton

Problem: ensure exactly one instance of a class exists (e.g. a configuration manager, a connection pool, a logger).

```
public class ConfigManager {
    private static volatile ConfigManager instance;
```

```
    private final Map<String, String> settings = new HashMap<>();

    private ConfigManager() { /* load config */ }

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null) {
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }

    public String get(String key) { return settings.get(key); }
}
```

Interview note: double-checked locking (above) is the classic thread-safe lazy-init version; an even simpler thread-safe alternative is an `enum` singleton or eager initialization via a `static final` field. Be ready to discuss why singletons can hurt testability (hidden global state, hard to mock) — a legitimate critique interviewers may probe.

21.2 Factory Method

Problem: create objects without exposing the exact instantiation logic to the caller, letting subclasses/config decide which concrete type to create.

```
interface Notification { void send(String message); }
class EmailNotification implements Notification {
    public void send(String message) { System.out.println("Email: " + message); }
}
class SmsNotification implements Notification {
    public void send(String message) { System.out.println("SMS: " + message); }
}
class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "EMAIL" -> new EmailNotification();
            case "SMS" -> new SmsNotification();
            default -> throw new IllegalArgumentException("Unknown type: " + type);
        };
    }
}
```

Used in: creating the right vehicle type in a parking lot, the right payment processor in a checkout flow, the right notification channel in a notification system (§26).

21.3 Builder

Problem: construct complex objects with many optional fields step by step, avoiding telescoping constructors (`new Order(a, b, null, null, c, null, d)`).

```
public class Order {
    private final String id;
    private final String customerId;
    private final List<String> items;
    private final String couponCode; // optional

    private Order(Builder b) {
        this.id = b.id; this.customerId = b.customerId;
        this.items = b.items; this.couponCode = b.couponCode;
    }

    public static class Builder {
        private String id, customerId, couponCode;
        private List<String> items = new ArrayList<>();
        public Builder id(String id) { this.id = id; return this; }
        public Builder customerId(String c) { this.customerId = c; return this; }
        public Builder addItem(String item) { this.items.add(item); return this; }
        public Builder couponCode(String c) { this.couponCode = c; return this; }
        public Order build() { return new Order(this); }
    }
}
// Usage: new Order.Builder().id("1").customerId("u1").addItem("book").build();
```

21.4 Observer

Problem: when one object's state changes, notify a dynamic list of dependent objects without tightly coupling them (classic pub-sub within a single process).

```
interface Observer { void update(String event); }
class StockTicker {
    private final List<Observer> observers = new ArrayList<>();
    public void subscribe(Observer o) { observers.add(o); }
    public void priceChanged(String symbol, double price) {
        for (Observer o : observers) o.update(symbol + " -> " + price);
    }
}
```

Used in: notification systems (§26), news feed fan-out (§27), any "watch for changes and react" requirement.

21.5 Strategy

Problem: define a family of interchangeable algorithms and select one at runtime, avoiding conditional logic scattered across the codebase.

```
interface PricingStrategy { double calculate(long durationMinutes); }
class HourlyPricing implements PricingStrategy {
    public double calculate(long m) { return Math.ceil(m / 60.0) * 2.0; }
}
class FlatRatePricing implements PricingStrategy {
    public double calculate(long m) { return 10.0; }
}
class ParkingTicket {
    private final PricingStrategy strategy;
    ParkingTicket(PricingStrategy strategy) { this.strategy = strategy; }
    double getFee(long durationMinutes) { return
strategy.calculate(durationMinutes); }
}
```

Used in: parking lot pricing (§44), rate limiter algorithm selection (§48), payment method selection.

21.6 Decorator

Problem: add responsibilities to an object dynamically without subclassing every combination (avoids combinatorial subclass explosion, e.g. `SmallCoffeeWithMilk`, `LargeCoffeeWithMilkAndSugar`, ...).

```
interface Coffee { double cost(); String description(); }
class BasicCoffee implements Coffee {
    public double cost() { return 2.0; }
    public String description() { return "Coffee"; }
}
abstract class CoffeeDecorator implements Coffee {
    protected final Coffee wrapped;
    CoffeeDecorator(Coffee c) { this.wrapped = c; }
}
class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee c) { super(c); }
    public double cost() { return wrapped.cost() + 0.5; }
    public String description() { return wrapped.description() + " + Milk"; }
}
// Usage: Coffee order = new MilkDecorator(new BasicCoffee());
```

21.7 Adapter

Problem: make an existing class's interface compatible with what the client code expects, without modifying the existing class (e.g. integrating a third-party payment SDK with an incompatible interface).

```
interface PaymentProcessor { void charge(double amount); }
class ThirdPartyGateway { void makePayment(int cents) { /* ... */ } }
class ThirdPartyAdapter implements PaymentProcessor {
    private final ThirdPartyGateway gateway = new ThirdPartyGateway();
    public void charge(double amount) { gateway.makePayment((int) (amount * 100));
}
}
```

21.8 Facade

Problem: provide a simple, unified interface to a complex subsystem of many classes (e.g. a `CheckoutFacade` that internally coordinates `InventoryService`, `PaymentService`, and `NotificationService` behind one `checkout()` call).

```
class CheckoutFacade {
    private final InventoryService inventory = new InventoryService();
    private final PaymentService payment = new PaymentService();
    private final NotificationService notifier = new NotificationService();

    public void checkout(String userId, String itemId, double amount) {
        inventory.reserve(itemId);
        payment.charge(userId, amount);
        notifier.notifyOrderPlaced(userId);
    }
}
```

21.9 Command

Problem: encapsulate a request/action as an object, enabling queuing, undo/redo, and logging of operations (decouples the invoker from the receiver executing the action).

```
interface Command { void execute(); void undo(); }
class LightOnCommand implements Command {
    private final Light light;
    LightOnCommand(Light light) { this.light = light; }
```

```
    public void execute() { light.on(); }
    public void undo() { light.off(); }
}
```

Used in: undo/redo systems, job schedulers (§40) queuing units of work, transactional outbox patterns.

21.10 State

Problem: an object's behavior changes based on its internal state, avoiding a giant conditional (`if status == X ... else if status == Y ...`) scattered across many methods.

```
interface OrderState {
    void next(OrderContext ctx);
}
class PlacedState implements OrderState {
    public void next(OrderContext ctx) { ctx.setState(new ShippedState()); }
}
class ShippedState implements OrderState {
    public void next(OrderContext ctx) { ctx.setState(new DeliveredState()); }
}
class DeliveredState implements OrderState {
    public void next(OrderContext ctx) { /* terminal state */ }
}
class OrderContext {
    private OrderState state = new PlacedState();
    void setState(OrderState s) { this.state = s; }
    void advance() { state.next(this); }
}
```

Used in: order lifecycle (placed→shipped→delivered), elevator state (§45, idle/moving/door-open), traffic light systems, ticket booking seat state (§31/§50).

21.11 Quick Pattern Selection Table

Need	Pattern
Exactly one instance	Singleton
Create objects without exposing `new SomeClass()` logic	Factory Method
Construct complex object step-by-step with optional fields	Builder
Notify many dependents on a state change	Observer

Need	Pattern
Swap an algorithm/behavior at runtime	Strategy
Add behavior dynamically without subclass explosion	Decorator
Make incompatible interfaces work together	Adapter
Simplify a complex subsystem behind one interface	Facade
Encapsulate an action for queuing/undo/logging	Command
Behavior changes based on internal state/lifecycle	State

Part 5 — HLD Case Studies

22. Design a URL Shortener

Problem

Design a service (like bit.ly/TinyURL) that converts long URLs into short aliases and redirects users from the short URL back to the original.

Requirements

Functional: (1) Given a long URL, generate a unique short URL. (2) Given a short URL, redirect to the original long URL. (3) (Optional) Support custom aliases. (4) (Optional) Support link expiration. (5) (Optional) Basic click analytics. **Non-Functional:** High availability (redirects must always work). Low latency redirects (<100ms p99). Short URLs must be as short as possible while remaining unique. Read-heavy (reads ≫ writes — most links get many more clicks than the one creation event).

Estimation

Assume 100M new URLs/month created, 100:1 read:write ratio.

```
Write QPS avg  = 100,000,000 / (30*86400) ≈ 38.6/sec  -> peak (x3) ≈ 116/sec
Read QPS avg   = 38.6 × 100 ≈ 3,860/sec               -> peak (x3) ≈ 11,580/sec
```

```
Storage/record ≈ 500 bytes (short code + long URL + metadata)
Storage/month  = 100M × 500B = 50 GB/month -> 5 years ≈ 3 TB (with 3x replication)
```

Conclusion: read-heavy by two orders of magnitude → caching the redirect path is the single highest-leverage design decision. Storage easily fits a single well-indexed DB for years — no sharding needed at this scale (mention sharding as a future step if scale grows 10x further).

API Design

```
POST /api/v1/urls        { "long_url": "...", "custom_alias": "optional",
"ttl_days": "optional" }
                          -> 201 { "short_url": "https://sho.rt/abc123" }
GET  /{shortCode}         -> 302 Found, Location: <long_url>
GET  /api/v1/urls/{shortCode}/stats  -> { "clicks": 4213, "created_at": "..." }
```

`POST` uses an idempotency key if the client wants safe retries. `GET /{shortCode}` is the hot path — must be blazing fast and heavily cached.

High-Level Design

```
Client -> CDN/Edge (optional, cache redirects) -> Load Balancer -> App Servers
(stateless)
                                                                    |         |
                                                          Cache (Redis)
Database (Postgres)
                                                         (shortcode->longURL)
(source of truth)
```

- **Write path:** client submits long URL → app server generates a unique short code (see Deep Dive) → writes `(short_code, long_url, created_at, expiry)` to the DB → optionally pre-warms the cache.
- **Read path:** client hits `/{shortCode}` → app server checks Redis cache first (cache-aside, §9.2) → cache hit: redirect immediately (<10ms). Cache miss: read from DB, populate cache, then redirect. Click analytics recorded **asynchronously** (fire an event to a queue, don't block the redirect on writing an analytics row — the redirect is the latency-critical path, analytics is not).

Database Schema

```
CREATE TABLE urls (
    short_code   VARCHAR(10) PRIMARY KEY,
    long_url     TEXT NOT NULL,
    created_at   TIMESTAMP NOT NULL DEFAULT now(),
    expires_at   TIMESTAMP NULL,
    created_by   BIGINT NULL,       -- nullable if anonymous
    click_count  BIGINT DEFAULT 0   -- optionally updated async/batched, not
synchronously
);
CREATE INDEX idx_urls_created_by ON urls(created_by);
```

A key-value store (DynamoDB) would also work well here (simple key → value lookup, no joins needed) — a valid alternative to Postgres, worth mentioning as a trade-off.

Deep Dive: Generating Unique Short Codes

This is the classic deep-dive interviewers push on. Three approaches, in order of what a strong candidate proposes:

1. **Hash-based (MD5/SHA-256 of the long URL, take first 6-8 chars, base62-encode)** — simple, but **collisions are possible** (two different long URLs producing the same truncated hash, or the same long URL submitted twice needing the same/different code depending on requirements) — requires a collision-check-and-retry loop against the DB, adding latency and complexity under load.
2. **Counter + Base62 encoding (recommended)** — maintain a globally unique, monotonically increasing counter (e.g. via a dedicated auto-increment DB sequence, or a distributed ID generator, §37), then base62-encode the counter (`[a-zA-Z0-9]`, 62 characters) to get a short, guaranteed-unique code with no collision checks needed at all. `62^6 ≈ 56.8 billion` possible 6-character codes — comfortably covers years of growth at the estimated 100M/month rate before needing 7 characters. **Distributing the counter** (so a single counter service isn't a bottleneck/SPOF): use **range-based allocation** — each app server periodically requests a range of, say, 10,000 IDs from a central counter service, hands them out locally without a network call per request, and requests a new range when exhausted. Trade-off: some IDs are "wasted" if a server crashes with unused IDs in its range (acceptable — the ID space is huge).

3. **Pre-generated pool of random unique codes** — a background job continuously generates random base62 strings, checks uniqueness, and stores unused codes in a pool; the write path just pops one off the pool (fast, no runtime collision check). Adds operational complexity (a whole subsystem just for code generation) — usually overkill vs option 2, but worth mentioning as an alternative.

Recommended answer: counter + base62 with range-based distribution — no collisions by construction, no extra collision-check round trip on the hot write path, and scales horizontally since each server hands out IDs from its own pre-fetched range.

Deep Dive: Custom Aliases & Expiration

Custom aliases need a uniqueness check against the DB (unlike generated codes) — a simple `INSERT ... ON CONFLICT DO NOTHING` (Postgres) or a unique constraint returning a clear "alias taken" error covers this; no need for anything fancier. Expiration is handled by a `expires_at` column checked on read (return 404/410 if expired) plus a periodic background job (or TTL feature if using Redis/DynamoDB) to physically delete/archive expired rows, keeping the active dataset small and cache-friendly.

Bottlenecks & Scaling

- **Cache is the critical path** — if Redis goes down, every read falls through to the DB; mitigate with Redis replication (a replica set) and a circuit breaker that degrades gracefully (serve from DB directly, slower but not down).
- **Hot links (viral URLs)** — a single short code getting an extreme spike in reads is naturally absorbed by the cache (one hot key, cheaply served from memory) — no special handling needed beyond normal caching, unlike write-hot-key problems elsewhere.
- **At 10x scale:** move from a single DB to read replicas (cheap, since already read-heavy); only consider sharding by short_code hash if a single primary can't keep up with even the (small) write volume, which is unlikely at any realistic scale for this system.

Trade-offs

- Counter-based codes are sequential/guessable (`abc123`, `abc124`, ...) — if unguessability matters (privacy), add a bit of shuffling (e.g. XOR/permute the

counter bits before base62-encoding) rather than switching to pure random (which reintroduces collision risk).
- SQL (Postgres) chosen for simplicity/consistency; a KV store (DynamoDB) is an equally valid choice given the access pattern is a pure key lookup with no relational needs.

Follow-up Interview Questions

- *"How would you support analytics on click counts without slowing down the redirect?"* — Fire an async event (Kafka/SQS) from the redirect handler, aggregate downstream in a separate analytics pipeline; never do a synchronous DB write on the read-critical path.
- *"How do you prevent someone from shortening a malicious URL?"* — Check submitted URLs against a blocklist/threat-intel API (e.g. Google Safe Browsing) at creation time, and/or scan periodically.
- *"What if we need this globally distributed with low latency everywhere?"* — Multi-region deployment with regional read replicas/caches, and route users to their nearest region via GeoDNS (§5.5, §11.1); writes can go to a region-local primary with async cross-region replication if slightly stale reads across regions are acceptable (they usually are for URL metadata).

---

23. Design Pastebin

Problem

Design a service (like Pastebin/GitHub Gist) where users paste text/code snippets and get a shareable URL; others can view (and optionally the pastes expire or have visibility settings).

Requirements

Functional: (1) Create a paste (text up to some max size, e.g. 10MB) and get a unique URL. (2) Retrieve a paste by its URL. (3) Optional expiration (e.g. burn-after-read, 1 day, 1 week, never). (4) Optional syntax highlighting (client-side concern, not core to backend design). (5) Optional custom URL/paste visibility (public/private/unlisted).

Non-Functional: High availability for reads. Low latency reads. Durable storage (users expect pastes to persist for the chosen duration). Read-heavy, similar profile to the URL shortener but with **larger payloads** (text blobs, not just a URL string) — this is the key difference driving the design.

Estimation

Assume 1M new pastes/day, avg paste size 10KB, 10:1 read:write ratio.

```
Write QPS avg = 1,000,000 / 86,400 ≈ 11.6/sec
Read QPS avg  = 116/sec
Storage/day   = 1,000,000 × 10KB = 10 GB/day -> 1 year ≈ 3.65 TB (before
replication)
```

Conclusion: moderate write volume, but paste **content size** is the dominant cost driver (unlike the URL shortener where each record was tiny) — this pushes toward storing the paste body in **object storage**, not inline in the relational DB row (§18.6).

API Design

```
POST /api/v1/pastes   { "content": "...", "expiration": "1d", "visibility":
"unlisted" }
                       -> 201 { "paste_id": "xy9z1a", "url":
"https://paste.io/xy9z1a" }
GET  /api/v1/pastes/{paste_id}  -> { "content": "...", "created_at": "...",
"expires_at": "..." }
DELETE /api/v1/pastes/{paste_id}  (owner only, if accounts exist)
```

High-Level Design

```
Client -> LB -> App Servers -> [ metadata: Postgres/DynamoDB ]
                            -> [ content blob: Object Storage (S3) ]
                            -> [ Cache (Redis): hot paste content, cache-aside ]
```

- **Write path:** app server generates a unique paste ID (same base62/counter approach as §22), uploads the paste **content** to object storage keyed by

`paste_id`, and writes **metadata only** (paste_id, size, created_at, expires_at, visibility, S3 key) to the relational DB.

- **Read path:** look up metadata (check expiry/visibility), then fetch content from object storage (or serve from cache if hot) directly (potentially via a CDN-signed URL for public pastes, bypassing the app server entirely for the content bytes on cache hit).

Database Schema

```
CREATE TABLE pastes (
    paste_id     VARCHAR(10) PRIMARY KEY,
    s3_key       VARCHAR(255) NOT NULL,
    size_bytes   INT NOT NULL,
    visibility   VARCHAR(10) NOT NULL DEFAULT 'unlisted', --
public/private/unlisted
    owner_id     BIGINT NULL,
    created_at   TIMESTAMP NOT NULL DEFAULT now(),
    expires_at   TIMESTAMP NULL,
    burn_after_read BOOLEAN DEFAULT false
);
```

Deep Dive: Large Content Storage & "Burn After Read"

Storing large text blobs directly in a relational DB row bloats table size, hurts backup/replication performance, and wastes an expensive resource (relational storage/IOPS) on a cheap-storage need — always split metadata (small, relational, needs querying) from content (large, opaque, object storage) — this is the single most important design decision here, directly reusing §18.6's principle. **Burn-after-read** pastes need care: a naive "read-then-delete" has a race condition if two requests read concurrently before the delete completes — solve with an **atomic conditional delete** (e.g. a DB row update using `UPDATE pastes SET consumed=true WHERE paste_id=? AND consumed=false RETURNING *` — only one concurrent request will see `RETURNING` return a row; the loser gets nothing and returns 404/410 "already consumed").

Deep Dive: Expiration Cleanup at Scale

Rather than a full table scan looking for expired rows (expensive as the table grows), use: (a) an index on `expires_at`, (b) a background job that periodically

deletes/archives rows past expiry in small batches (avoiding one giant slow transaction locking the table), and (c) object storage lifecycle policies (S3 supports native "expire objects after N days" rules) to automatically clean up the corresponding blob without needing custom code for that half of the cleanup.

Bottlenecks & Scaling

- Large pastes (near the max size limit) could be abused for storage-cost attacks — enforce a max size at the API layer and consider per-user/per-IP paste-creation rate limits (§12.4/§24).
- At 10x scale, metadata DB read replicas + a cache for hot/viral pastes (same pattern as §22) comfortably absorb read growth; object storage scales independently and needs no special handling (that's the point of using it).

Trade-offs

- Storing small pastes (say <1KB) inline in the DB row alongside metadata (skipping object storage entirely) is a reasonable micro-optimization to avoid an extra network hop for the common case of tiny snippets — worth mentioning as a size-based hybrid strategy.

Follow-up Interview Questions

- *"How would you support private pastes with access control?"* — Add an `owner_id` + `visibility` check in the read path; for private pastes, require an authenticated request matching `owner_id` (or a shared secret/signed URL for "unlisted but link-only" pastes).
- *"How would you prevent someone scraping every paste by guessing IDs?"* — Use sufficiently long, random (not sequential/guessable) IDs for `unlisted`/`private` pastes specifically (unlike the URL shortener, guessability matters more here since pastes may contain sensitive text) — trade off the collision-free simplicity of counters for randomness + a uniqueness check on this specific field.

---

24. Design a Rate Limiter

Problem

Design a service/middleware that limits the number of requests a client (per user/IP/API key) can make in a given time window, protecting backend services from overload and abuse.

Requirements

Functional: (1) Limit requests per client to N per time window (e.g. 100/minute). (2) Support different limits per API/tier (free vs paid users). (3) Reject over-limit requests with a clear signal (HTTP 429 + `Retry-After` header). **Non-Functional:** The rate limiter itself must add minimal latency (it's on the critical path of *every* request). Must work correctly across a fleet of many app servers (not just per-instance). Should be resilient — if the rate limiter's own store is briefly unavailable, prefer to fail open (allow requests) or fail closed (reject) based on the specific system's risk tolerance (usually fail open for general APIs, fail closed for something like login attempt limiting where availability of the limiter matters less than the security property).

Estimation

A rate limiter check happens on **every single request** to the protected service — so at 50,000 req/sec of protected traffic, the limiter must sustain 50,000 checks/sec with sub-millisecond added latency. This immediately implies: the rate limiter's state store must be an **in-memory, low-latency store (Redis)**, never a relational DB round trip per request.

API Design

This is typically a **library/middleware/sidecar**, not a client-facing REST API — but the "API" is the contract with the request pipeline:

```
boolean allow(String clientKey)  // true = proceed, false = reject with 429
```

Exposed to clients only indirectly via response headers:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1699999999
```

High-Level Design

```
Client -> API Gateway / LB --check--> Rate Limiter (Redis-backed) --allow--> App
Servers
                                                  |
                                        (deny -> 429 response, short-circuit before
hitting
                                         app servers at all -- protects them from the
load)
```

Placing the rate limiter at the **API Gateway/edge** (rather than inside each individual app server) is the standard choice: it's checked once per request regardless of which downstream service handles it, and rejected requests never even reach (and waste capacity on) the actual application servers.

Deep Dive: Choosing the Algorithm (see also §12.4 for the summary table)

- **Token Bucket (recommended default):** each client has a bucket of capacity `B` tokens, refilling at rate `R` tokens/sec (up to max `B`). Each request consumes 1 token; if empty, reject. Naturally allows short bursts up to `B` while enforcing a long-run average rate of `R`. Implemented efficiently in Redis with a single key per client storing `(tokens, last_refill_timestamp)`, refilled lazily on each check (compute elapsed time × refill rate since last check, cap at B) — avoids needing a background refill job.
- **Sliding Window Counter (recommended for precision + memory efficiency at scale):** approximate the current window's count as a weighted blend of the previous and current fixed window counters: `count = current_window_count + previous_window_count × (1 - elapsed_fraction_of_current_window)`. Much cheaper in memory than a sliding window log (which stores every timestamp), while avoiding the fixed-window boundary-burst problem.

- **Fixed Window Counter:** simplest (`INCR` a Redis key with a TTL equal to the window size), but allows up to 2x the stated limit right at window boundaries — acceptable for loose/best-effort limiting, not for strict guarantees.

Deep Dive: Making It Work Across a Fleet (Distributed Rate Limiting)

A naive per-app-server in-memory counter is wrong the moment you have more than one app server — a client could get N requests through *each* server, exceeding the intended global limit by a factor of (number of servers). Fix: centralize counter state in a **shared Redis instance/cluster**, keyed by client identity (`rate_limit:{client_id}`), so all app server instances (or gateway instances) check/update the *same* counter. At very high QPS, even Redis can become a bottleneck/hot key for a single mega-client — mitigate with:

- **Local + global two-tier limiting:** each app server keeps an approximate local counter (cheap, no network call) and only syncs to the global Redis counter periodically or when nearing the limit, trading some precision for much lower Redis load.
- **Sharding the rate limiter's Redis** by client ID hash if a single Redis instance's throughput becomes the bottleneck (rare — Redis handles very high QPS for simple INCR- style operations).

Bottlenecks & Scaling

- The rate limiter's own store (Redis) becoming unavailable is the biggest single risk (since it's on every request's critical path) — run Redis with replication and a documented fail-open/fail-closed policy for that failure mode.
- Ensure the rate-limiting check itself is O(1) (a single Redis command or a small Lua script executed atomically server-side, avoiding read-then-write race conditions between the app server and Redis) — never do a "read counter, check in app code, write counter" as 3 separate round trips (race condition + 3x latency); use Redis's atomic `INCR` or a Lua script for check-and-increment in one atomic operation.

Trade-offs

- Token bucket allows bursts (good UX for legitimate bursty clients, e.g. a mobile app syncing after being offline) but is slightly more complex to implement correctly than a fixed window counter.
- Fail-open (allow requests when the limiter is down) protects availability of the protected service's core function but temporarily loses the protection the limiter provides — the right choice depends on whether "occasionally under-protected" or "occasionally over-restrictive" is the worse failure mode for the specific system.

Follow-up Interview Questions

- *"How would you rate-limit by multiple dimensions at once (per-user AND per-IP AND global)?"* — Check multiple independent limiter keys per request (e.g. `rate_limit:user:{id}`, `rate_limit:ip:{ip}`, `rate_limit:global`) and reject if **any** is exceeded; each can have its own bucket size/refill rate.
- *"How do you rate-limit at a CDN/edge layer across hundreds of global PoPs without a single central Redis becoming the bottleneck/adding cross-region latency?"* — Use **approximate, locally-enforced limits per edge node** (each PoP enforces a fraction of the global limit independently, refreshed/reconciled periodically) — trading strict global precision for latency and avoiding a cross-region round trip on every request; this is genuinely how large-scale edge rate limiting works in practice (e.g. Cloudflare's published approach).
- *"What's the difference between rate limiting and throttling/backpressure?"* — Rate limiting rejects excess requests outright (protects the service, costs the client a failed request); backpressure/throttling slows down or queues excess requests instead of rejecting them (better client experience, but requires the sender to be able to wait/queue, and doesn't bound worst-case latency the way a hard rate limit does).

25. Design a Web Crawler

Problem

Design a system that systematically browses the web, downloads pages, extracts links, and follows them to discover and index new pages (the foundation of a search engine's data collection layer).

Requirements

Functional: (1) Given a set of seed URLs, crawl pages and extract outbound links. (2) Avoid re-crawling the same URL repeatedly within some freshness window. (3) Respect `robots.txt` and per-site crawl-rate politeness. (4) Extract and store page content for downstream processing (indexing). **Non-Functional:** Scalable to billions of pages. Extensible (should be easy to add content-type-specific parsers). Must be **polite** (not overwhelm any single web server) and robust to malformed HTML, infinite redirect loops, and adversarial content (crawler traps).

Estimation

Assume crawling 1 billion pages/month, avg page size 100KB (including embedded resources metadata, not necessarily images):

```
Pages/sec avg = 1,000,000,000 / (30*86400) ≈ 386/sec  -> plan for much higher burst
capacity
Storage/month = 1B × 100KB = 100 TB/month (raw HTML) -- clearly needs object
storage + likely
                compression, and a lot of it
```

Conclusion: this is a large-scale, storage-heavy, highly parallel batch-style system — very different in shape from the request/response systems earlier (no single "user request" to optimize latency for; the optimization target is **throughput and coverage**, not per-request latency).

API Design

Mostly an internal batch system, not a public-facing API — the "interfaces" are internal queue contracts:

```
URLFrontier.addUrl(url, priority)
URLFrontier.getNextUrl() -> url   // for a fetcher worker to claim next
ContentStore.save(url, htmlContent, fetchedAt)
```

High-Level Design

```
Seed URLs -> URL Frontier (priority queue, per-host politeness queues)
                |
        Fetcher Workers (pool, horizontally scaled) --HTTP GET--> Internet
                |
        DNS Resolver (cache!)      Robots.txt Cache
                |
        Content Store (object storage, raw HTML) + Content Dedup (hash check)
                |
        Link Extractor -> new URLs -> back into URL Frontier (with dedup check)
                |
        Downstream: Indexer (out of scope for the crawler itself)
```

- **URL Frontier** is the heart of the system — a distributed priority queue that decides what to crawl next, balancing overall priority (freshness needs, page importance) against strict **per-host politeness** (never hammer one server with concurrent/rapid requests).
- **Fetcher Workers** are simple, horizontally-scalable stateless workers that pull a URL, fetch it (respecting robots.txt and timeouts), and hand the result off.
- **Dedup** happens at two levels: URL-level (don't re-queue a URL already crawled recently — check a seen-URL store, e.g. a Bloom filter for cheap approximate membership testing at massive scale) and content-level (two different URLs can have identical/near-identical content — hash the content and skip reprocessing if already seen, common with mirrored/ syndicated content).

Deep Dive: The URL Frontier & Politeness

A single global FIFO queue is wrong — it would let a fetcher worker crawl the same host back-to-back at full speed (impolite, likely to get IP-banned), and gives no way to prioritize important/fresh pages. The standard design (from the original Mercator crawler paper) uses **two layers of queues**:

1. **Front queues** — prioritized by crawl priority/freshness needs; a URL is assigned to a front queue based on its priority score.
2. **Back queues** — one queue **per host**, each processed by only one fetcher at a time, with an enforced minimum delay between requests to the same host. A separate **mapping table** tracks which host is assigned to which back queue and whether it's currently "busy" (being fetched) — a worker only pulls from a back queue whose host isn't currently in-flight and whose politeness delay has elapsed. This structure guarantees per-host politeness by construction while still allowing massive overall parallelism (many *different* hosts fetched concurrently).

Deep Dive: Avoiding Crawler Traps & Duplicate Content

- **Crawler traps** — pages that generate infinite/near-infinite URLs (e.g. a calendar page linking to "next month" forever, or session-ID-embedded URLs creating unique URLs per visit). Mitigate with: a max crawl depth per site, a max URL count per host, detecting and normalizing URL patterns (strip known tracking/session params before dedup-checking), and anomaly detection (a host suddenly generating far more unique URLs than typical gets flagged/throttled).
- **Near-duplicate content** across different URLs (mirrors, syndicated articles, URL parameter variations of the same page) — use a **content fingerprint** (e.g. SimHash or MinHash over shingled content, not a plain cryptographic hash, since near-duplicates differ slightly byte-for-byte) to detect and skip/deprioritize near-duplicate pages without needing exact byte-for-byte matches.
- **URL normalization** (lowercase host, remove default ports, sort query params, strip fragments) before dedup-checking, so trivially-different URL strings pointing at the same resource aren't treated as distinct.

Bottlenecks & Scaling

- **DNS resolution** for millions of distinct hosts can itself become a bottleneck — run a local caching DNS resolver layer in front of fetcher workers.
- **Seen-URL set** at billions of URLs is too large for an in-memory exact set on one machine — use a **distributed Bloom filter** (accepts a small false-positive rate — occasionally re-crawling/skipping a URL unnecessarily — in exchange for massive memory savings) partitioned across multiple nodes.
- The Frontier itself must be partitioned/distributed (e.g. by host hash) across many machines as scale grows, since a single machine can't hold politeness state for every host on the internet.

Trade-offs

- Freshness vs load: recrawling every page frequently keeps content fresh but multiplies crawl cost — prioritize recrawl frequency by observed page change rate (a page that historically changes daily gets recrawled far more often than a static one).

- Breadth-first vs priority-based crawling: BFS is simple and gives broad coverage quickly; priority-based (e.g. PageRank-influenced) crawling finds "important" pages faster but adds complexity in maintaining priority scores.

Follow-up Interview Questions

- *"How do you handle a site that returns different content based on user-agent (cloaking) or requires JavaScript rendering (SPA sites)?"* — Maintain a headless-browser rendering tier (e.g. Puppeteer/Playwright workers) for JS-heavy sites, used selectively (expensive) rather than for every page; detect via a heuristic (near-empty initial HTML) or a known-site allowlist.
- *"How would you prioritize re-crawling news sites vs static reference pages?"* — Maintain a per-URL/per-domain "change frequency" estimate (updated based on observed diffs across crawls) feeding into the Frontier's priority score — news domains naturally earn a much higher recrawl priority than a static "About Us" page.

---

26. Design a Notification System

Problem

Design a system that sends notifications (push, email, SMS, in-app) to users, triggered by various events across a platform (e.g. "someone liked your post", "your order shipped"), reliably and at scale, across multiple channels.

Requirements

Functional: (1) Accept notification requests from many internal services (order service, social service, marketing). (2) Deliver via multiple channels: push notification (iOS/Android), email, SMS, in-app. (3) Respect user preferences (opted out of marketing emails, quiet hours). (4) Support templating (localized, personalized content). (5) Retry failed deliveries. **Non-Functional:** High throughput (millions of notifications/day, bursty around events like a flash sale). At-least-once delivery (missing a critical notification is worse than an occasional duplicate). Must not become a bottleneck for the triggering services (fully asynchronous from their perspective).

Estimation

Assume 500M notifications/day across all channels, heavily bursty (a single marketing campaign can trigger 50M notifications within an hour):

```
Avg rate  = 500,000,000 / 86,400 ≈ 5,787/sec
Peak rate (campaign burst) could be 50-100x average momentarily -> system must
handle huge
bursts via queuing/buffering rather than provisioning for peak at all times (cost-
prohibitive)
```

Conclusion: the extreme burstiness (not steady-state volume) is the dominant design driver → the system **must** be queue-based (absorb bursts, process at a sustainable rate) rather than trying to synchronously handle traffic spikes.

API Design

```
POST /internal/v1/notifications
{
  "user_id": "123",
  "event_type": "ORDER_SHIPPED",
  "channels": ["push", "email"],
  "template_data": { "order_id": "789", "tracking_url": "..." }
}
-> 202 Accepted { "notification_id": "n_abc" }   // async -- accepted, not "sent"
yet
```

High-Level Design

```
Triggering Services -> Notification API (validates, enqueues) -> Kafka (topic per
priority/type)
                                                                        |
                                                          Notification Workers
(consumer group)
                                                                        |
                                  +---------------------+-----------------+----------
--------+
                                  |                     |                 |
|
                          Push Provider          Email Provider     SMS Provider
In-App Store
                        (APNs/FCM)             (SendGrid/SES)     (Twilio)
(write to DB,
```

```
served on next

app open)
```

- **Notification API** does minimal synchronous work: validate the request, check basic user preferences (a fast cache lookup — has this user opted out entirely?), and enqueue to Kafka — responds `202 Accepted` immediately. All the actual channel-specific delivery logic happens asynchronously in worker processes.
- **Workers** consume from Kafka, resolve full user preferences + contact info (from a user profile service/cache), render the templated content (localized/personalized), and call the relevant third-party provider (APNs/FCM for push, SendGrid/SES for email, Twilio for SMS).
- **Priority topics**: separate Kafka topics (or partitions) for high-priority (transactional: "your payment failed") vs low-priority (marketing) notifications, so a huge marketing campaign burst doesn't delay time-sensitive transactional notifications — consumers can be scaled/prioritized independently per topic.

Database Schema

```
CREATE TABLE notification_log (
    notification_id BIGINT PRIMARY KEY,
    user_id      BIGINT NOT NULL,
    event_type   VARCHAR(50),
    channel      VARCHAR(20),
    status       VARCHAR(20),  -- QUEUED, SENT, FAILED, DELIVERED
    created_at   TIMESTAMP,
    sent_at      TIMESTAMP NULL
);
CREATE TABLE user_preferences (
    user_id BIGINT PRIMARY KEY,
    push_enabled BOOLEAN, email_enabled BOOLEAN, sms_enabled BOOLEAN,
    quiet_hours_start TIME, quiet_hours_end TIME
);
```

`notification_log` is write-heavy and mostly append-only — a good candidate for a wide-column store (Cassandra) at very large scale instead of a relational DB, partitioned by `user_id` for fast "show me my notification history" queries.

Deep Dive: Reliable, At-Least-Once Delivery with Idempotency

Workers can crash mid-delivery, or a third-party provider call can time out without a clear success/fail signal — the system must retry, which risks duplicate sends. Solution: use Kafka's at-least-once consumer semantics (§10.4) combined with an **idempotency key** per notification attempt (e.g. `notification_id + channel`), checked against `notification_log` before actually calling the provider — if already marked `SENT`/`DELIVERED`, skip re-sending even if the message is redelivered. For failed sends, use **exponential backoff with jitter** for retries (avoid a "retry storm" against a temporarily-struggling third-party provider), and after N retries, move to a dead-letter queue (§10.5) for manual/automated follow-up investigation.

Deep Dive: Respecting User Preferences & Quiet Hours at Scale

Checking user preferences shouldn't require a synchronous DB call per notification at worker time for every message (expensive at millions/day) — cache user preference data aggressively (Redis, TTL-based invalidation on preference change events) so the check is a fast in-memory lookup. Quiet hours (e.g. don't push notifications between 10pm-8am user local time) require knowing the user's **timezone**, and either delaying the send (re-enqueue with a scheduled delivery time) or holding it in a **delay queue** (Kafka doesn't natively support per-message delayed delivery well; common approaches: a separate "scheduled" datastore polled periodically, or SQS's native delay-queue feature, or a dedicated scheduler service, §40).

Bottlenecks & Scaling

- **Third-party provider rate limits** (APNs/FCM/SendGrid all have their own throughput caps) — the notification system must itself rate-limit its outbound calls per provider (§24) and queue/backoff gracefully rather than losing notifications when a provider throttles it.
- **Consumer lag during huge bursts** — auto-scale worker consumer groups based on Kafka consumer lag metrics; because the system is queue-based, a burst causes *temporary delay*, not dropped notifications or an outage — a key resilience property to call out explicitly.

Trade-offs

- Fan-out to all channels always vs letting user choose exactly one preferred channel per event type: richer preference granularity improves UX but adds significant preference- management complexity — scope this explicitly with the interviewer (§2's "confirm scope" habit applies directly here).

Follow-up Interview Questions

- *"How would you support 'digest' notifications (batch multiple events into one summary notification, e.g. 'You have 5 new likes') instead of one notification per event?"* — Buffer events per user in a short time window (e.g. via a sliding window keyed by user ID in Redis, or a scheduled batch job), and periodically flush a single combined notification instead of sending each event immediately — trades a small latency delay for drastically less notification spam.
- *"How do you track whether a push notification was actually delivered/opened?"* — Providers (APNs/FCM) give delivery receipts/webhooks; opens are tracked via a tracking pixel (email) or an app-side "notification opened" event sent back to an analytics endpoint, joined back to `notification_id` for funnel analysis.

---

27. Design a News Feed (Twitter/Facebook)

Problem

Design a system that shows each user a personalized, reverse-chronological (or ranked) feed of posts from accounts/pages they follow.

Requirements

Functional: (1) Users post content (text/media). (2) Users follow other users. (3) Users view a feed composed of posts from people they follow. (4) (Optional) Ranking beyond simple reverse-chronological. **Non-Functional:** Very read-heavy (feed views ≫ posts created). Low latency feed loads (<200ms). Must handle **extreme fan-out skew** — some accounts have hundreds of millions of followers ("celebrity problem"), others have a handful.

Estimation

Assume 500M DAU, each viewing their feed 10x/day, 100M posts/day created:

```
Feed read QPS avg  = 500M × 10 / 86,400 ≈ 57,900/sec  -> peak (x3) ≈ 173,700/sec
Post write QPS avg = 100M / 86,400 ≈ 1,157/sec
Avg followers/user: highly skewed (power law) -- median follower count is small
(tens-
  hundreds), but a small fraction of accounts have 10M-300M+ followers
```

Conclusion: massively read-heavy, and the follower-count **skew** (not the average) is what actually drives the hardest design decision (the fan-out strategy below) — averages lie here; the tail matters most.

API Design

```
POST /api/v1/posts          { "content": "...", "media_ids": [...] } -> 201 {
"post_id": "..." }
GET  /api/v1/feed?cursor=<opaque_cursor>&limit=20  -> { "posts": [...],
"next_cursor": "..." }
POST /api/v1/follow          { "target_user_id": "..." }
```

Feed pagination uses **cursor-based pagination** (§12.3), not offset-based — feeds are constantly changing (new posts inserted), so offset pagination would cause skipped/ duplicated items as users scroll.

High-Level Design — The Core Trade-off: Fan-out on Write vs Fan-out on Read

```
Option A: Fan-out on Write (push)
  User posts -> Post Service -> for each follower: insert post_id into follower's
                                  precomputed feed (e.g. a Redis list/sorted set
per user)
  Feed read: just fetch the precomputed feed list for this user -- O(1)-ish, very
fast reads

Option B: Fan-out on Read (pull)
  User posts -> Post Service -> just store the post (no fan-out at write time)
  Feed read: fetch the list of people I follow -> fetch recent posts from each of
them ->
             merge/sort by time -- expensive at read time, especially for users
```

```
following
             thousands of accounts
```

The hybrid approach (what real systems actually do): fan-out on write for the vast majority of users (fast reads, and write cost is bounded by *their* typically-small follower count), but **fan-out on read for celebrity accounts** (accounts above some follower-count threshold, e.g. 1M+ followers) — writing a single post out to 300 million individual feed lists would be a catastrophic write amplification (300M writes for one post!). Instead, celebrity posts are fetched at read time and merged into the follower's feed alongside their precomputed (fanned-out) feed from normal accounts they follow.

Database & Storage Design

```
-- Source of truth
CREATE TABLE posts (
    post_id BIGINT PRIMARY KEY, author_id BIGINT, content TEXT, created_at
TIMESTAMP
);
CREATE TABLE follows (
    follower_id BIGINT, followee_id BIGINT, PRIMARY KEY (follower_id, followee_id)
);
```

```
-- Precomputed feed (Redis sorted set per user, score = timestamp, for fast fan-
out-on-write reads)
ZADD feed:{user_id}  <timestamp>  <post_id>
ZREVRANGE feed:{user_id} 0 19    -- fetch latest 20 for this user's feed
```

The `follows` table (relational, for the social graph and for determining who to fan out to) is distinct from the precomputed feed cache (Redis, purely a fast-read materialized view rebuilt/appended incrementally as posts come in).

Deep Dive: Handling the Celebrity/Hot-Key Fan-out Problem

1. Classify accounts by follower count (a simple threshold check, cached, updated periodically — doesn't need to be perfectly real-time).
2. **Normal accounts (below threshold):** on post, enqueue an async fan-out job (don't do it synchronously in the post-creation request — could be pushing to

hundreds/thousands of follower feed lists) that appends the new `post_id` to every follower's precomputed Redis feed list.
3. **Celebrity accounts (above threshold): skip fan-out entirely** at write time. On feed read, in addition to fetching the user's own precomputed feed, separately fetch the most recent posts from each celebrity the user follows (typically a small number of celebrities per user, even if celebrities individually have huge follower counts) and merge them into the response in-memory, sorted by time. This hybrid is the standard real-world answer (used in some form by Twitter/Instagram) and is a very strong deep-dive answer precisely because it shows you understand *why* a uniform approach (pure push or pure pull) fails at the tails, and how to combine both.

Deep Dive: Ranking Beyond Reverse-Chronological

Real feeds (Facebook/Instagram) don't show pure reverse-chronological order — a ranking model scores candidate posts (recency, predicted engagement, relationship strength, content type diversity) and reorders them. Architecturally this adds a **candidate generation** stage (gather a pool of maybe 500-1000 candidate posts from follows + recommendations) followed by a **ranking** stage (score and sort the pool, typically via an ML model service) before returning the final top-20 to the client — this two-stage "generate then rank" pattern is standard for any large-scale ranked feed/recommendation system, and is worth mentioning even if full ML ranking details are out of scope for the interview.

Bottlenecks & Scaling

- **Precomputed feed storage** grows with (avg followers × posts/day) — bound it by only keeping the most recent ~1000 post IDs per user's feed list (trim older entries), since users rarely scroll back further than that.
- **Async fan-out queue backlog** during a burst (a very active, widely-followed-but-not- quite-celebrity account posting) — auto-scale fan-out workers, and the threshold for "celebrity treatment" can be tuned/extended dynamically if a particular account's fan-out job is taking too long.

Trade-offs

- Fan-out on write: faster reads, but wasted write work if a user never actually checks their feed before new posts push older ones out anyway (some inherent waste is accepted as the cost of consistently fast reads).
- Pure fan-out on read: no write amplification at all, but every feed read becomes an expensive fan-in across all followed accounts — untenable at hundreds-of-millions-of- users scale for the majority of normal (non-celebrity) reads.

Follow-up Interview Questions

- *"How do you handle a user unfollowing someone — does their precomputed feed need to be scrubbed of that person's old posts immediately?"* — Usually **no** — acceptable to let already-fanned-out posts age out naturally (eventual consistency for this edge case is fine; a strict UX requirement here would need an active removal job, adding real complexity for a rare, low-severity edge case).
- *"How would you support 'Stories' (ephemeral content, expires after 24h) in this architecture?"* — Similar fan-out approach but with a **TTL** on the feed entries (and the underlying content in object storage) — reuse the same Redis sorted-set feed pattern with an expiry-aware read path that filters out expired entries.

28. Design a Chat/Messaging App (WhatsApp)

Problem

Design a real-time one-on-one and group messaging system with delivery/read receipts, online presence, and message durability.

Requirements

Functional: (1) Send/receive 1:1 messages in real-time. (2) Group messaging (up to ~256 members). (3) Delivery status (sent/delivered/read receipts). (4) Online/last-seen presence. (5) Message history persisted and retrievable across devices. **Non-Functional:** Low latency delivery (near-instant, <1s for online recipients). High availability. At-least-once delivery with client-side dedup (messages must never silently

disappear). Should work across flaky mobile networks (offline support, reconnect/resync).

Estimation

Assume 500M DAU, each sending 50 messages/day:

```
Messages/day = 25B -> avg QPS ≈ 289,000/sec -> peak (x3) ≈ 867,000/sec
Avg message size ≈ 100 bytes (text) -- media handled separately via object storage
Storage/day  = 25B × 100B = 2.5 TB/day (metadata+text) -> significant, needs a
               write-optimized, horizontally-scalable store (not a single Postgres
instance)
Concurrent WebSocket connections: up to 500M (if all DAU online at once,
realistically a
  large fraction concurrently during peak) -> connection-holding infrastructure
must scale
  horizontally and independently of message-processing throughput
```

Conclusion: two distinct scaling problems: (1) **very high message throughput** (favors a wide-column/log-structured store, §8.2/8.3) and (2) **holding millions of concurrent stateful connections** (favors a dedicated, horizontally-scaled connection-management tier, distinct from stateless REST app servers).

API Design

Primarily **WebSocket-based** (not REST) for the real-time send/receive path, since this needs bidirectional, low-latency, persistent connections (§11.4):

```
WS connect (authenticated) -> client stays connected
Client -> Server: { "type": "SEND_MESSAGE", "to": "user_or_group_id", "content":
"...", "client_msg_id": "..." }
Server -> Client: { "type": "MESSAGE", "from": "...", "content": "...", "msg_id":
"...", "timestamp": "..." }
Server -> Client: { "type": "ACK", "client_msg_id": "...", "server_msg_id": "..." }
Server -> Client: { "type": "DELIVERY_RECEIPT", "msg_id": "...", "status":
"delivered|read" }
```

A REST API handles non-realtime concerns: `GET /api/v1/conversations/{id}/messages?cursor=` (history/pagination), user/group management.

High-Level Design

```
Client (mobile/web) --WebSocket--> Connection Gateway (stateful, horizontally
scaled)
                                          |
                              (tracks: which gateway instance holds which user's
connection --
                               stored in a shared registry, e.g. Redis: user_id ->
gateway_instance_id)
                                          |
                                    Message Service
                                    /            \
                            Message Store        Presence Service (online/last-seen,
Redis)
                        (Cassandra/DynamoDB,
                         partitioned by conversation_id)
```

- **Connection Gateway** — a fleet of servers holding long-lived WebSocket connections. Because connections are stateful (unlike the stateless HTTP app servers elsewhere in this guide), routing a message *to* a specific user requires knowing *which* gateway instance currently holds their connection — solved via a **connection registry** (Redis mapping `user_id → gateway_instance_id`), updated on connect/disconnect.
- **Sending flow:** sender's gateway receives the message → writes it durably to the Message Store (source of truth, so it survives even if the recipient is offline) → looks up the recipient's connection registry entry → if online, forwards the message to the recipient's specific gateway instance (via an internal pub-sub, e.g. Redis Pub/Sub or a lightweight internal message bus between gateway instances) which pushes it down their WebSocket → if offline, the message waits in the store; a push notification (§26) is sent instead, and the message is delivered on next reconnect (client syncs unread/undelivered messages since its last known state).

Database Schema

```
-- Wide-column model (Cassandra-style), partitioned by conversation_id for fast,
ordered
-- retrieval of a conversation's history (the dominant access pattern)
Table: messages
  Partition Key: conversation_id
  Clustering Key: message_id (time-ordered, e.g. Snowflake-style ID, see §37)
  Columns: sender_id, content, created_at, delivered_to (set), read_by (set)
```

Partitioning by `conversation_id` means fetching a conversation's history is a single-partition range scan (fast); it does mean a single, extremely active group's data lives on one partition — acceptable in practice since even large groups (~256 members) don't approach the message *volume* of, say, a celebrity's fan-out problem (§27).

Deep Dive: Exactly-Once-Feeling Delivery (At-Least-Once + Dedup)

Networks are unreliable (mobile especially) — a client may send a message, not receive the server's ACK (due to a network blip), and retry, risking a duplicate. Solution: the client generates a **client-side unique message ID** (`client_msg_id`, e.g. a UUID) at send time and includes it in every retry of the same logical message; the server stores/checks this ID (per conversation) before persisting, so a retried send with the same `client_msg_id` is recognized as a duplicate and simply re-acknowledged rather than stored/delivered twice — the same idempotency-key pattern used throughout this guide (§10.4, §12.5), applied here to a WebSocket-based protocol instead of REST.

Deep Dive: Multi-Device Sync & Offline Support

Users expect the same conversation history across phone + web + desktop. Each device maintains a **local cursor** (the last message ID/timestamp it has synced). On reconnect (app foreground, network restored), the client sends its last-known cursor per conversation; the server returns everything since that cursor (a straightforward range query against the partitioned message store) — this is the same cursor-based "catch up" pattern used for feed pagination (§12.3, §27), just applied to sync instead of scroll. Read/delivery receipts need to be tracked **per-device or reconciled across devices** (a message is "read" once any one of the user's devices has seen it, typically) — a small but real product-requirements nuance worth surfacing with the interviewer.

Bottlenecks & Scaling

- **Connection Gateway fleet** scales by adding more instances; the connection registry (Redis) must handle very high read/write rates (a connect/disconnect and periodic heartbeat per active connection) — shard the registry by user ID hash if a single Redis instance becomes the bottleneck.
- **Group message fan-out**: sending to a 256-person group means writing/delivering to up to 256 recipients per message — bounded and small enough to handle synchronously/quickly per message, unlike the millions-of-followers feed fan-out problem (§27) — group size caps exist partly *because* of this scaling concern.

Trade-offs

- WebSockets require holding persistent connections (real infra cost/complexity vs plain REST) — justified here because near-instant bidirectional delivery is a core, explicit requirement (unlike, say, a notification system where a short delay is fine).
- End-to-end encryption (mentioned as a real WhatsApp feature) fundamentally changes the server's role to "blind relay" (can't inspect/index message content) — worth a one-line mention if the interviewer probes security, but usually out of scope for a 45-min HLD.

Follow-up Interview Questions

- *"How would you show 'typing...' indicators?"* — Lightweight, ephemeral, non-durable events (don't persist to the message store at all) sent directly through the same gateway pub-sub path used for message delivery — no need for durability/at-least-once guarantees here since a missed typing indicator has near-zero consequence.
- *"How do you handle a user with hundreds of devices/sessions (e.g. logged in on many browsers)?"* — The connection registry maps `user_id → set of (gateway_instance, connection_id)` pairs rather than a single value, and delivery fans out to all active sessions for that user.

---

29. Design Video Streaming (YouTube/Netflix)

Problem

Design a system for uploading, processing, storing, and streaming video content to millions of users with adaptive quality based on network conditions.

Requirements

Functional: (1) Users upload videos. (2) Videos are processed into multiple resolutions/ bitrates. (3) Users stream videos with quality adapting to their network/device. (4) Basic metadata (title, description, view count). **Non-Functional:** Extremely read-heavy (views ≫ uploads by orders of magnitude). Massive storage and bandwidth requirements. Low startup latency (video should start playing quickly) and smooth playback without buffering across varying network conditions globally.

Estimation

Assume 500M DAU watching avg 30 min/day, 500 hours of video uploaded/minute (a commonly cited real-world YouTube-scale figure for illustration):

```
Total watch time/day = 500M × 30 min = 15B minutes/day
Upload volume/day    = 500 hours/min × 1440 min/day = 720,000 hours of raw
video/day
```

Conclusion: the read (streaming/bandwidth) side dwarfs the write (upload/processing) side by many orders of magnitude — this screams **CDN-first architecture**; almost no playback request should ever reach origin storage/servers directly.

API Design

```
POST /api/v1/videos/upload    (resumable/chunked upload, returns an upload session)
GET  /api/v1/videos/{id}/manifest   -> adaptive streaming manifest (HLS .m3u8 /
DASH .mpd)
```

```
GET  /api/v1/videos/{id}/metadata   -> { title, description, view_count, duration,
... }
```

Actual video **segment** bytes are fetched by the client player directly from CDN URLs listed in the manifest — not proxied through the application's API servers at all.

High-Level Design

```
Upload path:
Client -> Upload Service (chunked/resumable, §18.4) -> Raw video in Object Storage
                                                                  |
                                                       Transcoding Pipeline (async,
queue-driven):
                                                         - Split into segments
                                                         - Encode at multiple
resolutions/bitrates

(240p/360p/480p/720p/1080p/4K)
                                                         - Generate adaptive
streaming manifest
                                                                  |
                                                       Processed segments -> Object
Storage -> CDN

Playback path:
Client -> Metadata Service (video info, manifest URL)
Client -> CDN (fetch manifest, then fetch video segments directly) -- origin barely
touched
        -> adaptive bitrate logic in the player selects segment quality per network
conditions
```

Deep Dive: Transcoding Pipeline

Raw uploaded video must be converted into multiple resolutions/bitrates (so different devices/network conditions can be served appropriately) — this is CPU-intensive and **must be asynchronous** (never block the upload response on hours of processing). Pipeline: (1) upload triggers an event onto a queue, (2) a fleet of transcoding workers (often GPU/specialized-hardware-accelerated) picks up the job, splits the video into short segments (e.g. 2-10 seconds each — this is what makes adaptive streaming and seeking efficient, §18.4), encodes each segment at each target resolution/bitrate, and uploads results back to object storage, (3) once all renditions are ready, a manifest file (HLS `.m3u8` or DASH `.mpd`) listing all available quality variants and their segment

URLs is generated, (4) only then is the video marked "ready to stream" and surfaced to viewers — until then, upload status shows "processing."

Deep Dive: Adaptive Bitrate Streaming (ABR)

The client video player, not the server, makes real-time quality decisions: it continuously monitors its own download throughput and buffer health, and for each upcoming segment, requests whichever quality variant fits current conditions (higher quality if buffer is healthy and bandwidth is high, lower/safer quality if bandwidth drops or the buffer is running low, to avoid stalling/rebuffering). This client-driven design (rather than a server trying to guess/push a quality) is precisely why videos are pre-encoded into **multiple discrete quality variants ahead of time** and split into short, independently- fetchable segments — it turns "streaming" into "the client repeatedly fetching small, independent, well-chosen HTTP files," which composes naturally with standard CDN caching (§9.7) instead of needing a stateful streaming protocol/connection per viewer.

Deep Dive: CDN Strategy for Massive Read Scale

Video is by far the most bandwidth-heavy content type in most systems — a well-designed architecture ensures the **overwhelming majority of playback bytes never touch origin infrastructure at all**: (1) push popular/new content proactively to CDN edges (push CDN, §9.7) rather than waiting for the first viewer's request to trigger a pull, (2) use **multiple CDN providers** and/or a company's own edge cache network (large-scale video platforms often operate custom CDN infrastructure, sometimes even embedding caching appliances inside ISP networks) for cost and redundancy, (3) origin storage/servers are sized for the much smaller "cold/long-tail content" case and for the initial transcoding-to-CDN push, not for serving popular-content playback traffic directly.

Bottlenecks & Scaling

- **Transcoding throughput** — a viral upload event or generally high upload volume can queue up transcoding jobs; auto-scale the worker fleet based on queue depth, and consider prioritizing shorter/lower-resolution renditions first (get *some* watchable quality available quickly, backfill higher resolutions after) to reduce perceived "processing" wait time.

- **View count updates** at massive scale (a popular video's view counter is a hot key) — batch/debounce increments (buffer view events and flush aggregated counts periodically, e.g. every few seconds) rather than a synchronous DB increment per view — trades perfectly real-time accuracy for enormously reduced write load, an acceptable trade-off for a vanity metric like view count.

Trade-offs

- Encoding many resolution/bitrate variants upfront costs significant compute + storage (multiple copies of every video) but is what enables smooth, network-adaptive playback — the alternative (transcode-on-demand per request) would add unacceptable startup latency.
- Segment length is a genuine trade-off: shorter segments enable faster quality-switching reaction time (better adaptability) but add per-segment HTTP request overhead; longer segments reduce overhead but react more slowly to changing network conditions.

Follow-up Interview Questions

- *"How would you support live streaming (not just pre-recorded video)?"* — Fundamentally different pipeline: encode and segment in near-real-time as the stream arrives (much tighter latency budget on the transcoding step itself), shorter segment durations, and the manifest is continuously appended to rather than finalized upfront — live introduces a real latency-vs-buffering trade-off that recorded video doesn't have.
- *"How do you handle copyright/content moderation at upload time without delaying availability too much?"* — Run automated content-matching (e.g. audio/video fingerprinting against a known-content database) asynchronously alongside transcoding; don't block "ready to view" status on a full manual review for most content, but do flag/ restrict distribution automatically for matched content while any human review completes.

---

30. Design Ride Sharing (Uber)

Problem

Design a system that matches riders requesting a trip with nearby available drivers, tracks the trip in real-time, and handles pricing.

Requirements

Functional: (1) Rider requests a ride from location A to B. (2) System finds and matches a nearby available driver. (3) Both parties see real-time location during the trip. (4) Dynamic/surge pricing based on supply/demand. (5) Trip completion and payment. **Non-Functional:** Very low latency matching (riders won't wait long for a match). High availability (a ride request failing has real-world consequences). Must handle extremely frequent location updates (every driver's GPS ticks every few seconds) at massive geographic scale.

Estimation

Assume 5M concurrent active drivers globally, each sending a location update every 4 seconds, plus 1M ride requests/hour at peak:

```
Location update QPS = 5,000,000 / 4 ≈ 1,250,000 updates/sec -- this dwarfs ride-
request
                        QPS and is the dominant write load of the entire system
Ride request QPS (peak) = 1,000,000 / 3600 ≈ 278/sec (comparatively tiny)
```

Conclusion: the true scaling challenge here is **ingesting and querying massive-volume, constantly-updating geospatial data efficiently** — not the comparatively low-volume ride- matching requests themselves. The entire architecture should be optimized around this.

API Design

```
POST /api/v1/drivers/{id}/location   { "lat": ..., "lng": ..., "timestamp": ... }
(very high frequency, from driver app)
POST /api/v1/rides/request            { "pickup": {lat,lng}, "dropoff": {lat,lng} }
-> { "ride_id", "estimated_fare" }
GET  /api/v1/rides/{id}/status        -> real-time status + driver location (poll
or WebSocket)
POST /api/v1/rides/{id}/complete
```

Real-time trip tracking uses WebSockets/SSE (§11.4) rather than polling, for the same latency reasons as the chat app (§28).

High-Level Design

```
Driver App --location ping (high freq)--> Location Ingestion Service -> Geospatial
Index
                                                                          (in-memory,
e.g. Redis
                                                                           with
geospatial commands,
                                                                           or
Quadtree/Geohash-based)
Rider App --ride request--> Matching Service --query nearby drivers--> Geospatial
Index
                                  |
                            Pricing Service (surge calc based on supply/demand
ratio per region)
                                  |
                            Trip Service (assigns driver, tracks state machine,
§21.10 State pattern)
                                  |
                            Notification (§26) to both parties
```

- **Geospatial Index** is the core data structure — it must answer "which drivers are within X km of this point?" extremely fast, continuously updated as drivers move.
- **Matching Service** queries nearby available drivers, applies matching logic (nearest first, but also considering driver rating, vehicle type requested, acceptance likelihood), and sends a ride offer to the chosen driver (with a short timeout to accept before offering to the next candidate).

Deep Dive: Geospatial Indexing at Scale

Naively scanning all drivers' lat/lng and computing distance to the rider for every request doesn't scale (millions of drivers, hundreds of requests/sec). Standard solutions:

- **Geohashing** — encode (lat, lng) into a string where nearby locations share common string prefixes (recursively subdividing the world into a grid of cells). Nearby drivers can be found efficiently by querying drivers whose geohash shares a prefix with the rider's geohash (checking the current cell and adjacent cells to handle points near a cell boundary). Geohashes map naturally onto Redis sorted

sets or dedicated geospatial commands (`GEOADD`/`GEORADIUS`), making this simple to implement on top of infrastructure already in this guide's toolbox.

- **Quadtree** — recursively subdivide 2D space into 4 quadrants, subdividing further only where driver density is high — naturally adapts to non-uniform density (dense in cities, sparse in rural areas), unlike a fixed-size geohash grid. A nearest-drivers query traverses down the tree to the relevant region.
- **S2/H3 (hierarchical hexagonal/spherical grids)** — more sophisticated cell-based systems (used by Google/Uber respectively in real life) that better handle the distortion of mapping a sphere (Earth) onto flat grid cells, especially near poles; mentioning these by name shows awareness of production-grade solutions beyond the textbook geohash/quadtree answer.
- Given the location-update volume estimated above (1.25M/sec), the geospatial index must live **in-memory** (Redis-backed geospatial index, sharded by region/geohash-prefix across multiple Redis instances) — a disk-backed relational DB could not sustain this write rate for a continuously-updating index.

Deep Dive: The Matching Algorithm & Driver Assignment Race Condition

Once nearby candidate drivers are found, a real system must avoid **double-booking** a driver (two simultaneous ride requests both trying to assign the same driver). Solve with an **atomic conditional assignment**: when offering a ride to a driver, atomically mark them "pending offer" (e.g. a Redis `SETNX`-style conditional write, or a DB row update with an optimistic version check) so a concurrent request can't also grab them; if the driver doesn't accept within a short timeout, atomically release them back to "available" and offer the ride to the next candidate. This is the same fundamental atomic check-and-set pattern used for burn-after-read pastes (§23) and distributed locks (§15.9), applied here to driver assignment.

Deep Dive: Surge Pricing

A pricing service continuously computes a **supply/demand ratio per geographic region** (e.g. active ride requests vs available drivers within a region/geohash cell, recalculated on a short interval like every 1-5 minutes) and applies a multiplier to base fare when demand outstrips supply — incentivizing more drivers to come online/reposition toward high-demand areas, and moderating rider demand at peak times. This is a relatively simple real-time aggregation (count requests, count available

drivers, per region) rather than a complex ML system in its most basic form — a good, honest scope to present in an interview unless explicitly asked to go deeper into pricing ML.

Bottlenecks & Scaling

- **Geospatial index write throughput** (driver location pings) is the dominant load — shard by region (a driver's updates only ever hit the shard for their current region, naturally partitioning load geographically, which also matches real usage — a matching query for a NYC rider never needs to consult Tokyo's shard).
- **Regional independence** — ride matching is inherently regional (you'd never match a rider in NYC with a driver in Tokyo), making this system unusually well-suited to full geographic sharding/regional deployment compared to most other case studies in this guide.

Trade-offs

- More frequent location pings (e.g. every 2s instead of 4s) improve matching/tracking accuracy but linearly increase write load and battery/bandwidth usage on the driver's phone — tune based on product needs (denser urban matching may warrant more frequent pings than sparse rural areas).
- Nearest-driver-first matching is simple and fast but doesn't account for driver preference/rating/vehicle-type fit as richly as a more sophisticated scoring function would — a real trade-off between matching speed and matching quality.

Follow-up Interview Questions

- *"How do you estimate ETA for driver arrival and trip duration?"* — Typically delegated to a mapping/routing service (e.g. built on top of road-network graph data + historical/ real-time traffic data) — worth naming as a distinct subsystem rather than trying to design routing/pathfinding from scratch in this interview.
- *"What happens if a driver goes offline mid-trip (app crash, network loss)?"* — The trip state machine (§21.10) should handle a "driver connection lost" state with a grace period/retry before escalating to rider notification/support intervention, rather than silently failing.

31. Design Ticket Booking (BookMyShow/Ticketmaster)

Problem

Design a system for browsing events (movies/concerts), viewing seat availability, and booking specific seats — with the hard constraint that **no two people can book the same seat**.

Requirements

Functional: (1) Browse events/showtimes. (2) View real-time seat map/availability. (3) Select and book specific seat(s), with payment. (4) Hold a seat temporarily during checkout (so it's not sold to someone else mid-payment). **Non-Functional: Strong consistency required for seat booking** — this is the one place in this system where correctness trumps everything else (unlike most of this guide's read-heavy systems, this is fundamentally a "unique resource allocation" problem, closer in spirit to inventory/payments than to a feed or cache). Must handle huge, sudden demand spikes (popular event on-sale moments — thousands of users hitting "buy" for the same handful of front-row seats within seconds).

Estimation

A single, massively popular on-sale event might see 100,000 concurrent users competing for 10,000 seats within the first minute — the system must gracefully handle **massive contention on a small, fixed set of resources**, which is a fundamentally different shape of scaling problem than the read-heavy fan-out or geospatial-write problems seen elsewhere in this guide — here the bottleneck is **correctness under contention**, not raw throughput.

API Design

```
GET  /api/v1/events/{id}/showtimes/{showtimeId}/seats   -> seat map with status per
seat
POST /api/v1/bookings/hold      { "showtime_id": "...", "seat_ids": [...] }
```

```
                                    -> 201 { "hold_id": "...", "expires_at": "..." }
(e.g. 5-min hold)
POST /api/v1/bookings/{hold_id}/confirm   { "payment_token": "..." } -> 200 {
"booking_id" }
```

The **explicit two-phase flow (hold, then confirm)** is itself a key design decision — it gives the user time to enter payment details without losing the seat to someone else, while still guaranteeing the seat is exclusively theirs during that window.

High-Level Design

```
Client -> LB -> Booking Service -> Seat Lock Manager (strongly consistent, e.g.
Redis w/
                                       atomic ops, or a relational DB row
lock/version check)
                      |
               Payment Service (external gateway) -> on success: confirm booking,
release lock
                                                          permanently as "sold"
                      |
               (on hold expiry / payment failure: release lock back to "available")
```

Database Schema

```sql
CREATE TABLE seats (
    seat_id      BIGINT PRIMARY KEY,
    showtime_id  BIGINT NOT NULL,
    seat_label   VARCHAR(10),   -- e.g. "A12"
    status       VARCHAR(10) NOT NULL DEFAULT 'AVAILABLE', -- AVAILABLE, HELD,
BOOKED
    held_by      BIGINT NULL,
    hold_expires_at TIMESTAMP NULL,
    version      INT NOT NULL DEFAULT 0  -- for optimistic concurrency control
);
CREATE INDEX idx_seats_showtime ON seats(showtime_id, status);
```

Deep Dive: Guaranteeing No Double-Booking Under Massive Contention

This is the entire crux of the interview. Two valid approaches:

1. **Pessimistic locking (row-level lock in a relational DB):**

```
BEGIN;
SELECT * FROM seats WHERE seat_id = ? AND status = 'AVAILABLE' FOR UPDATE;
-- if a row is returned, this transaction now holds an exclusive lock on it
UPDATE seats SET status = 'HELD', held_by = ?, hold_expires_at = now() +
interval '5 min'
  WHERE seat_id = ?;
COMMIT;
```

   `FOR UPDATE` ensures any other transaction trying to select/lock the same row blocks until this one commits — guaranteeing only one request can successfully move a seat from `AVAILABLE` to `HELD`. Simple and correct, but concurrent requests for the *same* seat serialize (queue up on the lock) — fine for reasonable contention levels, since each lock is held only briefly.
2. **Optimistic concurrency control (version check, better under very high read-to-write ratios or when using a store without native row locks):**

```
UPDATE seats SET status = 'HELD', held_by = ?, version = version + 1
  WHERE seat_id = ? AND status = 'AVAILABLE' AND version = ?;
-- check rows_affected == 1; if 0, someone else won the race -- return "seat
no longer available"
```

   No lock is held while "thinking" — the client read the seat's current version, and the update only succeeds if nothing changed it in between; a losing request gets a clear "someone else got it first" response instantly rather than waiting in a lock queue. **Recommended interview answer:** either is valid and defensible — pessimistic locking is simpler to reason about correctness for (recommended default to state first), with optimistic control as a follow-up alternative if the interviewer pushes on "what if lock contention/wait times become a problem at extreme scale?"

Deep Dive: Automatic Hold Expiry (Releasing Abandoned Holds)

A user who holds a seat then abandons checkout (closes the tab) must not permanently lock that seat forever. Options: (a) a background job periodically scans for `status='HELD' AND hold_expires_at < now()` and resets them to `AVAILABLE` (simple, but introduces a polling delay before the seat is freed), or (b) use a TTL-native

store (Redis key with `EXPIRE`, or a scheduled job via a delayed queue) so expiry happens automatically/ near-instantly without a polling scan — preferred at high contention scale since a popular event with thousands of abandoned holds needs those seats freed up fast for other waiting buyers, not stuck until the next polling cycle.

Deep Dive: Handling the On-Sale Traffic Spike (Virtual Waiting Room)

For an extremely popular on-sale (100K users, 10K seats, all hitting "buy" within seconds), even with correct locking, letting all 100K requests hit the booking service simultaneously would overwhelm it. Real systems (Ticketmaster) use a **virtual waiting room**: incoming users are queued (often via a lightweight, high-throughput edge-level queueing service) and admitted to the actual booking flow in controlled batches matching the system's sustainable throughput — smoothing an instantaneous spike into a manageable, steady admission rate. This is conceptually the same "queue absorbs bursts" principle as §10.1, applied to *user admission* rather than backend message processing.

Bottlenecks & Scaling

- **Hot seats table for a popular showtime** — since all contention concentrates on one showtime's small set of seat rows, this doesn't benefit from typical horizontal sharding (sharding by showtime just moves the same hotspot onto one shard) — the waiting room (above) is the actual scaling lever here, not database scaling.
- **Read-heavy seat map browsing** (many more people browsing/viewing the seat map than actually booking) can and should be cached/served from read replicas — only the actual hold/confirm operations need the strong-consistency path.

Trade-offs

- A short hold duration (e.g. 5 min) frees up abandoned seats faster (good for other buyers) but risks frustrating slower users who don't complete payment in time — a genuine UX-vs-fairness trade-off to acknowledge.
- Waiting rooms add user-facing friction/delay but are what makes fair, correct booking possible at extreme spike scale at all — worth framing as a necessary trade-off, not a flaw.

Follow-up Interview Questions

- *"How would you ensure fairness (e.g. first-come-first-served) in the waiting room rather than random/arbitrary admission?"* — Assign an admission ticket/position at waiting-room entry time (e.g. based on arrival timestamp), and admit strictly in that order — requires the waiting room itself to maintain an ordered queue, not just a rate limiter.
- *"How do you handle partial group bookings (a family wants 4 adjacent seats specifically, not just any 4 available seats)?"* — Requires holding all requested seats **atomically together** (all-or-nothing) — extend the locking transaction to acquire all seat locks in one transaction/Lua script, and release all held seats if even one in the requested group isn't available, rather than partially succeeding.

---

32. Design E-commerce Checkout (Amazon)

Problem

Design the checkout flow for an e-commerce platform: cart, inventory check, payment, and order creation — must be correct (no overselling, no double-charging) and resilient (partial failures shouldn't corrupt state).

Requirements

Functional: (1) Add items to cart. (2) Checkout: validate inventory, calculate total (price + tax + shipping - discounts), charge payment, create an order, decrement inventory. (3) Handle payment failure/partial failure gracefully (don't charge without an order, don't create an order without a successful charge, don't oversell inventory). **Non-Functional:** Strong consistency for inventory and payment (this is a "money and unique resource" problem like §31, not a read-heavy content problem). High availability (checkout is the revenue-critical path). Must survive partial failures across the multiple services typically involved (inventory, payment, order, shipping) without ending up in an inconsistent state (e.g. charged but no order created).

Estimation

Assume 10M orders/day, with peak traffic (flash sale/Black Friday) at 20-50x average — checkout systems must be provisioned and architected for extreme peak-to-average ratios far beyond most other systems in this guide, similar in spirit to the ticket-booking spike problem (§31) but sustained over hours instead of seconds.

API Design

```
POST /api/v1/cart/items            { "product_id": "...", "qty": 2 }
POST /api/v1/checkout               { "cart_id": "...", "payment_method": "...",
                                      "idempotency_key": "..." }
                                    -> 201 { "order_id": "...", "status":

"CONFIRMED" }
GET  /api/v1/orders/{id}
```

The `idempotency_key` (§12.5) on checkout is **non-negotiable** — a client-side retry (e.g. after a timeout, unsure if the first request succeeded) must never risk double-charging or double-ordering.

High-Level Design — This Is Fundamentally a Distributed Transaction Problem

```
Client -> Checkout Service (orchestrator)
            |-- 1. reserve inventory ---------> Inventory Service (decrement
stock, or fail if insufficient)
            |-- 2. charge payment ------------> Payment Service (external gateway)
            |-- 3. create order --------------> Order Service
            |-- 4. (async) trigger shipping --> Fulfillment Service
```

This multi-step, multi-service flow is a textbook case for the **Saga pattern** (§8.8, §13.7) rather than a classic 2PC distributed transaction (2PC's blocking nature and requirement that all participants be simultaneously available doesn't fit well with calling an *external* payment gateway, which the checkout service doesn't control).

Deep Dive: The Checkout Saga (Orchestration Style)

A central **Checkout Orchestrator** explicitly drives the sequence and handles failure at each step with a **compensating action**:

1. **Reserve inventory** (atomically decrement stock only if sufficient — same optimistic/pessimistic pattern as seat booking, §31 — mark as "reserved," not yet "sold," pending payment). *Compensation if a later step fails:* release the reservation back to available stock.
2. **Charge payment** via the external gateway, using the same `idempotency_key` passed through so a retried call to the *gateway itself* (e.g. due to a timeout on the checkout service's side) doesn't double-charge — this only works if the payment gateway supports idempotency keys itself (major real ones do). *Compensation if a later step fails:* issue a refund/reversal.
3. **Create the order record** (source of truth that payment succeeded and what was bought). *This step should be designed to essentially never fail* (a simple DB insert) — if it somehow does, compensate by refunding (step 2's compensation) and releasing inventory (step 1's compensation).
4. **Trigger fulfillment** — asynchronous, via a queue (§10) — failure here doesn't need to unwind the whole order (the order is already valid/paid; fulfillment can be retried independently without touching payment/inventory at all). Each step's compensating action must itself be **idempotent** (in case the compensation itself is retried) — the same idempotency discipline threaded through every layer.

Deep Dive: Preventing Overselling (Inventory Reservation)

Exactly analogous to seat-booking (§31): use an atomic conditional decrement (`UPDATE inventory SET stock = stock - ? WHERE product_id = ? AND stock >= ?`, checking `rows_affected == 1`) rather than a naive read-then-write, to prevent two concurrent checkouts from both succeeding when only one unit of stock remains. At very high contention (a single viral product during a flash sale), the same "hot row" problem as §31 applies — mitigated by the same techniques (short critical sections, or a queue-based admission control for extreme spikes).

Deep Dive: Handling Partial Failures Without Corrupting State

The single biggest risk in this system is ending up "charged but no order" or "order but never charged" — both are unacceptable. The Saga's compensating-action discipline (above) addresses the *logical* flow, but the orchestrator itself can crash mid-saga — solved by **persisting saga state** (which step it was on) durably (e.g. in a dedicated saga-state table) before each step, so a recovering orchestrator (or a monitoring job) can detect an **incomplete saga** (e.g. inventory reserved + payment charged, but no order created within a reasonable time) and resume/compensate it correctly rather than leaving it stuck indefinitely — this durable, resumable saga state is what separates a production-grade design from a "happy path only" one, and is a strong thing to proactively mention.

Bottlenecks & Scaling

- **Hot product inventory rows** during flash sales — same mitigation family as §31 (short critical sections, potentially a virtual queue for extreme spikes on a single hot SKU).
- **External payment gateway latency/availability** is outside your control but sits on the critical checkout path — use timeouts + circuit breakers (§6.6) so a slow/down payment gateway degrades gracefully (clear "try again" message) rather than exhausting your own service's thread/connection pool.

Trade-offs

- Orchestration-style saga (a central coordinator) is easier to reason about/debug the overall flow (one place shows the full sequence) versus choreography (decentralized, event-driven) which scales team ownership better but is harder to trace end-to-end — for a checkout flow specifically (revenue-critical, needs strong debuggability), orchestration is generally the safer default answer.

Follow-up Interview Questions

- *"How would you handle a payment that succeeds on the gateway's side but the response times out before your service receives it?"* — Never blindly retry a raw charge call without an idempotency key (§12.5) for exactly this reason; on ambiguous timeout, query the gateway's transaction-status endpoint (most support this) before deciding whether to retry or treat as succeeded.

- *"How do you handle cart abandonment and stock reservation cleanup?"* — Same TTL/expiry pattern as seat holds (§31) — a reservation not converted to a completed order within a window (e.g. 15 minutes) is automatically released back to available stock.

---

33. Design Search Autocomplete/Typeahead

Problem

Design a system that suggests search query completions as a user types (e.g. typing "sys" suggests "system design", "system administrator", ...), ranked by popularity/relevance, with very low latency (must feel instant on every keystroke).

Requirements

Functional: (1) Given a prefix, return the top-K most likely completions. (2) Rankings should reflect real query popularity (trending/popular queries surface first). (3) Update suggestions over time as query patterns change (yesterday's popular query may not be today's). **Non-Functional:** Extremely low latency (<50-100ms, ideally single-digit ms, since it fires on *every keystroke*, not just on submit). Very high read QPS (every character typed by every user is a request). Suggestions can be slightly stale (don't need to reflect the literal most recent second's query — eventual consistency is perfectly fine here, unlike §31/§32).

Estimation

If autocomplete fires on every keystroke for a search box used by 500M DAU doing ~5 searches/day (~5 characters typed per search on average before selecting a suggestion):

```
Autocomplete request QPS ≈ 500M × 5 searches × 5 keystrokes / 86,400 ≈ 145,000/sec
average,
  with much higher peaks -- this is an enormous read QPS number that must be served
```

```
with
  very low, consistent latency
```

Conclusion: the sheer read QPS combined with the strict latency requirement means the serving path **must** be a precomputed, in-memory data structure — there is no room for a database query (even a cached one with network hop) on this hot path; this needs to be closer to "in-process/edge-cached lookup" territory.

API Design

```
GET /api/v1/autocomplete?prefix=sys&limit=10
    -> { "suggestions": ["system design", "system administrator", "systemd", ...] }
```

A single, simple, read-only GET endpoint — the complexity is entirely in how the backend serves it fast, not in the API surface itself.

High-Level Design

```
Query Logs (from the main search service) -> Aggregation Pipeline (offline/batch or
                                               streaming, computes query frequency
counts)
                                                       |
                                                Trie Builder (builds a precomputed Trie
annotated
                                                 with top-K completions + frequency at
each node)
                                                       |
                                                Trie snapshots pushed to Autocomplete
Servers
                                                 (serve entirely from in-memory Trie,
sharded/
                                                  replicated across many instances
behind a CDN/
                                                  edge cache for the most common
prefixes)
```

- **Offline aggregation:** query logs are periodically (e.g. every few minutes to hourly) aggregated into frequency counts per query string — this doesn't need to be real-time (staleness of minutes-to-an-hour is fine, per the NFRs above).
    - Doing this in real time for every keystroke would be wasteful; batch/streaming aggregation (e.g. a scheduled Spark/Flink job, or a

streaming aggregation over a Kafka topic of search queries) is the standard, correct-scope answer.

- **Trie construction:** build a Trie (prefix tree) where each node represents a character, and each node caches its **top-K most frequent completions** in its subtree (precomputed during Trie construction, not computed at query time) — so a lookup for a given prefix is just a Trie traversal to that prefix's node followed by returning its precomputed top-K list, an O(prefix length) operation with no further ranking computation needed at serve time.
- **Serving:** the built Trie (or a sharded/partitioned version of it, e.g. by first character or first few characters, if it's too large for one machine's memory) is loaded entirely into memory on a fleet of autocomplete servers, refreshed periodically (e.g. every few minutes to an hour) by loading the latest Trie snapshot — trading a small amount of staleness for extremely fast, computation-free serving.

Deep Dive: Building & Updating the Trie Efficiently

Rebuilding the entire Trie from scratch on every update cycle works fine at moderate scale but can become slow/wasteful at extreme scale — real systems often use an **incremental/merge approach**: maintain the aggregated frequency counts in a scalable store (e.g. a wide-column DB or a simple key-value counter store), and periodically (re)build the Trie as a batch job that reads current counts and constructs a fresh, immutable Trie snapshot — then **atomically swap** the old in-memory Trie for the new one on each serving instance (e.g. build in the background, swap a pointer once ready) so there's no partial/inconsistent state visible to readers during a refresh, and no need to lock/pause serving during a rebuild.

Deep Dive: Handling Trending/Real-Time Spikes (Breaking News Query)

The batch-refreshed Trie (minutes-to-hours stale) won't reflect a suddenly-trending query (e.g. breaking news) fast enough for some products' needs. A common augmentation: run a **separate, lightweight real-time counting layer** (e.g. a sliding-window count in Redis per query string, updated as queries stream in) for very recent activity, and **blend** its top results with the precomputed Trie's results at serve time (e.g. boost/merge in any query showing an unusually sharp recent spike) — this is an

optional refinement worth mentioning if the interviewer asks about freshness/trending, without over-complicating the baseline design.

Bottlenecks & Scaling

- **Memory size of the full Trie** at web-scale query vocabulary (potentially many millions of distinct query strings) — mitigate by only keeping the trie for say, the top N million most-searched prefixes/queries (long-tail extremely rare queries don't need autocomplete support anyway), and/or sharding the Trie across multiple servers by prefix range.
- **Serving latency tail** — because this is on the critical "instant feedback" path, put an edge/CDN cache in front of the most common prefixes (a small number of very short, very common prefixes like "a", "th", "wh" account for a disproportionate share of traffic) to shave off even the in-memory-server round trip for the hottest queries.

Trade-offs

- Precomputed Trie: extremely fast reads, but staleness on the order of the refresh interval, and rebuild cost scales with vocabulary size — versus a live/on-demand ranking computation (essentially never chosen here) which would be fresh but far too slow for the per-keystroke latency budget.
- Global top-K per prefix (same suggestions for everyone) is far simpler than personalized autocomplete (factoring in a specific user's own search history/location) — explicitly scope which one you're building, since personalization multiplies complexity substantially (a per-user or per-user-segment ranking layer, not just one global Trie).

Follow-up Interview Questions

- *"How would you personalize suggestions per user (e.g. boost their own frequent past searches)?"* — Blend the global Trie's precomputed top-K with a small, per-user recent-query cache (Redis, keyed by user ID, small bounded size) at serve time — merge and re-rank the two result sets rather than replacing the global system entirely.
- *"How do you filter out offensive/inappropriate autocomplete suggestions?"* — Maintain a blocklist checked at Trie-build time (offline, doesn't affect serving

latency) — filter or suppress flagged terms before they ever enter the precomputed suggestion set, rather than trying to filter at serve time.

34. Design a Distributed Cache (Redis)

Problem

Design a distributed, in-memory key-value caching system (essentially, design the core of something like Redis/Memcached) that supports fast get/set operations, scales horizontally, and handles node failures gracefully.

Requirements

Functional: (1) `GET(key)`, `SET(key, value, ttl)`, `DELETE(key)`. (2) Data distributed across multiple nodes (doesn't fit on one machine's memory at scale). (3) Support for common eviction (§9.4) when memory is full. **Non-Functional:** Very low latency (sub-millisecond). High availability (a single node failure shouldn't cause a full outage or excessive data loss). Horizontal scalability (add/remove nodes as the working set grows).

Estimation

A cache holding 500GB of hot data with a per-node memory budget of ~50GB (leaving headroom for OS/overhead) needs at least 10 nodes just for capacity, and likely more when accounting for replication factor — this is inherently a **multi-node system by design**, not something that starts on one box and grows later (unlike, say, the URL shortener's DB).

API Design (Client-Facing Protocol, Not HTTP)

Cache clients typically use a lightweight binary or simple text protocol (like Redis's RESP protocol) over a persistent TCP connection for minimal per-request overhead — not a REST/JSON API, since even JSON parsing overhead matters at sub-millisecond latency budgets:

```
SET key value [EX seconds]
GET key
DEL key
```

High-Level Design

```
Client (with client-side hashing logic, or a smart client library)
    -> hash(key) via Consistent Hashing (§14) -> determine owning node
    -> direct connection to that specific cache Node

Cache Node:
  - In-memory hash table (the actual key-value store)
  - Eviction policy (LRU/LFU, §9.4) when memory limit reached
  - Replication to 1+ replica nodes (for durability/failover)
  - Persistence (optional): periodic snapshot (RDB-style) and/or append-only log
(AOF-style)
    for recovery after a restart -- a cache doesn't strictly need this, but many
real
    deployments want faster warm-up after a planned restart
```

A key architectural decision: does the **client** know the cluster topology and hash directly to the right node (client-side sharding, lower latency — no extra hop, but every client needs cluster-topology awareness and must handle topology changes), or does a **proxy/router layer** sit in front and forward requests to the right node (simpler clients, but adds a network hop and the proxy itself must scale/be highly available)? Real systems support both patterns (Redis Cluster uses smart clients with redirect-based topology discovery; Twemproxy is a classic example of the proxy approach).

Deep Dive: Partitioning the Keyspace

Use **consistent hashing with virtual nodes** (§14) so that adding/removing a cache node only remaps a small fraction of keys rather than nearly all of them. Redis Cluster's actual approach is a close relative: a fixed **16,384 hash slots**, each key mapped to a slot via `CRC16(key) % 16384`, and slots are assigned to nodes (not keys directly) — rebalancing means moving whole slots between nodes rather than recomputing a full hash ring, a practical middle ground between "modulus by node count" (bad, reshuffles everything) and "per-key consistent hashing ring" (great in theory, this fixed-slot approach is an engineering simplification that gets most of the same benefit with simpler operational tooling).

Deep Dive: Replication & Failover

Each cache node has one or more **replicas** (asynchronous replication is standard here — waiting for synchronous replica acknowledgment on every cache write would defeat the purpose of using a cache for speed). On primary node failure: a **failure detector** (heartbeat-based, often using a gossip protocol, §15.5, among cluster nodes themselves to reach consensus that a node is actually down and not just slow/partitioned) triggers **promotion** of a replica to primary — clients (or the proxy layer) are then redirected to the newly-promoted node. Because replication is async, a small window of recently-written data can be lost on failover (an accepted trade-off for a cache — the source of truth should always be the underlying database, never the cache alone, precisely because of this possibility).

Deep Dive: Eviction Under Memory Pressure

When a node's memory limit is reached, it must evict something to make room for new writes — using LRU or LFU (§9.4) as the policy, implemented efficiently: an approximate LRU (sampling a small random subset of keys and evicting the oldest-accessed among the sample, rather than maintaining a perfectly precise global LRU order, which would require an expensive doubly-linked-list update on every single access) is what Redis actually uses in practice — a great example of trading a small amount of eviction precision for significantly better performance at scale, worth mentioning explicitly as a real-world engineering trade-off.

Bottlenecks & Scaling

- **Hot keys** (a single extremely popular key, e.g. a viral post's like-count) can overwhelm the single node that owns it even if the cluster overall is well-balanced — mitigate with **local (in-process) caching of hot keys on top of the distributed cache** (a small L1 cache inside each app server for the tiniest handful of the very hottest keys) or by explicitly replicating/sharding that specific key's value across multiple nodes and having clients round-robin between the copies.
- **Cluster resharding operations** (adding capacity) should move data in the background without blocking live traffic — slot/key migration is done incrementally with the source node still serving reads/writes for not-yet-migrated keys during the transition.

Trade-offs

- Async replication (fast, but small data-loss window on failover) vs sync replication (safer, but adds latency to every write, defeating much of the point of an in-memory cache) — async is almost always the right choice specifically *because* it's a cache (recoverable from the source-of-truth DB), not a database itself.
- In-memory only vs optional persistence: persistence enables faster cluster restart/recovery (warm cache immediately) at the cost of disk I/O overhead — a reasonable addition but not a core requirement of "cache" as a concept.

Follow-up Interview Questions

- *"How is this different from designing the database itself (why isn't a cache just a small, fast database)?"* — A cache explicitly trades some durability/consistency guarantees for speed (async replication, approximate eviction, optional persistence, no complex transactions/joins) — because it always sits in front of a real source-of- truth database that can rebuild any lost cache state, unlike a primary database which cannot make that trade-off.
- *"How would you implement pub-sub (like Redis Pub/Sub) on top of this same infrastructure?"* — A separate mechanism from the key-value store itself: maintain per-topic subscriber lists on each node and forward published messages to all subscribed connections (potentially across nodes via internal node-to-node forwarding) — notably **not durable** (messages aren't stored for later delivery, matching the "at-most-once, fire-and-forget" semantics discussed in §10.4).

---

35. Design Distributed File Storage (Dropbox/Google Drive)

Problem

Design a cloud file storage and synchronization system: users upload files/folders, access them from multiple devices, and changes sync automatically across all of a user's devices.

Requirements

Functional: (1) Upload/download files. (2) Sync automatically across a user's multiple devices. (3) Folder hierarchy support. (4) File versioning (undo/history). (5) Sharing files/folders with other users. **Non-Functional:** Handle very large files efficiently (don't re-upload/re-download an entire large file for a tiny change). Bandwidth-efficient sync (mobile/limited connections). Strong eventual consistency across devices (all devices converge to the same state, with sensible conflict resolution when the same file is edited offline on two devices).

Estimation

Assume 100M users, average 5GB of files each, average file change rate low (most files don't change often once uploaded):

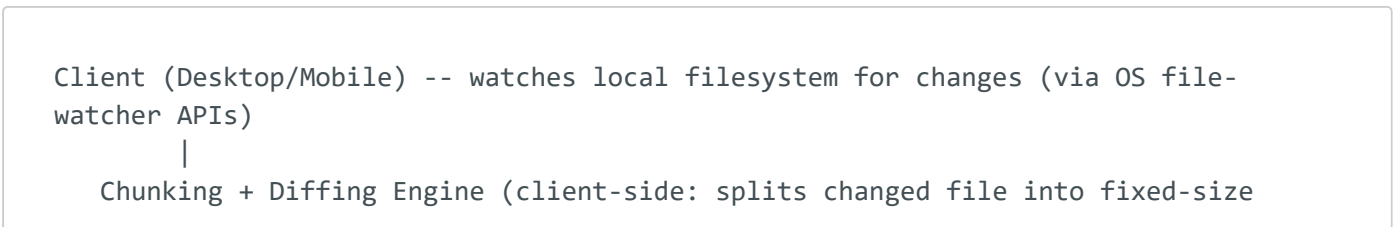
```
Total storage ≈ 100M × 5GB = 500 PB (before any replication factor) -- clearly
requires
  a globally distributed object storage system (§18), not a single storage cluster
```

Conclusion: storage volume alone confirms object storage (§18.1) as the content store; the *actual* hard problem here is efficient **sync and change detection**, not raw storage capacity (which off-the-shelf object storage already solves).

API Design

```
POST /api/v1/files/upload          (chunked, resumable — see Deep Dive below)
GET  /api/v1/files/{id}/download
GET  /api/v1/sync/changes?since=<cursor>   -> list of changed files since last sync
(delta sync)
POST /api/v1/files/{id}/share       { "target_user_id": "..." }
```

High-Level Design



```
blocks,
                                computes a hash per block)
        |
   Sync Client -> Metadata Service (which blocks does the server already have?
which are
                                      new/changed?) -> uploads only NEW/CHANGED
blocks
        |
   Block Storage (object storage, deduplicated -- identical blocks across different
files/
                   versions/users stored once, referenced by hash)
        |
   Metadata DB (file/folder hierarchy, per-file list of block hashes composing it,
version
                 history, per-user/per-device sync state)
        |
   Notification Service -> other devices notified of the change (push, so they pull
the
                            delta and stay in sync without polling)
```

Deep Dive: Chunking, Block-Level Deduplication, and Delta Sync

Splitting files into fixed-size (or content-defined, see below) chunks (§18.4) is the core efficiency mechanism: when a file changes, only the chunks that actually changed need to be re-uploaded/re-downloaded (not the entire file) — critical for large files (a 2GB video edited slightly shouldn't require re-transferring 2GB). Each chunk is identified by a **content hash** (e.g. SHA-256 of its bytes); the metadata service stores, per file, the **ordered list of chunk hashes** that compose it. On upload, the client first sends just the list of chunk hashes for the new/changed file version; the server responds with which of those hashes it **doesn't already have**, and the client uploads only those — this achieves both **delta sync** (only changed parts) and **deduplication** (if a chunk's content hash already exists anywhere in the system — even from a different file or a different user — it's never stored twice, just referenced again). **Fixed-size vs content-defined chunking:** fixed-size chunking (e.g. every 4MB) is simple, but has a subtle problem — inserting even one byte near the start of a file shifts every subsequent chunk boundary, making nearly all "chunks" appear changed even though most content didn't actually change. **Content-defined chunking** (using a rolling hash, e.g. Rabin fingerprinting, to determine chunk boundaries based on the content itself rather than a fixed byte offset) makes chunk boundaries resilient to small insertions/deletions elsewhere in the file — a small edit only affects the chunks immediately around it, not the entire rest of the file's chunk boundaries. This is a genuinely impressive, senior-level deep-dive point to raise.

Deep Dive: Efficient Multi-Device Sync (Avoiding Polling)

Rather than every device periodically polling "anything changed?" (wasteful, adds latency), use a **push-based notification** (similar mechanism to §26/§28): when a change is committed, the server notifies all of the user's *other* active devices (via a long-lived connection or push notification) that a change occurred; devices then pull the actual delta via a **cursor-based sync endpoint** (`/sync/changes?since=<cursor>`, exactly the same pattern as feed sync §27 and chat sync §28 — a recurring, reusable pattern across very different domains) — the notification itself carries no payload beyond "something changed, go sync," keeping the push mechanism lightweight regardless of how large the actual change is.

Deep Dive: Conflict Resolution (Concurrent Offline Edits)

If a user edits the same file on two devices while offline, then both reconnect, the system must detect and resolve the conflict. Standard approach: track a **version vector/vector clock per file** (§15.6) or simply a monotonically increasing version number per file, updated on each sync; if both devices' pending changes are based on the *same* prior version (no conflict, standard case — just apply in order), but if two divergent edits both claim to be "based on version N" and both arrive, the system detects a **true conflict** and resolves it typically by: keeping both versions as separate files (e.g. `document.docx` and `document (conflicted copy from DeviceB).docx`) rather than silently picking a winner and losing the other user's work — data loss avoidance trumps automatic merging for opaque binary file content (unlike, say, a CRDT-based collaborative text editor which *can* auto-merge because it understands the content's structure).

Bottlenecks & Scaling

- **Metadata service** (file hierarchy, chunk lists, version history) is relational/graph-like in nature (folders contain files/folders) and can become a bottleneck at scale — typically sharded by user ID (a user's entire file tree lives together, matching the dominant access pattern of "browse my files").

- **Small-file overhead**: many tiny files each needing their own chunk-hash round trip before upload adds per-file overhead — batch metadata checks for many small files into a single request where possible.

Trade-offs

- Content-defined chunking is more compute-intensive client-side (computing a rolling hash) than fixed-size chunking, but dramatically reduces bandwidth for the common case of small edits to large files — the right trade-off for this specific product, less clearly so for a system where files are typically replaced wholesale rather than incrementally edited.
- Keeping conflicted copies rather than auto-merging is safer (no silent data loss) but pushes the resolution burden onto the user — an explicit, defensible product trade-off to state rather than silently assume.

Follow-up Interview Questions

- *"How would you support very large team/shared drives with thousands of collaborators?"* — Separate the sharing/permission model (an ACL-like table: file/folder → user/group → permission level) from the sync mechanism itself — sync logic doesn't need to change, but the metadata service needs an efficient way to check "can this user see this file" at read time, likely via a cached permission-check layer given how frequently it's invoked.
- *"How do you handle a file that's actively being written to (e.g. a large video export in progress) — do you sync partial/in-progress writes?"* — Detect file stability (e.g. wait for a period of no modification, or watch for a corresponding "file close" OS event) before triggering a sync, to avoid syncing a corrupt/incomplete intermediate state.

---

36. Design a Payment/Wallet System

Problem

Design a digital wallet/payment system that lets users hold balances, transfer money to other users, and make payments — with absolute correctness guarantees (money

must never be created or destroyed by a bug).

Requirements

Functional: (1) Users have a wallet balance. (2) Transfer money between users (debit one, credit another). (3) Top-up (add funds via external payment method) and withdraw. (4) Full transaction history/audit trail. **Non-Functional: Absolute correctness — the single hardest non-functional requirement in this entire guide.** Every transaction must be atomic (money is never partially deducted from one account without being credited elsewhere), durable (a confirmed transaction is never lost), and fully auditable (every balance change traceable to a specific transaction, for compliance and dispute resolution). Idempotent (network retries must never cause duplicate transfers). Available (but correctness trumps availability here far more than in almost any other system in this guide — an unavailable wallet is better than a wallet that loses/duplicates money).

Estimation

Financial systems are typically **not** the highest-QPS system in a company's stack (orders of magnitude lower QPS than, say, a news feed) — the design challenge here is not raw throughput but **correctness under concurrency and failure**, making this conceptually closer to the ticket-booking (§31) and checkout (§32) deep dives than to the high-QPS systems earlier in this guide.

API Design

```
POST /api/v1/wallets/{id}/transfer
{ "to_wallet_id": "...", "amount": 5000, "currency": "USD", "idempotency_key":
"..." }
-> 201 { "transaction_id": "...", "status": "COMPLETED" }
GET  /api/v1/wallets/{id}/balance
GET  /api/v1/wallets/{id}/transactions?cursor=...
```

`amount` in **integer minor units** (cents), never a floating-point dollar amount — a foundational, non-negotiable rule for any financial system (floating-point rounding errors are unacceptable when real money is involved).

High-Level Design

```
Client -> Payment API -> Ledger Service (the single source of truth for all balance
changes)
                              |
                    Relational DB (strongly consistent, ACID transactions,
§8.6/8.8)
                              |
                    Double-Entry Ledger table (append-only, immutable)
```

Unlike almost every other case study in this guide, this system should **not** reach for NoSQL/eventual consistency/async processing for the core balance-changing operation — a traditional ACID-compliant relational database, used correctly, is the right tool here. This is worth stating explicitly as a deliberate, justified choice (not a default) — "given the correctness requirements, I'm deliberately choosing strong consistency and ACID transactions over the eventually-consistent/high-throughput patterns used elsewhere, because a wrong balance is unacceptable in a way that a stale feed or slightly-delayed notification is not."

Database Schema — Double-Entry Bookkeeping

The single most important design decision in this entire case study: **never store a simple mutable "balance" column that's directly incremented/decremented.** Instead, use double-entry bookkeeping (borrowed from real-world accounting): every transaction creates **two ledger entries** — a debit from one account and a credit to another, whose amounts always sum to zero — and the current balance is always **derived** (sum of all ledger entries for that account), never stored as a single mutable field that a bug could corrupt independently of the transaction history.

```
CREATE TABLE ledger_entries (
    entry_id      BIGINT PRIMARY KEY,
    transaction_id BIGINT NOT NULL,   -- groups the debit+credit pair
    account_id    BIGINT NOT NULL,
    amount        BIGINT NOT NULL,    -- signed, in minor units; negative = debit,
positive = credit
    created_at    TIMESTAMP NOT NULL,
    idempotency_key VARCHAR(64) UNIQUE  -- prevents duplicate transaction
processing
);
```

```
-- balance for an account = SELECT SUM(amount) FROM ledger_entries WHERE account_id
= ?
```

This design makes the system **self-auditing by construction**: the sum of all entries across all accounts for any transaction must always be zero (money is conserved), and the full history of every balance change is inherently preserved (it's never overwritten, only appended to) — a corruption bug would have to violate an explicit, checkable invariant rather than silently producing a wrong number.

Deep Dive: Atomicity of a Transfer

A transfer (debit account A, credit account B) must happen **atomically** — using a single database transaction wrapping both ledger entry inserts:

```
BEGIN;
INSERT INTO ledger_entries (transaction_id, account_id, amount, idempotency_key)
  VALUES (?, 'A', -5000, ?);
INSERT INTO ledger_entries (transaction_id, account_id, amount, idempotency_key)
  VALUES (?, 'B', 5000, ?);
-- also check account A's derived balance >= 5000 before allowing this, within the
same
-- transaction, to prevent overdraft (unless overdraft is explicitly a supported
feature)
COMMIT;
```

If either insert fails (or the balance check fails), the entire transaction rolls back — neither entry is applied, guaranteeing the system is never left in a half-completed state where money vanished from one account without appearing in the other.

Deep Dive: Idempotency for Financial Operations (Critical, Not Optional)

Every transaction request carries a **client-supplied idempotency key** (§12.5), stored with a `UNIQUE` constraint — if a network retry resends the same logical transfer request (same idempotency key), the unique constraint violation is caught and the system returns the **original** transaction's result instead of processing a second transfer. This is not an optional nice-to-have here — it is the single most important correctness mechanism in the whole system, given that any HTTP client can and will retry on timeout, and a naive retry without this protection could double-transfer real money.

Deep Dive: Handling Cross-Shard/Cross-System Transfers (Scaling Beyond One DB)

At extreme scale, a single relational DB instance may not sustain the transaction volume, requiring sharding accounts across multiple DB instances (e.g. by account ID hash) — but this reintroduces the **cross-shard transaction problem** (§8.5, §8.8): a transfer between accounts on different shards can no longer use a single local ACID transaction. Solution: the **Saga pattern** (§13.7), same as checkout (§32) — reserve/debit on shard A, then credit on shard B, with a compensating reversal if the credit step fails — combined with the durable saga-state tracking discussed in §32's deep dive, since an incomplete cross-shard transfer is an even more serious failure mode here than an incomplete order.

Bottlenecks & Scaling

- **Single-account hot spots** (a popular merchant account receiving huge transaction volume) — since correctness requires serializing writes to a given account's balance computation/overdraft-check, an extremely busy account can become a bottleneck; mitigate with careful transaction isolation tuning and, at extreme scale, techniques like sharding a single logical account's ledger entries while maintaining strict ordering guarantees for balance-affecting operations.
- **Read replicas** can serve transaction history/statement queries (read-only, doesn't need the strictest consistency) while all balance-affecting writes go through the primary — a good example of applying different consistency requirements to different operations on the *same* data, echoing the mixed-consistency principle from §7.5.

Trade-offs

- Strong consistency/ACID transactions cost throughput/latency compared to the eventually- consistent patterns used elsewhere in this guide — an explicit, justified trade-off given the domain, not a default choice, and worth stating as such.
- Double-entry ledger (append-only, derived balances) costs more storage and read-time computation (summing entries) than a simple mutable balance column — mitigated in practice with periodic **balance snapshots** (materialized, cached current balance, refreshed on each transaction) so reads don't need to re-sum the

entire history every time, while the immutable ledger remains the source of truth for audits/recovery.

Follow-up Interview Questions

- *"How would you detect and prevent fraud (e.g. unusual transfer patterns)?"* — A separate, asynchronous fraud-detection pipeline (consuming a stream of completed transactions, scoring them against rules/ML models) that can flag/freeze accounts for review — kept deliberately **out of the synchronous transfer path** (fraud scoring shouldn't add latency/failure risk to the core, already-correctness-critical transfer flow).
- *"How do you reconcile your ledger against an external payment processor's records (e.g. Stripe) to catch discrepancies?"* — A scheduled reconciliation job comparing your ledger's recorded transactions against the external processor's transaction log/webhook events, flagging any mismatches for manual/automated investigation — a standard, expected practice in real financial systems that's worth naming even briefly.

37. Design a Distributed Unique ID Generator

Problem

Design a service that generates globally unique IDs at high throughput across a distributed system, without a single central bottleneck (e.g. an auto-increment column doesn't scale across multiple database shards/services).

Requirements

Functional: (1) Generate a unique ID on demand. (2) IDs should ideally be **roughly time-sortable** (newer IDs are numerically larger — useful for default sort order, pagination, and index locality). (3) Compact representation (a 64-bit integer is far more efficient in storage/indexes than a 128-bit UUID string). **Non-Functional:** Very high throughput (potentially millions of IDs/sec across the whole system). No single point of

failure/bottleneck. No coordination required between ID generator instances for every single ID (that would defeat the purpose of distributing it).

Estimation

A large-scale system might need to generate 1M+ unique IDs/sec across all its ID-generating services combined (every new post, message, order, etc. needs one) — this must be handled by **many independent generator instances working without synchronous coordination per ID**, since any design requiring a network round trip to a central coordinator for every single ID would itself become the bottleneck this system exists to avoid.

API Design

Typically an internal library/service call, not a public REST API:

```
long id = IdGenerator.nextId();   // called in-process, extremely low latency, no
network call
```

High-Level Design: Snowflake-Style ID Generation (The Standard Answer)

The widely-used approach (originated at Twitter, adopted broadly) is to compose a 64-bit ID from several bit-fields that together guarantee uniqueness **without any coordination between generator instances at ID-generation time**:

```
| 1 bit (unused/sign) | 41 bits: timestamp (ms since custom epoch) |
| 10 bits: machine/worker ID | 12 bits: sequence number (per-ms, per-worker) |
```

- **Timestamp (41 bits)** — milliseconds since a custom epoch (not Unix epoch, to maximize the usable range) — this is what makes IDs roughly time-sortable: an ID generated later has a larger timestamp component, and since it's the most significant bits, it dominates the numeric ordering.
- **Machine/Worker ID (10 bits)** — a unique identifier assigned to each ID-generator instance (up to 1024 concurrent workers) — this is the field that guarantees

uniqueness *across* machines without those machines ever needing to talk to each other; each worker's ID space is disjoint from every other worker's by construction.
- **Sequence number (12 bits)** — a per-worker, per-millisecond counter (up to 4096 IDs per worker per millisecond) that guarantees uniqueness for multiple IDs generated by the *same* worker within the *same* millisecond — reset to 0 at the start of each new millisecond. This combination guarantees **global uniqueness with zero coordination on the hot path** — each worker independently generates IDs using only its own local clock, its own pre-assigned worker ID, and its own local counter.

Deep Dive: Assigning Unique Worker IDs

The one piece of coordination this design *does* need is assigning each generator instance a **unique worker ID** (0-1023) — but this happens rarely (once per instance startup, not per ID generated), so it can afford to use a proper coordination service: **ZooKeeper/etcd** (§15.4) is the standard answer — each worker instance, on startup, claims an available worker ID slot (e.g. via an ephemeral znode/key, released automatically if the worker crashes/disconnects) from a small, well-known coordination namespace. This keeps coordination overhead vanishingly small (happens on the order of "once per server restart," not "once per generated ID").

Deep Dive: Clock Skew / Clock Going Backwards

Because the timestamp component is central to both uniqueness (combined with the sequence number) and sort-order guarantees, a worker's system clock moving **backwards** (e.g. due to an NTP correction) is a genuine risk — it could generate an ID that collides with (or sorts before) an ID it already generated moments before the clock adjustment. Standard mitigation: each worker tracks the last timestamp it used; if the current clock reading is **less than** the last used timestamp, the worker refuses to generate new IDs until the clock catches up (a brief stall, logged/alerted as an operational event) rather than risk generating a duplicate/out-of-order ID — correctness over availability for this specific, rare edge case.

Alternative Approaches (Know Them, But Justify Preferring Snowflake)

- **UUID (v4, random)** — trivially unique with no coordination at all, but 128 bits (larger storage/index footprint), not time-sortable (random — causes poor index locality/ B-tree page splits when used as a primary key in high-insert-rate tables), and not human-comparable for debugging.
- **Database auto-increment** — simple and naturally sortable, but a single point of bottleneck/failure and doesn't work at all once you have multiple independent databases/shards each wanting to generate IDs (no coordination between separate auto-increment sequences).
- **Centralized ID-generation service (ticket server)** — a dedicated service (e.g. backed by a DB sequence) handing out ID ranges/blocks to callers (similar to the counter range-allocation approach in §22's deep dive) — works, but adds a network hop per range-request (though not per-ID, if ranges are batched) and the service itself, while scalable, is an additional piece of infrastructure to run and keep highly available. **Recommended interview answer:** Snowflake-style generation for anything needing high throughput + sortability + zero per-ID coordination (the overwhelmingly common real-world choice); UUIDs for simplicity when sortability/compactness don't matter and you want zero infrastructure at all.

Bottlenecks & Scaling

- Fundamentally, this design has **no runtime bottleneck** by construction (each worker operates fully independently) — the only scaling concern is having enough distinct worker-ID slots (1024 in the classic 10-bit scheme; extend the bit allocation, e.g. fewer sequence bits for more worker-ID bits, if you need more concurrent workers than that, trading off max IDs/ms/worker for more total workers).

Trade-offs

- 12 sequence bits allow 4096 IDs/ms/worker (4.096M IDs/sec/worker) — more than enough for virtually any single service instance; the bit-width split (timestamp/worker/sequence) is tunable based on the actual expected worker count and per-worker throughput needs.

- Roughly-sortable (not strictly monotonic across different workers, since two different workers could generate IDs in the same millisecond in either order relative to each other) — "roughly time-ordered" is the honest, correct claim, not "strictly ordered."

Follow-up Interview Questions

- *"Why not just use the database's own auto-increment feature?"* — Doesn't scale once there's more than one database/shard generating IDs independently (each would restart or overlap its own sequence), and centralizes ID generation behind a single DB's throughput ceiling — directly answered by the "Alternative Approaches" comparison above.
- *"How would you make IDs strictly (not just roughly) time-ordered across all workers?"* — Would require actual coordination (e.g. all workers checking in with a central sequencer for real-time ordering), which reintroduces the exact bottleneck this design is built to avoid — a good moment to explicitly state that "roughly sortable, fully uncoordinated" is the right trade-off for virtually all real use cases (pagination, default sort) that don't need cryptographically/strictly perfect cross-worker ordering.

---

38. Design a Proximity Service (Yelp/Nearby)

Problem

Design a service that, given a user's location and a search radius/category, returns nearby points of interest (restaurants, shops, etc.) — a read-heavy, geospatially-indexed search system.

Requirements

Functional: (1) Given (lat, lng, radius, category), return nearby matching businesses, ranked by distance (and optionally rating/relevance). (2) Businesses can be added/updated (new listings, changed hours, closures) relatively infrequently compared to search volume. **Non-Functional:** Very read-heavy (searches ≫ business

updates, unlike the ride-sharing case study §30 where location *writes* dominated — a useful contrast to explicitly draw in an interview). Low search latency. Business location data changes rarely (businesses don't move), which is the key structural difference from §30's constantly-moving driver locations — this changes which data-structure trade-offs make sense.

Estimation

Assume 100M searches/day, 50M business listings (mostly static):

```
Search QPS avg ≈ 100,000,000 / 86,400 ≈ 1,157/sec -> peak (x3) ≈ 3,470/sec
Business update QPS: comparatively tiny (listings change rarely -- hours, closures,
new
  openings), nowhere near the ~1.25M/sec driver-location write rate from §30
```

Conclusion: because business data is relatively static (unlike constantly-moving drivers), the geospatial index here can be **read-optimized and rebuilt/updated periodically** rather than needing to support extremely high-frequency in-place updates — a meaningfully different, simpler engineering problem than §30 despite superficially similar "find nearby things" framing, and a great point to explicitly contrast if asked "how is this different from the Uber driver-matching problem?"

API Design

```
GET /api/v1/search?lat=..&lng=..&radius_km=5&category=restaurant&cursor=...
    -> { "results": [{ "business_id", "name", "distance_km", "rating" }, ...] }
```

High-Level Design

```
Client -> LB -> Search Service -> Geospatial Index (Quadtree/Geohash/S2, §30's deep
dive
                                      applies directly here too)
                      |
              Business Metadata Store (name, category, hours, rating --
relational/document
                                          DB, updated on the relatively rare add/edit
events)
                      |
              Cache layer (popular/dense areas' search results cached, since
```

```
business data
                              changes slowly, cache TTLs can be much longer than,
say, a
                              live driver-location cache would tolerate)
```

Because business locations change rarely, this system can afford a periodically-**rebuilt** static geospatial index (e.g. rebuild a Quadtree/geohash-grid structure on a schedule — every few minutes to hours — from the current business metadata) rather than needing the continuously-mutating, extremely-high-write-throughput in-memory index that the ride-sharing driver-location problem (§30) required. This is worth calling out explicitly as the single biggest architectural simplification versus §30.

Deep Dive: Geospatial Index Choice & Search Algorithm

Reuse the geohashing/Quadtree/S2 concepts from §30's deep dive, but optimized for a mostly-static dataset:

- Build a **Quadtree** (or geohash-based grid) once from the current set of business locations, subdividing more finely in dense areas (e.g. a city center) and coarsely in sparse ones (rural areas) — this non-uniform subdivision is exactly why a Quadtree (or similar adaptive structure) usually beats a fixed-size grid for this kind of naturally clustered, non-uniform point data.
- A search query descends the tree from the root, pruning branches whose bounding region doesn't intersect the search radius, collecting candidate businesses in intersecting leaf nodes, then computing exact distance for final ranking/filtering among those candidates (avoiding an expensive exact-distance computation against every single business in the dataset).
- Since the index is read-heavy and rebuilt periodically rather than continuously mutated, it can be **fully replicated (not sharded) across many search-serving nodes** if it fits comfortably in memory per node (50M businesses' geospatial index is a relatively modest memory footprint) — trading some replication storage overhead for simpler, coordination- free horizontal read scaling (every node has a complete, independent copy — no cross-node query fan-out needed at all).

Deep Dive: Handling Category Filtering & Ranking Efficiently

A pure "nearest N points" geospatial query isn't quite enough — users filter by category ("restaurants") and want ranking beyond pure distance (rating, popularity, promoted listings). Two reasonable approaches: (a) maintain a **separate geospatial index per category** (more indexes to maintain/rebuild, but each query only searches the relevant one — simpler per-query logic), or (b) maintain one combined geospatial index and apply category/other filters as a post-filter on the candidate set returned from the tree traversal (simpler index maintenance, but the candidate set from a tree traversal needs to be large enough to survive filtering without under-returning results — usually fine given this system's scale). Ranking (distance blended with rating/relevance/promoted status) is typically a lightweight scoring pass over the already-small candidate set (tens-hundreds of results, not the whole 50M dataset), so it doesn't need special infrastructure beyond the geospatial candidate retrieval already described.

Bottlenecks & Scaling

- **Popular/dense search areas** (e.g. searches centered on a major city downtown) can be effectively cached (given the slow-changing nature of business data) — a search result cache keyed by a coarsened/bucketed location (e.g. round lat/lng to the nearest ~500m grid cell) + category, with a TTL of minutes-to-hours, absorbs a large fraction of repeat/similar queries without hitting the geospatial index at all.
- **Index rebuild cost** grows with total business count — mitigate with **incremental index updates** for the common case (a single business's hours changing doesn't need a full tree rebuild — just update that leaf's metadata) reserving full rebuilds for larger-scale changes (bulk imports, periodic full refresh for structural rebalancing).

Trade-offs

- Fully replicating the geospatial index per search node (rather than sharding it) simplifies queries enormously (no cross-node fan-out) at the cost of redundant memory usage across nodes — a good trade-off specifically because the index is comparatively small and slow-changing; this would **not** be the right trade-off for §30's constantly-mutating, much-larger-scale driver location index.

Follow-up Interview Questions

- *"How would you handle a business temporarily marked 'closed for the day' without a full reindex?"* — Store frequently-changing attributes (open/closed status, current wait time) separately from the geospatial index itself (e.g. a fast key-value lookup keyed by business ID, checked/joined in after retrieving geospatial candidates) — so volatile attributes don't require touching the comparatively expensive-to-rebuild spatial structure at all.
- *"How is this fundamentally different from the Uber driver-location problem (§30), given both are 'find nearby things'?"* — Write frequency: businesses are near-static (index can be periodically rebuilt/replicated), drivers move continuously (index must be continuously, cheaply mutable, favoring an in-memory sharded structure over a replicated static one) — a great question to proactively invite/address, since it demonstrates you understand that superficially similar problems can have very different right answers based on their actual read/write characteristics.

---

39. Design Instagram/Photo Sharing

Problem

Design a photo/video sharing platform: users upload media, follow other users, and view a feed of content from people they follow (with likes/comments).

Requirements

Functional: (1) Upload photos/videos with captions. (2) Follow/unfollow users. (3) View a feed of posts from followed accounts. (4) Like and comment on posts. **Non-Functional:** Extremely read-heavy (views ≫ uploads). Media (photos/videos) dominate storage and bandwidth. Feed generation faces the same celebrity/fan-out skew problem as Twitter (§27) — this case study intentionally overlaps significantly with §27 and §29; treat it as **combining the news feed fan-out problem (§27) with the media storage/delivery problem (§29)** rather than re-deriving everything from scratch, which is itself a strong, efficient way to answer this in an interview ("this is essentially §27's feed architecture plus §29's media pipeline — let me focus my time on what's genuinely different: [X]").

Estimation

Assume 500M DAU, 100M photos/videos uploaded/day, avg photo 3MB / avg video clip 20MB:

```
Upload storage/day ≈ mix of photo+video, roughly 100M × ~5MB blended avg = 500
TB/day --
  clearly needs object storage (§18) + CDN (§9.7), same conclusion as §29
Feed read QPS: same order of magnitude and same skew characteristics as §27's
estimation
```

Conclusion: reuse §27's fan-out-on-write (with celebrity fan-out-on-read exception) for the social feed graph problem, and reuse §29's upload-processing-pipeline pattern (async transcoding/resizing, object storage, CDN delivery) for the media problem — the genuinely novel part of this specific case study is combining them and handling **image-specific** processing (thumbnailing/multiple resolutions for responsive display) rather than video's adaptive-bitrate-streaming-specific concerns.

API Design

```
POST /api/v1/posts     (multipart upload: media + caption) -> 201 { "post_id":
"..." }
GET  /api/v1/feed?cursor=...   -> same shape/pattern as §27
POST /api/v1/posts/{id}/like
POST /api/v1/posts/{id}/comments   { "text": "..." }
```

High-Level Design

```
Upload path: Client -> Upload Service -> Object Storage (original) -> async Image
Processing
             Pipeline (generate multiple resolutions: thumbnail/medium/full,
similar in
             spirit to §29's multi-bitrate transcoding but for images) -> processed
images
             -> Object Storage -> CDN

Feed path: reuses §27's architecture wholesale (precomputed feed lists w/ celebrity
           fan-out-on-read exception) -- feed entries reference post_id + a CDN URL
for
           the appropriate image resolution for the requesting device/viewport
```

Deep Dive: Image Processing Pipeline (The Genuinely New Piece vs §27/§29)

Unlike video's adaptive-bitrate-streaming needs, images need: (1) **multiple fixed resolutions generated at upload time** (thumbnail for feed grid view, medium for feed detail view, full for zoomed/download) — this is simpler than video transcoding (no segmenting, no bitrate ladders, just a handful of resize+compress operations per image), so it can often complete fast enough to feel closer to "synchronous" from the user's perspective (though still done asynchronously off the critical upload-response path for resilience/consistency with the rest of this guide's patterns), and (2) **client-adaptive image serving** — the client requests the resolution appropriate for its viewport/screen density (e.g. via an image CDN that supports on-the-fly resizing parameters in the URL, or pre-generated fixed breakpoints) rather than always downloading the full-resolution original, saving significant bandwidth for the far-more-common "scrolling through a feed of thumbnails" case versus "viewing one photo full-screen."

Deep Dive: Likes/Comments at Scale (Hot Post Problem)

A viral post can receive an extremely high rate of concurrent likes — naively incrementing a `like_count` column with `UPDATE posts SET like_count = like_count + 1 WHERE post_id = ?` for every single like creates a hot-row contention problem at that scale (many concurrent transactions all trying to update the exact same row). Mitigations, in order of increasing sophistication: (a) **batch/debounce** — buffer like events for a short window (e.g. in Redis, `INCR` an in-memory counter) and periodically flush the aggregated delta to the durable `like_count` column, rather than a DB write per individual like (same "batched counter" pattern already introduced for video view counts, §29); (b) store individual `likes` as rows in their own table (`post_id, user_id, created_at`, useful for "did I like this" checks and "who liked this" lists) **separately** from the aggregate count, so the high-contention aggregate-count updates and the simple per-user like-record inserts don't compete for the same row/lock.

Bottlenecks & Scaling

- Same fan-out/celebrity considerations as §27 apply directly to the feed.
- Same CDN-first, minimize-origin-traffic principle as §29 applies directly to media delivery — the overwhelming majority of image views should be served from CDN edge caches, never hitting origin object storage or app servers.

Trade-offs

- Generating multiple image resolutions upfront (at upload time) costs storage (multiple copies per image) and upload-time processing, in exchange for much better feed-scrolling bandwidth/performance — directly analogous to §29's multi-bitrate video trade-off, just cheaper/simpler for still images.

Follow-up Interview Questions

- *"How would you implement Instagram Stories (24h ephemeral content)?"* — Same answer as §27's follow-up on Stories — reuse the feed architecture with a TTL on entries and underlying storage.
- *"This looks a lot like combining two systems you already covered — was that intentional?"* — Yes, and saying so explicitly in an interview (rather than pretending to design everything from first principles again) is a *strength*, not a shortcut — it demonstrates you recognize when a new prompt is a recombination of already-solved sub-problems, which is exactly how real engineers approach new systems in practice.

40. Design a Distributed Task Scheduler/Cron

Problem

Design a system that executes scheduled/recurring jobs (like cron) reliably across a distributed fleet of workers, ensuring each job runs exactly once (not zero times, not multiple times) even as workers/schedulers themselves fail.

Requirements

Functional: (1) Schedule a job to run once at a specific future time, or on a recurring basis (e.g. cron expression `0 * * * *`). (2) Execute jobs reliably even if the worker handling it crashes mid-execution. (3) Support job priorities and dependencies (optional, often out of scope for a 45-min version). (4) Provide job execution history/status. **Non-Functional:** Jobs must run **exactly once** (or at minimum, effectively-once via retries + idempotent job logic — echoing §10.4's honest framing) even with worker/scheduler failures. Scalable to millions of scheduled jobs. Low scheduling overhead/latency (a job scheduled for 3:00:00 PM shouldn't actually start at 3:00:05 PM due to system overhead).

Estimation

Assume 10M scheduled jobs total (mix of one-off and recurring), with a peak of 50,000 jobs due to fire within the same minute (e.g. many jobs scheduled at common times like the top of the hour) — the scheduler must efficiently find "which jobs are due right now" among potentially millions of stored jobs, repeatedly, at a fine time granularity (e.g. every few seconds) without scanning the entire job table each time.

API Design

```
POST /api/v1/jobs
{ "job_type": "send_report", "payload": {...}, "schedule": "cron:0 9 * * *" |
"run_at": "2026-09-05T15:00:00Z" }
-> 201 { "job_id": "..." }
DELETE /api/v1/jobs/{id}     (cancel)
GET    /api/v1/jobs/{id}/history
```

High-Level Design

```
Job Registration API -> Job Store (DB: job definition, schedule, next_run_time,
status)
                              |
                    Scheduler/Poller (periodically queries: "which jobs have
                                      next_run_time <= now AND status =
'PENDING'?")
                              |
                    (claims due jobs atomically, pushes to) -> Execution Queue
(Kafka/SQS)
                              |
                    Worker Pool (consumes queue, executes job logic, reports
```

```
success/failure)
                                  |
                      (on success: for recurring jobs, compute + write next
next_run_time;
                       on failure: retry with backoff or mark failed after max
attempts)
```

Deep Dive: Efficiently Finding Due Jobs at Scale

Scanning a table of millions of jobs every few seconds to find "which are due now" doesn't scale with a naive full-table scan. Solution: an **index on** `next_run_time`, so the poller's query (`WHERE next_run_time <= now() AND status = 'PENDING' LIMIT N`) is an efficient index range scan, not a full scan — combined with a **time-bucketed/sharded polling approach** at very large scale (e.g. partition jobs across multiple poller instances by job ID hash, each responsible for scanning and dispatching only its own partition, so no single poller instance needs to handle the full 10M-job dataset alone).

Deep Dive: Exactly-Once Execution & Preventing Duplicate Runs

Two failure scenarios must both be handled: (1) **two poller instances both claim the same due job** (if polling is naively "read due jobs, then separately mark them claimed" — a classic read-then-write race condition, same category of bug as seat-double-booking §31 and inventory overselling §32) — solved with the same **atomic conditional update** pattern used throughout this guide: `UPDATE jobs SET status='CLAIMED', claimed_by=?, claimed_at=now() WHERE job_id=? AND status='PENDING'`, checking `rows_affected==1`, ensuring only one poller instance successfully claims any given due job. (2) **a worker crashes mid-execution**, leaving a job stuck in `CLAIMED`/`RUNNING` forever — solved with a **claim lease/timeout** (analogous to §31's seat-hold expiry and §15.9's distributed-lock lease pattern): if a job remains `RUNNING` past an expected max-duration threshold without a completion signal, a watchdog process resets it back to `PENDING` so it can be re-claimed and retried — combined with **idempotent job logic** (the actual job handler code should tolerate being run twice for the same logical invocation, e.g. via the same idempotency-key discipline used throughout this guide) as the final safety net, since guaranteeing literal, perfect exactly-once execution across arbitrary failure modes is fundamentally

as hard as the distributed consensus problems discussed in §15 — "effectively-once via at-least-once + idempotency" is the same honest, correct framing used for message delivery in §10.4.

Deep Dive: Handling Recurring Jobs (Cron Expressions)

For a recurring job, after successful execution, compute the **next occurrence** from the cron expression (a well-understood, standard parsing/calculation problem — most languages have mature libraries for this) and write it as the new `next_run_time`, keeping the same job record rather than creating a new one each time — this keeps the jobs table size bounded by the number of *distinct scheduled jobs*, not by the number of times they've ever executed (execution history is tracked separately, in an append-only log/table, which *can* grow large and should be partitioned/archived independently).

Bottlenecks & Scaling

- **Thundering herd at common schedule times** (many jobs scheduled for "top of the hour") — the execution queue (Kafka/SQS) naturally absorbs this burst (§10.1's core value proposition), decoupling the burst of "due right now" jobs from the steady-state throughput of the worker pool actually executing them.
- **Poller/scheduler itself** must be horizontally scaled and partitioned (as discussed above) to avoid becoming a single bottleneck/SPOF as total job count grows.

Trade-offs

- Polling-based due-job detection (check every few seconds) has some inherent scheduling latency/jitter (a job may run up to one polling interval late) — acceptable for the vast majority of scheduled-job use cases (reports, cleanup jobs, etc.); genuinely latency-critical "fire at exactly this millisecond" needs would require a different approach (e.g. a min-heap-based in-memory timer service) rarely justified at typical cron job precision requirements.

Follow-up Interview Questions

- *"How would you support job dependencies (job B only runs after job A succeeds)?"* — Model as a DAG (directed acyclic graph) of job dependencies; a job only becomes eligible ("PENDING," ready to be claimed) once all its dependency jobs report success — requires the scheduler to track dependency-completion state, not just time-based readiness.
- *"How do you prevent a single misbehaving job from blocking the whole worker pool (e.g. an infinite loop)?"* — Enforce a hard execution timeout per job (kill/interrupt the worker process handling it after a configured max duration) and isolate job execution (e.g. separate process/container per job, or at minimum a strict resource/time limit) so one bad job can't starve the pool for everything else.

---

41. Design Ad Click Aggregation/Analytics

Problem

Design a system that ingests a massive stream of ad click/impression events and produces near-real-time and historical aggregated analytics (e.g. "clicks per ad campaign per hour") for advertisers and internal reporting.

Requirements

Functional: (1) Ingest click/impression events at very high volume. (2) Aggregate counts by multiple dimensions (ad, campaign, advertiser, time window, geography). (3) Support both near-real-time dashboards (last few minutes) and historical queries (arbitrary date ranges). **Non-Functional:** Extremely high write/ingestion throughput (this is fundamentally a **write-heavy, analytics-oriented** system — a useful contrast to most of this guide's read-heavy case studies). Must handle **duplicate/out-of-order events** gracefully (network retries, mobile clients sending delayed events). Aggregation results can tolerate a small amount of latency (seconds-to-minutes) — this is not a strict-real-time system, easing the consistency requirements considerably compared to, say, the payment system (§36).

Estimation

Assume 1M ad impressions/sec at peak across a large ad network (billions of impressions/day industry-wide is a realistic order of magnitude for a major ad platform):

```
Ingest QPS ≈ 1,000,000/sec at peak -- this single number alone rules out any design
  involving a synchronous relational-DB write per event; must be a streaming/batch
  aggregation pipeline
```

Conclusion: the ingestion volume alone dictates a **streaming aggregation architecture** (Kafka + a stream processor) rather than any design that tries to write/query raw individual events synchronously per request — this is a fundamentally different architectural family from almost every other case study in this guide (which centered on request/response latency for a single user's action).

API Design

Ingestion is typically a lightweight, fire-and-forget event API (often just a tracking pixel/beacon call from an ad-serving client, not a rich REST contract):

```
POST /track/click   { "ad_id": "...", "campaign_id": "...", "timestamp": "...",
"user_region": "..." }
-> 204 No Content (fire-and-forget, client doesn't wait for/process a meaningful
response)
```

Analytics **querying** is a separate, much lower-volume API:

```
GET /api/v1/analytics/campaigns/{id}?granularity=hour&from=...&to=...
    -> [{ "hour": "2026-09-01T14:00:00Z", "clicks": 45231, "impressions": 892103 },
...]
```

High-Level Design

```
Ad Clients -> Ingestion Endpoint (lightweight, just validates + writes to Kafka,
minimal
              processing -- this must be extremely fast/cheap given the 1M/sec
volume)
                    |
              Kafka (raw event stream, partitioned e.g. by ad_id or campaign_id)
                    |
              Stream Processor (Flink/Spark Streaming/Kafka Streams) --
```

```
                windowed aggregation (e.g. tumbling 1-minute windows, grouped by
                ad_id/campaign_id/region)
                    |
              Aggregated results written to:
                - a fast-serving OLAP store (e.g. Druid, ClickHouse, or a pre-
aggregated
                  time-series table) for near-real-time dashboard queries
                - a data warehouse (e.g. long-term columnar storage) for
historical/ad-hoc
                  analytical queries
```

Deep Dive: Stream Processing & Windowed Aggregation

Rather than aggregating each event against a live counter synchronously (which wouldn't scale at this ingestion rate and wouldn't handle late/out-of-order events correctly), a **stream processing framework** (Flink, Spark Streaming, or Kafka Streams) consumes the raw Kafka event stream and performs **windowed aggregation**: grouping events into time windows (e.g. tumbling 1-minute windows — non-overlapping, fixed-size) by the relevant dimensions (ad_id, campaign_id, region), computing counts/sums per window, and emitting the aggregated result once the window closes. This is dramatically more efficient than a per-event synchronous database write/increment at this volume, and naturally produces the "clicks per hour/minute" style aggregates the analytics API needs to serve.

Deep Dive: Handling Late-Arriving and Duplicate Events

Real-world event streams have clock skew, network delay, and client retries — an event "belonging" to the 2:00-2:01pm window might arrive at 2:03pm (late), and the same click might be sent twice (duplicate, e.g. a mobile client retry after a flaky network response).

- **Late events:** stream processors support a **watermark** concept — a heuristic estimate of "how late can an event be and still be included in its correct window" (e.g. "allow up to 5 minutes of lateness"). Events arriving within the watermark are still correctly aggregated into their proper window (even though the window's *output* is delayed by the watermark duration); events arriving *after* the watermark has passed are either dropped or routed to a separate "late data" correction

pipeline (updating an already-emitted aggregate) — a genuine, honest trade-off between result latency and result completeness that's worth explicitly naming.
- **Duplicate events:** deduplicate using a unique event ID (assigned by the ad-serving client at click time) checked against a **short-lived deduplication window** (e.g. a Bloom filter or a TTL'd set of recently-seen event IDs, sized to the maximum expected duplicate-arrival delay) — the same idempotency-key philosophy applied throughout this guide (§10.4, §12.5), applied here at streaming-ingestion scale rather than per-request API scale.

Deep Dive: Serving Both Real-Time and Historical Queries

A single storage system rarely serves both needs well: near-real-time dashboards need **fast, recent, incrementally-updated aggregates** (favoring an OLAP-style store like Druid/ClickHouse optimized for exactly this access pattern), while historical/ad-hoc analysis needs to **scan large date ranges flexibly** (favoring a columnar data warehouse/data lake, e.g. built on object storage with a query engine on top). The standard architecture (sometimes called the **Lambda** or **Kappa** architecture) writes the stream-processed aggregates to both destinations — a fast-serving layer for the "last few hours/days, drill-down dashboard" use case, and a durable, complete historical store for everything else — rather than forcing one storage technology to do both jobs adequately.

Bottlenecks & Scaling

- **Ingestion endpoint** must be extremely lightweight (essentially just "validate + write to Kafka") — any heavier synchronous processing at this stage (e.g. a DB lookup or complex validation) would become the bottleneck given the 1M/sec+ volume; push all meaningful processing downstream into the stream processing layer.
- **Kafka partition count and stream-processor parallelism** must scale together with ingestion volume — partitioning by `ad_id`/`campaign_id` keeps all events for a given entity in order on the same partition (needed for correct windowed aggregation per entity) while still spreading load across many partitions/processing tasks overall.

Trade-offs

- Watermark duration (how long to wait for late events before finalizing a window's output) is a direct trade-off between dashboard freshness (shorter watermark = faster results) and result completeness/accuracy (longer watermark = fewer missed late events) — the right value depends on the specific product's tolerance for either "slightly delayed" or "slightly incomplete-then-corrected" numbers.

Follow-up Interview Questions

- *"How would you detect and filter out fraudulent clicks (click farms/bots) before they pollute the aggregates and bill advertisers incorrectly?"* — A separate, asynchronous fraud-scoring pipeline (similar in spirit to the payment system's fraud-detection follow-up, §36) consuming the same raw event stream, flagging suspicious events (e.g. by IP reputation, click velocity, device fingerprinting) — ideally filtering fraudulent events out **before** they're counted in advertiser-facing/billing aggregates, which argues for running fraud detection as an early stage in the pipeline rather than a downstream correction.
- *"How would this differ if you needed strictly exact, immediately-consistent counts (e.g. for real-time bidding decisions) rather than analytics dashboards?"* — That's a fundamentally different problem (favoring a low-latency in-memory counter store like Redis for the specific narrow counters needed at decision time) layered *alongside* this broader analytics pipeline, not a replacement for it — a good moment to distinguish "operational, decision-time counters" from "analytical, reporting-time aggregates" as two legitimately different systems with different requirements, even though both start from similar raw events.

---

42. Design a Leaderboard System

Problem

Design a system that maintains and serves real-time rankings (e.g. for a game, showing top players by score, and a given player's own rank) at very high update frequency.

Requirements

Functional: (1) Update a player's score (frequently, e.g. after every game/match). (2) Retrieve the top-N leaderboard. (3) Retrieve a specific player's current rank (even if they're not in the visible top-N — "show me my rank, #45,213"). (4) Support multiple leaderboards (global, regional, friends-only, per-game-mode, time- windowed like "this week's leaderboard"). **Non-Functional:** Very high write throughput (score updates happen constantly across millions of concurrent players). Low-latency reads for both "top N" and "my specific rank" queries. Rankings should reflect recent updates quickly (near-real-time, though perfect real-time strictness usually isn't required).

Estimation

Assume 50M daily active players, each completing ~10 scored matches/day:

```
Score update QPS avg ≈ 50M × 10 / 86,400 ≈ 5,787/sec -> peak (x3) ≈ 17,360/sec
Leaderboard read QPS: typically much higher than updates (players check rank far
more
  often than they finish a match) -- read-heavy, similar shape to many earlier case
studies
```

Conclusion: this needs a data structure that supports **both** extremely fast "get top-N" **and** fast "get this specific member's rank" queries, updated at high frequency — a plain relational `ORDER BY score DESC LIMIT N` query re-run on every read (naive approach) becomes prohibitively expensive at this update/read frequency, especially for the "what's my rank" query (which would need counting how many players rank above a specific score, expensive without the right data structure).

API Design

```
POST /api/v1/leaderboards/{id}/scores    { "player_id": "...", "score_delta": 150 }
GET  /api/v1/leaderboards/{id}/top?limit=100
GET  /api/v1/leaderboards/{id}/players/{player_id}/rank   -> { "rank": 45213,
"score": 8420 }
```

High-Level Design — Sorted Set as the Core Data Structure

```
Client -> API -> Leaderboard Service -> Redis Sorted Set (ZSET), one per
leaderboard
                                        (member = player_id, score = current
score)
```

Redis's `ZSET` (an in-memory data structure combining a hash map and a skip list) is close to a **purpose-built solution** for this exact problem: it natively supports all the operations this system needs in **O(log N)** time:

- `ZINCRBY leaderboard:global 150 player123` — atomically update a player's score.
- `ZREVRANGE leaderboard:global 0 99 WITHSCORES` — get the top 100 players and their scores.
- `ZREVRANK leaderboard:global player123` — get a specific player's current rank (0-indexed position among all members, descending by score) directly, without scanning/counting. This single data structure choice elegantly solves the "both top-N and individual-rank queries must be fast" requirement that made a naive relational approach awkward — a strong, concise deep-dive point precisely because it shows recognizing when an off-the-shelf data structure (already covered in this guide's caching toolbox, §9/§34) is a much better fit than building something bespoke.

Deep Dive: Scaling Beyond a Single Redis Instance

A single Redis sorted set for, say, 50M players is likely still fine memory-wise (each member is a small fixed-size entry), but at even larger scale, or for very high write throughput exceeding a single Redis instance's capacity, **partition the leaderboard**: one common approach is **sharding by score range or by a hash of player ID into multiple sub-leaderboards**, each an independent sorted set, with a periodic (or on-demand) **merge step** to compute the true global top-N by combining the top candidates from each shard (since the true global top-100 must be among the union of each shard's own top-100 — you never need more than the top-K from each of the M shards to correctly compute the global top-K, a nice property that bounds the merge step's cost regardless of total player count).

Deep Dive: Time-Windowed Leaderboards ("This Week's Leaderboard")

For a weekly-resetting leaderboard, rather than trying to "subtract" old scores from a single persistent sorted set (fragile, error-prone), use a **fresh sorted set per time window** (e.g. `leaderboard:weekly:2026-W35`), simply creating a new, empty ZSET at the start of each week and updating it alongside the persistent all-time leaderboard on every score event (a single score update fans out to write to however many active leaderboards — global, weekly, monthly, friends — a player's update is relevant to, which is a small, bounded, known set per event, not an expensive fan-out problem like the social feed's follower fan-out §27). Old weekly leaderboards are simply archived/expired (TTL) once no longer needed, avoiding any need for complex "undo" logic.

Bottlenecks & Scaling

- **Extremely popular/global leaderboards** (a single global leaderboard for a massive-scale game) concentrate all writes onto one sorted set — the sharding approach above directly addresses this if a single Redis instance's throughput becomes limiting.
- **Friends-only leaderboards** (potentially one per player, e.g. "my 50 friends' scores") scale differently — rather than a separate ZSET per player (wasteful, most friend groups are small), often computed **on read** by fetching each friend's score individually (or via a small batch `ZMSCORE`/`ZSCORE` set of lookups against the global leaderboard) and sorting client-side/service-side — a good example of choosing fan-out-on-read (§27's terminology) for a case where the "fan-out" set (friends) is small and personal to each reader, rather than trying to precompute/maintain a dedicated structure per player.

Trade-offs

- Redis ZSET is in-memory — extremely fast, but the whole leaderboard's data must fit in memory (usually not a real constraint even for hundreds of millions of players, given how compact each entry is) and standard Redis persistence/replication trade-offs (§34) apply for durability if the leaderboard data itself is considered important enough not to regenerate from a source-of-truth score-history log if lost.

Follow-up Interview Questions

- *"How would you handle tie-breaking (two players with the exact same score) fairly and deterministically?"* — Use a composite sort key (e.g. combine score with a secondary tie- breaker like "earliest time reaching this score," encoded into the ZSET's score value itself via a small fractional/composite encoding, or a secondary sort applied at read time) — must be decided and applied consistently, since Redis ZSET ties otherwise break by lexicographic member-name order, which is rarely the desired product behavior.
- *"How do you prevent score manipulation/cheating from corrupting the leaderboard?"* — Server-side score validation/anti-cheat logic must live in the **game/match service** producing score-update events, not in the leaderboard system itself, which should treat incoming score deltas as already-trusted — worth explicitly scoping this out as "the leaderboard system's job is to rank scores correctly and fast; validating that scores are legitimate is a different service's responsibility" to avoid over-scoping the answer.

43. Design a Log Aggregation System (Splunk-like)

Problem

Design a system that collects logs from thousands of distributed servers/services, centralizes them, and makes them searchable/queryable for debugging and monitoring — directly enabling the "Logging" pillar of observability discussed in §17.1.

Requirements

Functional: (1) Collect log lines from many hosts/services in near-real-time. (2) Support full-text search and structured filtering (by service, time range, log level, specific fields). (3) Support dashboards/alerts built on log-derived metrics. **Non-Functional:** Very high ingestion throughput (every service in a large infrastructure emits logs constantly). Search queries should return results quickly even across a huge historical volume. Must not lose logs during ingestion spikes or transient failures

(though some systems accept a small, bounded loss rate under extreme duress as an explicit, documented trade-off — worth stating as a scoping decision, not an oversight).

Estimation

Assume 10,000 hosts each producing 10MB of logs/day:

```
Total ingestion ≈ 10,000 × 10MB = 100 GB/day -- modest by "big data" standards but
  represents a continuous, always-on write stream from thousands of independent
sources
Retention: if searchable history needs 30 days -> 3 TB "hot" (frequently queried)
plus
  potentially much more in cheaper "cold" archival storage for compliance/rare
lookups
```

Conclusion: similar shape to the ad-click analytics problem (§41) — a write-heavy ingestion pipeline feeding into a store optimized for the actual read pattern (here: full-text + field-filtered search, rather than time-windowed numeric aggregation) — reuse much of §41's architectural thinking (streaming ingestion buffer, tiered hot/cold storage) while swapping the serving layer for a search-optimized store instead of an OLAP store.

API Design

Ingestion is typically via a lightweight **agent** running on each host (not a per-log-line REST call from application code):

```
Log Agent (e.g. Filebeat/Fluentd) tails local log files -> ships batched log lines
to a
  central Ingestion API/endpoint
```

Search is the user-facing API/UI:

```
GET /api/v1/logs/search?query=error+AND+service:payment&from=...&to=...&limit=100
    -> matching log entries, most recent first
```

High-Level Design

```
Hosts (each running a lightweight Log Agent) -> Ingestion Layer (Kafka --
buffers/absorbs
                                                  bursty log volume from thousands
of
                                                  independent sources, same "queue
absorbs
                                                  burst" principle as §10.1/§26/
§41)
                                                        |
                                                Log Processor (parses/structures
raw log
                                                  lines -- extracts fields like
timestamp,
                                                  level, service, message; enriches
with
                                                  metadata like host/region)
                                                        |
                                          Search/Indexing Store (Elasticsearch --
inverted
                                            index, §8.2, purpose-built for full-text
search)
                                                        |
                                          (older data tiered out to cheaper Object
Storage
                                           for long-term/compliance retention, no
longer
                                           actively indexed for fast search)
```

Deep Dive: Log Shipping & Buffering at the Edge

Each host runs a lightweight agent (Filebeat, Fluentd, or similar) that tails local log files, batches lines (rather than sending one network call per log line — far too much overhead at this volume), and ships batches to the central ingestion endpoint. The agent must handle **local buffering** (e.g. to local disk) if the central ingestion pipeline is temporarily slow/unavailable — so a brief central-side hiccup doesn't cause the *host* to drop logs; the agent retries shipping once the pipeline recovers. This distributes some of the reliability burden to the edge (thousands of small, resilient buffers) rather than requiring the central system to be perfectly available at all times for zero log loss.

Deep Dive: Parsing & Structuring Raw Logs

Raw log lines are often unstructured/semi-structured text (varying formats across different services/languages) — the Log Processor stage parses these into structured

fields (timestamp, log level, service name, request ID, message body, and any custom key-value fields) using per-log-format parsers/grok patterns, so the search store can support **structured filtering** (e.g. "service:payment AND level:ERROR") in addition to free-text search over the message body. Standardizing on **structured logging at the source** (services emit JSON logs with consistent field names in the first place, rather than free-form text requiring fragile regex parsing downstream) is a strongly recommended practice worth mentioning — it shifts parsing complexity/fragility out of the central pipeline and into a one-time logging-format convention each service adopts.

Deep Dive: Making Full-Text + Structured Search Fast at Scale

This is precisely what **Elasticsearch** (or similar, e.g. OpenSearch) is purpose-built for: it maintains an **inverted index** (§8.2) mapping each distinct term/token to the list of documents (log entries) containing it, enabling fast full-text search, combined with efficient structured field filtering (via dedicated per-field indexes) and it's natively **horizontally shardable** (an index is split into multiple shards distributed across nodes, and queries fan out to relevant shards in parallel and merge results) — directly matching the scale requirements here. Time-based **index rotation** (e.g. one Elasticsearch index per day, `logs-2026-09-01`, `logs-2026-09-02`, ...) is the standard practice: it makes both retention management (simply delete/archive entire old daily indices rather than deleting individual documents) and time-range-scoped queries (only search indices overlapping the requested time range, skipping irrelevant ones entirely) efficient.

Bottlenecks & Scaling

- **Search index write throughput** at 100GB+/day sustained ingestion requires adequate Elasticsearch cluster sizing (shard count, node count) tuned to the actual ingestion rate — a classic operational concern for any Elasticsearch-based system at scale.
- **Hot/cold data tiering**: keep only recent, actively-queried data (e.g. last 7-30 days) in the expensive, fully-indexed "hot" search tier; older data moves to cheaper object storage (compressed, minimally indexed or not indexed at all) — queries against very old data are rarer and can tolerate being slower/requiring an explicit "search cold storage" action, a reasonable and common trade-off.

Trade-offs

- Structured logging at the source (recommended) requires each service/team to adopt a consistent logging convention (an organizational, not just technical, effort) versus accepting messier free-text parsing centrally (less upfront effort, more fragile/lossy downstream) — worth naming as a real organizational trade-off, not just a technical one.
- Aggressive hot/cold tiering reduces cost significantly but adds latency/friction for the (hopefully rare) need to search old data — an explicit, product-level retention policy decision, not a purely technical one.

Follow-up Interview Questions

- *"How would you build alerting on top of this (e.g. 'alert if error rate exceeds X in 5 minutes')?"* — A separate, lightweight streaming aggregation layer (conceptually similar to §41's windowed aggregation) consuming the same ingestion stream in parallel with the search-indexing path, computing rolling error-rate metrics and triggering alerts — the alerting path doesn't need to query the (potentially slower, full-text) search index at all for this, since it only needs simple counts, not searchable content.
- *"How do you handle extremely high-cardinality fields (e.g. a unique request ID on every log line) without blowing up index size/cost?"* — High-cardinality fields are still useful for exact-match lookups (searching "show me every log line for this specific request ID" is a common, valuable debugging query) but shouldn't be used for aggregations/faceting (e.g. "count by request ID" is meaningless) — a reasonable answer distinguishes which fields are indexed for search-and-filter versus which might be excluded from more expensive aggregation-oriented indexing structures.

Part 6 — LLD Case Studies (Full Java Code)

44. LLD: Parking Lot

Problem

Design the class model for a multi-level parking lot that supports multiple vehicle types (motorcycle, car, bus), multiple spot sizes, and charges based on parking duration.

Requirements

- Support multiple vehicle types, each fitting only certain spot sizes (motorcycle fits any spot; car fits compact/large; bus needs a large spot, or several contiguous large spots).
- Track available/occupied spots per level, and across the whole lot.
- Issue a ticket on entry, calculate a fee on exit based on duration and vehicle type.
- Extensible: adding a new vehicle type or a new pricing scheme shouldn't require rewriting existing classes (Open/Closed Principle, §20.2).

Identify Entities

`Vehicle` (+ subtypes), `ParkingSpot` (+ size), `ParkingLevel`, `ParkingLot`, `Ticket`, `PricingStrategy` (Strategy pattern, §21.5).

Design Patterns Used

- **Strategy** for pricing (swap pricing schemes without touching `Ticket`/`ParkingLot`).
- **Factory Method** could create the right `Vehicle` subtype from user input (omitted here for brevity — see §21.2 for the general pattern).
- **Singleton** could apply to `ParkingLot` (usually exactly one instance per physical facility) — shown below.

Full Java Code

```
import java.time.LocalDateTime;
import java.time.Duration;
import java.util.*;

enum VehicleType { MOTORCYCLE, CAR, BUS }
enum SpotSize { SMALL, MEDIUM, LARGE }

abstract class Vehicle {
    private final String licensePlate;
    private final VehicleType type;
```

```
    Vehicle(String licensePlate, VehicleType type) {
        this.licensePlate = licensePlate;
        this.type = type;
    }
    public VehicleType getType() { return type; }
    public String getLicensePlate() { return licensePlate; }
    public abstract boolean canFitInSpot(SpotSize size);
}

class Motorcycle extends Vehicle {
    Motorcycle(String plate) { super(plate, VehicleType.MOTORCYCLE); }
    public boolean canFitInSpot(SpotSize size) { return true; } // fits any spot
}
class Car extends Vehicle {
    Car(String plate) { super(plate, VehicleType.CAR); }
    public boolean canFitInSpot(SpotSize size) { return size == SpotSize.MEDIUM ||
size == SpotSize.LARGE; }
}
class Bus extends Vehicle {
    Bus(String plate) { super(plate, VehicleType.BUS); }
    public boolean canFitInSpot(SpotSize size) { return size == SpotSize.LARGE; }
}

class ParkingSpot {
    private final String spotId;
    private final SpotSize size;
    private Vehicle parkedVehicle;

    ParkingSpot(String spotId, SpotSize size) { this.spotId = spotId; this.size =
size; }

    public boolean isAvailable() { return parkedVehicle == null; }
    public boolean canFit(Vehicle v) { return isAvailable() &&
v.canFitInSpot(size); }
    public void park(Vehicle v) {
        if (!canFit(v)) throw new IllegalStateException("Vehicle cannot fit in this
spot");
        this.parkedVehicle = v;
    }
    public void vacate() { this.parkedVehicle = null; }
    public String getSpotId() { return spotId; }
    public SpotSize getSize() { return size; }
}

class ParkingLevel {
    private final int levelNumber;
    private final List<ParkingSpot> spots;

    ParkingLevel(int levelNumber, List<ParkingSpot> spots) {
        this.levelNumber = levelNumber;
        this.spots = spots;
    }

    // Returns first available spot that fits the vehicle, or null if none (SRP:
level only
    // knows about its own spots, doesn't know about tickets/pricing)
    public Optional<ParkingSpot> findAvailableSpot(Vehicle v) {
        return spots.stream().filter(s -> s.canFit(v)).findFirst();
```

```
    }
    public int getLevelNumber() { return levelNumber; }
}

class Ticket {
    private final String ticketId;
    private final Vehicle vehicle;
    private final ParkingSpot spot;
    private final LocalDateTime entryTime;
    private LocalDateTime exitTime;

    Ticket(String ticketId, Vehicle vehicle, ParkingSpot spot) {
        this.ticketId = ticketId; this.vehicle = vehicle; this.spot = spot;
        this.entryTime = LocalDateTime.now();
    }
    public void markExit() { this.exitTime = LocalDateTime.now(); }
    public Duration getParkedDuration() {
        return Duration.between(entryTime, exitTime != null ? exitTime :
LocalDateTime.now());
    }
    public Vehicle getVehicle() { return vehicle; }
    public ParkingSpot getSpot() { return spot; }
    public String getTicketId() { return ticketId; }
}

// Strategy pattern (§21.5): pricing is pluggable without modifying
ParkingLot/Ticket
interface PricingStrategy {
    double calculateFee(Ticket ticket);
}
class HourlyPricingStrategy implements PricingStrategy {
    private final Map<VehicleType, Double> ratePerHour = Map.of(
        VehicleType.MOTORCYCLE, 1.0, VehicleType.CAR, 2.0, VehicleType.BUS, 5.0
    );
    public double calculateFee(Ticket ticket) {
        long hours = Math.max(1, ticket.getParkedDuration().toMinutes() / 60 + 1);
// round up
        return hours * ratePerHour.get(ticket.getVehicle().getType());
    }
}

// Singleton (§21.1): exactly one ParkingLot instance manages the whole facility
class ParkingLot {
    private static volatile ParkingLot instance;
    private final List<ParkingLevel> levels = new ArrayList<>();
    private final Map<String, Ticket> activeTickets = new HashMap<>();
    private PricingStrategy pricingStrategy;
    private int ticketCounter = 0;

    private ParkingLot() { this.pricingStrategy = new HourlyPricingStrategy(); }

    public static ParkingLot getInstance() {
        if (instance == null) {
            synchronized (ParkingLot.class) {
                if (instance == null) instance = new ParkingLot();
            }
        }
        return instance;
    }
```

```
    public void addLevel(ParkingLevel level) { levels.add(level); }
    public void setPricingStrategy(PricingStrategy strategy) { this.pricingStrategy
= strategy; }

    public Ticket parkVehicle(Vehicle vehicle) {
        for (ParkingLevel level : levels) {
            Optional<ParkingSpot> spot = level.findAvailableSpot(vehicle);
            if (spot.isPresent()) {
                spot.get().park(vehicle);
                Ticket ticket = new Ticket("T" + (++ticketCounter), vehicle,
spot.get());
                activeTickets.put(ticket.getTicketId(), ticket);
                return ticket;
            }
        }
        throw new IllegalStateException("Parking lot full for vehicle type: " +
vehicle.getType());
    }

    public double exitVehicle(String ticketId) {
        Ticket ticket = activeTickets.get(ticketId);
        if (ticket == null) throw new IllegalArgumentException("Invalid ticket");
        ticket.markExit();
        double fee = pricingStrategy.calculateFee(ticket);
        ticket.getSpot().vacate();
        activeTickets.remove(ticketId);
        return fee;
    }
}
```

Extensibility Discussion

- **New vehicle type** (e.g. `Truck`) — add a new `Vehicle` subclass implementing `canFitInSpot`; zero changes to `ParkingSpot`, `ParkingLevel`, or `ParkingLot` (Open/Closed Principle, §20.2).
- **New pricing scheme** (e.g. flat-rate, or weekend surcharge) — implement a new `PricingStrategy`, inject via `setPricingStrategy` — zero changes to `Ticket` or the fee-calculation call site (Strategy pattern paying off directly).
- **Reserved/VIP spots** — extend `ParkingSpot` with a reservation flag, or introduce a `ReservedParkingSpot` subclass overriding `canFit` to also check reservation ownership.

---

45. LLD: Elevator System

Problem

Design the class model for a multi-elevator system in a building, handling internal (inside the elevator) and external (hallway call button) requests, dispatching the best elevator to each request.

Requirements

- Multiple elevators serving multiple floors.
- Handle external requests (a hallway button: "I want to go up/down from floor 5") and internal requests (inside the elevator: "take me to floor 10").
- Dispatch the most appropriate elevator to a new external request (e.g. nearest, moving in the same direction).
- Elevator has a clear state machine (idle, moving up, moving down, door open).

Identify Entities

`Elevator`, `ElevatorController` (dispatcher), `Request` (internal/external), `Direction`, `ElevatorState` (State pattern, §21.10).

Design Patterns Used

- **State** for elevator behavior (idle/moving up/moving down/door open).
- **Strategy** for the dispatch algorithm (nearest elevator vs look-based scheduling) — swappable without touching `ElevatorController`'s core loop.
- **Observer** could notify displays/indicators of elevator position changes (omitted for brevity, direct application of §21.4).

Full Java Code

```
import java.util.*;

enum Direction { UP, DOWN, IDLE }

class ExternalRequest {
    final int floor; final Direction direction;
    ExternalRequest(int floor, Direction direction) { this.floor = floor;
this.direction = direction; }
```

```
}

class Elevator {
    private final int id;
    private int currentFloor = 0;
    private Direction direction = Direction.IDLE;
    private final TreeSet<Integer> upStops = new TreeSet<>();
    private final TreeSet<Integer> downStops = new TreeSet<>
(Collections.reverseOrder());

    Elevator(int id) { this.id = id; }

    // Internal request: "take me to floor X" (from inside the elevator)
    public void requestFloor(int floor) {
        if (floor > currentFloor) upStops.add(floor);
        else if (floor < currentFloor) downStops.add(floor);
        if (direction == Direction.IDLE) {
            direction = floor > currentFloor ? Direction.UP : Direction.DOWN;
        }
    }

    // Called periodically (e.g. by a scheduler tick) to advance the elevator one
step
    public void step() {
        if (direction == Direction.UP) {
            if (!upStops.isEmpty()) {
                currentFloor++;
                if (upStops.contains(currentFloor)) {
                    upStops.remove(currentFloor);
                    openDoors();
                }
            }
            if (upStops.isEmpty()) direction = downStops.isEmpty() ? Direction.IDLE
: Direction.DOWN;
        } else if (direction == Direction.DOWN) {
            if (!downStops.isEmpty()) {
                currentFloor--;
                if (downStops.contains(currentFloor)) {
                    downStops.remove(currentFloor);
                    openDoors();
                }
            }
            if (downStops.isEmpty()) direction = upStops.isEmpty() ? Direction.IDLE
: Direction.UP;
        }
    }

    private void openDoors() { System.out.println("Elevator " + id + " opening
doors at floor " + currentFloor); }

    public int getCurrentFloor() { return currentFloor; }
    public Direction getDirection() { return direction; }
    public int getId() { return id; }

    // Used by the dispatcher to estimate suitability for a new external request
    public int distanceTo(int floor) { return Math.abs(currentFloor - floor); }
    public boolean isIdle() { return direction == Direction.IDLE; }
    public boolean isMovingTowards(int floor, Direction requestedDirection) {
        if (direction != requestedDirection) return false;
```

```
        return direction == Direction.UP ? floor >= currentFloor : floor <=
currentFloor;
    }
}

// Strategy pattern (§21.5): dispatch algorithm is swappable
interface DispatchStrategy {
    Elevator selectElevator(List<Elevator> elevators, ExternalRequest request);
}
class NearestElevatorStrategy implements DispatchStrategy {
    public Elevator selectElevator(List<Elevator> elevators, ExternalRequest
request) {
        // Prefer an idle elevator, or one already moving towards the request in
the same
        // direction; fall back to absolute nearest by distance.
        return elevators.stream()
            .filter(e -> e.isIdle() || e.isMovingTowards(request.floor,
request.direction))
            .min(Comparator.comparingInt(e -> e.distanceTo(request.floor)))
            .orElse(elevators.stream().min(Comparator.comparingInt(e ->
e.distanceTo(request.floor))).get());
    }
}

class ElevatorController {
    private final List<Elevator> elevators;
    private DispatchStrategy dispatchStrategy;

    ElevatorController(int numElevators, DispatchStrategy strategy) {
        elevators = new ArrayList<>();
        for (int i = 0; i < numElevators; i++) elevators.add(new Elevator(i));
        this.dispatchStrategy = strategy;
    }

    // External request: hallway button press
    public void handleExternalRequest(ExternalRequest request) {
        Elevator chosen = dispatchStrategy.selectElevator(elevators, request);
        chosen.requestFloor(request.floor);
    }

    // Internal request: button pressed inside a specific elevator
    public void handleInternalRequest(int elevatorId, int floor) {
        elevators.get(elevatorId).requestFloor(floor);
    }

    public void tick() { elevators.forEach(Elevator::step); }
}
```

Extensibility Discussion

- **New dispatch algorithm** (e.g. a "look" algorithm minimizing total direction changes across all elevators, or a load-balancing strategy considering each

elevator's queue depth) — implement a new `DispatchStrategy`; zero changes to `ElevatorController` or `Elevator`.

- **Elevator door state / safety sensors** — extend with a proper State pattern (§21.10) formalizing `Idle`/`MovingUp`/`MovingDown`/`DoorOpen`/`Maintenance` as explicit classes if the simple `Direction` enum + `TreeSet`-based stop tracking above needs to grow more complex state-dependent behavior (e.g. rejecting new requests while in `Maintenance`).
- **Priority/express requests** (e.g. an elevator reserved for a specific floor during a fire alarm) — add a priority field to `ExternalRequest`/`internalRequest`, and have the dispatch strategy special-case an override elevator — the `DispatchStrategy` interface already isolates this change from the rest of the system.

46. LLD: Splitwise (Expense Sharing)

Problem

Design the class model for an expense-sharing app: users split a bill (equally, by exact amounts, or by percentage), the system tracks who owes whom, and can simplify debts.

Requirements

- Add an expense paid by one user, split among a group of users in different ways (equal, exact amounts, percentage).
- Track running balances between every pair of users.
- Show simplified "who owes whom" (minimize the number of transactions needed to settle all debts).

Identify Entities

`User`, `Expense`, `Split` (+ subtypes: equal/exact/percent — Strategy pattern, §21.5), `ExpenseGroup`, `BalanceSheet`.

Design Patterns Used

- **Strategy** for split calculation (equal vs exact vs percentage).
- **Facade** (§21.8) could wrap the whole "add expense + update balances" flow behind one simple call for client code — shown as `SplitwiseService` below.

Full Java Code

```
import java.util.*;

class User {
    private final String userId;
    private final String name;
    User(String userId, String name) { this.userId = userId; this.name = name; }
    public String getUserId() { return userId; }
    public String getName() { return name; }
}

// Strategy pattern (§21.5): how is the total split among participants?
interface SplitStrategy {
    Map<String, Double> computeSplits(double totalAmount, List<String>
participantIds, Map<String, Double> params);
}
class EqualSplitStrategy implements SplitStrategy {
    public Map<String, Double> computeSplits(double total, List<String>
participants, Map<String, Double> params) {
        double share = total / participants.size();
        Map<String, Double> result = new HashMap<>();
        for (String p : participants) result.put(p, share);
        return result;
    }
}
class ExactSplitStrategy implements SplitStrategy {
    // params: userId -> exact amount they owe; must sum to totalAmount (validated)
    public Map<String, Double> computeSplits(double total, List<String>
participants, Map<String, Double> params) {
        double sum =
params.values().stream().mapToDouble(Double::doubleValue).sum();
        if (Math.abs(sum - total) > 0.01) throw new IllegalArgumentException("Exact
splits must sum to total");
        return new HashMap<>(params);
    }
}
class PercentSplitStrategy implements SplitStrategy {
    // params: userId -> percentage (must sum to 100)
    public Map<String, Double> computeSplits(double total, List<String>
participants, Map<String, Double> params) {
        double sum =
params.values().stream().mapToDouble(Double::doubleValue).sum();
        if (Math.abs(sum - 100.0) > 0.01) throw new
IllegalArgumentException("Percentages must sum to 100");
        Map<String, Double> result = new HashMap<>();
        params.forEach((user, pct) -> result.put(user, total * pct / 100.0));
        return result;
    }
```

```
}

class Expense {
    private final String expenseId;
    private final String paidByUserId;
    private final double amount;
    private final Map<String, Double> splits; // userId -> amount owed by that user

    Expense(String expenseId, String paidBy, double amount, Map<String, Double>
splits) {
        this.expenseId = expenseId; this.paidByUserId = paidBy; this.amount =
amount; this.splits = splits;
    }
    public String getPaidByUserId() { return paidByUserId; }
    public Map<String, Double> getSplits() { return splits; }
}

// Tracks pairwise net balances: balances.get(A).get(B) = amount B owes A (negative
= A owes B)
class BalanceSheet {
    private final Map<String, Map<String, Double>> balances = new HashMap<>();

    public void adjust(String creditor, String debtor, double amount) {
        balances.computeIfAbsent(creditor, k -> new HashMap<>()).merge(debtor,
amount, Double::sum);
        balances.computeIfAbsent(debtor, k -> new HashMap<>()).merge(creditor, -
amount, Double::sum);
    }

    public double getNetBalance(String userA, String userB) {
        return balances.getOrDefault(userA, Map.of()).getOrDefault(userB, 0.0);
    }

    // Simplify: compute each user's total net position, then greedily match
largest
    // creditor with largest debtor repeatedly -- minimizes number of settlement
transactions
    public List<String> simplifyDebts() {
        Map<String, Double> netPosition = new HashMap<>();
        balances.forEach((user, owedByOthers) ->
            netPosition.merge(user,
owedByOthers.values().stream().mapToDouble(Double::doubleValue).sum(),
Double::sum));

        PriorityQueue<Map.Entry<String, Double>> creditors =
            new PriorityQueue<>((a, b) -> Double.compare(b.getValue(),
a.getValue()));
        PriorityQueue<Map.Entry<String, Double>> debtors =
            new PriorityQueue<>((a, b) -> Double.compare(a.getValue(),
b.getValue()));
        netPosition.forEach((user, net) -> {
            if (net > 0.01) creditors.add(Map.entry(user, net));
            else if (net < -0.01) debtors.add(Map.entry(user, net));
        });

        List<String> transactions = new ArrayList<>();
        while (!creditors.isEmpty() && !debtors.isEmpty()) {
            var creditor = creditors.poll();
            var debtor = debtors.poll();
```

```
            double settleAmount = Math.min(creditor.getValue(), -
debtor.getValue());
            transactions.add(debtor.getKey() + " pays " + creditor.getKey() + ": "
+ settleAmount);
            double remainingCredit = creditor.getValue() - settleAmount;
            double remainingDebt = debtor.getValue() + settleAmount;
            if (remainingCredit > 0.01) creditors.add(Map.entry(creditor.getKey(),
remainingCredit));
            if (remainingDebt < -0.01) debtors.add(Map.entry(debtor.getKey(),
remainingDebt));
        }
        return transactions;
    }
}

// Facade (§21.8): simple entry point hiding split-calculation + balance-update
coordination
class SplitwiseService {
    private final BalanceSheet balanceSheet = new BalanceSheet();
    private final Map<String, SplitStrategy> strategies = Map.of(
        "EQUAL", new EqualSplitStrategy(), "EXACT", new ExactSplitStrategy(),
"PERCENT", new PercentSplitStrategy()
    );
    private int expenseCounter = 0;

    public Expense addExpense(String paidBy, double amount, List<String>
participants,
                              String splitType, Map<String, Double> params) {
        SplitStrategy strategy = strategies.get(splitType);
        Map<String, Double> splits = strategy.computeSplits(amount, participants,
params);
        Expense expense = new Expense("E" + (++expenseCounter), paidBy, amount,
splits);
        splits.forEach((user, owedAmount) -> {
            if (!user.equals(paidBy)) balanceSheet.adjust(paidBy, user,
owedAmount);
        });
        return expense;
    }

    public List<String> getSimplifiedDebts() { return balanceSheet.simplifyDebts();
}
    public double getBalanceBetween(String userA, String userB) { return
balanceSheet.getNetBalance(userA, userB); }
}
```

Extensibility Discussion

- **New split type** (e.g. "by shares," where each participant has a weight/share count) — implement a new `SplitStrategy`; zero changes to `Expense`, `BalanceSheet`, or `SplitwiseService`'s core flow.

- **Multi-currency support** — extend `Expense`/`BalanceSheet` to track currency alongside amount, and introduce a currency-conversion step before balance adjustment — isolated to the `adjust`/`computeSplits` boundary.
- **Group-scoped balances** (separate balance sheets per group vs one global balance sheet per user pair) — swap `BalanceSheet` to be keyed by `(groupId, userA, userB)` instead of just `(userA, userB)` — the simplification algorithm (`simplifyDebts`) is unaffected, just scoped per group when invoked.

---

47. LLD: LRU Cache

Problem

Design an in-memory cache with a fixed capacity that evicts the **Least Recently Used** entry when full — the classic LLD/coding-interview hybrid problem (directly implements the LRU eviction policy concept introduced in §9.4).

Requirements

- `get(key)` — O(1), returns value and marks the entry as most-recently-used.
- `put(key, value)` — O(1), inserts/updates; evicts the LRU entry if at capacity.
- Both operations must be **O(1)** — this constraint is what drives the specific data structure choice below (a naive approach using just a `LinkedHashMap`'s built-in access-order mode would also work and is worth mentioning as a one-line alternative, but implementing the underlying mechanism by hand is the actual point of this exercise).

Identify Entities

`Node` (doubly linked list node), `LRUCache` (combines a `HashMap` for O(1) lookup with a doubly linked list for O(1) reordering/eviction).

Design Patterns Used

No classic GoF pattern is central here — this is fundamentally a **data structure composition** exercise (hash map + doubly linked list), which is itself a very common

LLD interview archetype distinct from the OOP-modeling-heavy case studies elsewhere in this part.

Full Java Code

```
import java.util.HashMap;
import java.util.Map;

class LRUCache<K, V> {
    private class Node {
        K key; V value; Node prev, next;
        Node(K key, V value) { this.key = key; this.value = value; }
    }

    private final int capacity;
    private final Map<K, Node> cache = new HashMap<>();
    private final Node head; // dummy head (most-recently-used side)
    private final Node tail; // dummy tail (least-recently-used side)

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Node(null, null);
        tail = new Node(null, null);
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        Node node = cache.get(key);
        if (node == null) return null;
        moveToFront(node); // just accessed -- now most-recently-used
        return node.value;
    }

    public void put(K key, V value) {
        Node existing = cache.get(key);
        if (existing != null) {
            existing.value = value;
            moveToFront(existing);
            return;
        }
        if (cache.size() >= capacity) {
            Node lru = tail.prev; // node just before dummy tail = least-recently-
used
            removeNode(lru);
            cache.remove(lru.key);
        }
        Node newNode = new Node(key, value);
        cache.put(key, newNode);
        addToFront(newNode);
    }

    private void moveToFront(Node node) { removeNode(node); addToFront(node); }
```

```
    private void removeNode(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addToFront(Node node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }
}
```

Extensibility Discussion

- **LFU instead of LRU** — swap the reordering rule: instead of moving to front on every access, maintain a frequency count per key and evict the lowest-frequency entry (typically implemented with a frequency-bucketed structure — a map of frequency → doubly-linked-list of keys at that frequency — to keep operations O(1); a genuinely different, harder data structure, worth mentioning as the natural "next problem" if asked to extend this one).
- **TTL-based expiry alongside LRU** (§9.4's combined approach) — store an expiry timestamp per node, and check/lazily evict expired entries on access, or run a periodic background sweep — an additive change that doesn't disturb the core doubly-linked-list mechanism.
- **Thread-safety for concurrent access** — wrap critical sections in a lock (e.g. `synchronized` methods, or a `ReentrantReadWriteLock` if reads should be allowed to proceed concurrently more often than writes) — a natural, expected follow-up question (see below).

---

48. LLD: Rate Limiter (Code)

Problem

Implement the **Token Bucket** rate-limiting algorithm (introduced conceptually in §12.4 and §24) as a standalone, reusable class.

Requirements

- Each client has its own independent bucket (capacity `B`, refill rate `R` tokens/sec).
- `allowRequest(clientId)` returns `true`/`false` and must be efficient and thread-safe (real rate limiters are called concurrently from many request-handling threads).
- Refilling should be **lazy** (computed on each check based on elapsed time), not via a separate background thread per client (wouldn't scale to millions of clients).

Identify Entities

`TokenBucket` (per-client state), `RateLimiter` (manages buckets for all clients).

Design Patterns Used

- Could be viewed as an application of **Strategy** if multiple algorithms (token bucket, sliding window, §12.4) are meant to be swappable behind a common `RateLimiter` interface — shown below with the interface in place, one concrete implementation provided in full.

Full Java Code

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.locks.ReentrantLock;

interface RateLimiter {
    boolean allowRequest(String clientId);
}

class TokenBucketRateLimiter implements RateLimiter {
    private static class Bucket {
        double tokens;
        long lastRefillTimestampMillis;
        final ReentrantLock lock = new ReentrantLock();
        Bucket(double tokens, long timestamp) { this.tokens = tokens;
this.lastRefillTimestampMillis = timestamp; }
    }

    private final double capacity;
    private final double refillRatePerSecond;
    private final ConcurrentHashMap<String, Bucket> buckets = new
ConcurrentHashMap<>();

    public TokenBucketRateLimiter(double capacity, double refillRatePerSecond) {
        this.capacity = capacity;
        this.refillRatePerSecond = refillRatePerSecond;
    }
```

```
    @Override
    public boolean allowRequest(String clientId) {
        Bucket bucket = buckets.computeIfAbsent(clientId, id -> new
Bucket(capacity, System.currentTimeMillis()));
        bucket.lock.lock();
        try {
            refill(bucket);
            if (bucket.tokens >= 1.0) {
                bucket.tokens -= 1.0;
                return true;
            }
            return false;
        } finally {
            bucket.lock.unlock();
        }
    }

    // Lazy refill: compute how many tokens should have accumulated since the last
check,
    // based on elapsed wall-clock time -- avoids needing a background thread per
client.
    private void refill(Bucket bucket) {
        long now = System.currentTimeMillis();
        double elapsedSeconds = (now - bucket.lastRefillTimestampMillis) / 1000.0;
        double tokensToAdd = elapsedSeconds * refillRatePerSecond;
        if (tokensToAdd > 0) {
            bucket.tokens = Math.min(capacity, bucket.tokens + tokensToAdd);
            bucket.lastRefillTimestampMillis = now;
        }
    }
}
```

Extensibility Discussion

- **Sliding Window Counter algorithm** (§12.4) as an alternative — implement a second class, `SlidingWindowRateLimiter implements RateLimiter`, with the same interface — any calling code depends only on the `RateLimiter` interface (Dependency Inversion, §20.2), so swapping algorithms requires zero changes elsewhere.
- **Distributed rate limiting** (§24's deep dive on making this work across a fleet) — this in-process `ConcurrentHashMap`-based implementation is correct for a single JVM instance only; a distributed version would replace the `buckets` map with calls to a shared Redis instance (using a Lua script for the atomic "refill + check + decrement" sequence, avoiding the read-then-write race condition that a naive multi-step Redis interaction would introduce).
- **Per-tier limits** (different capacity/refill rate for free vs paid users) — pass the appropriate `capacity`/`refillRatePerSecond` per client at bucket-creation time

(e.g. looked up from a user-tier service) instead of using one global configuration for all clients — a small, additive change to the `computeIfAbsent` call site.

49. LLD: Vending Machine

Problem

Design the class model for a vending machine: select a product, insert money, dispense the product and change, handling out-of-stock and insufficient-payment cases correctly.

Requirements

- Support multiple products, each with a price and stock count.
- Accept coins/notes, track inserted amount, allow cancellation (refund).
- Dispense product + correct change if payment is sufficient.
- Clear state machine: idle → selecting → collecting payment → dispensing → returning change (State pattern, §21.10 — the canonical use case for this pattern).

Identify Entities

`Product`, `Inventory`, `VendingMachine` (context), `VendingMachineState` (+ concrete states).

Design Patterns Used

- **State** (§21.10) — the entire machine's behavior is state-dependent (e.g. "select product" is valid in `IdleState` but not in `DispensingState`); this is the textbook example for the State pattern.

Full Java Code

```
import java.util.*;
```

```
class Product {
    final String code; final String name; final double price;
    Product(String code, String name, double price) { this.code = code; this.name =
name; this.price = price; }
}

class Inventory {
    private final Map<String, Product> products = new HashMap<>();
    private final Map<String, Integer> stock = new HashMap<>();

    public void addProduct(Product p, int quantity) { products.put(p.code, p);
stock.put(p.code, quantity); }
    public Product getProduct(String code) { return products.get(code); }
    public boolean isInStock(String code) { return stock.getOrDefault(code, 0) > 0;
}
    public void decrementStock(String code) { stock.merge(code, -1, Integer::sum);
}
}

// State pattern (§21.10): each state defines its own valid transitions/behavior
interface VendingMachineState {
    void selectProduct(VendingMachine machine, String code);
    void insertMoney(VendingMachine machine, double amount);
    void dispense(VendingMachine machine);
    void cancel(VendingMachine machine);
}

class IdleState implements VendingMachineState {
    public void selectProduct(VendingMachine machine, String code) {
        if (!machine.getInventory().isInStock(code)) {
            System.out.println("Out of stock: " + code);
            return;
        }
        machine.setSelectedProduct(machine.getInventory().getProduct(code));
        machine.setState(new CollectingPaymentState());
    }
    public void insertMoney(VendingMachine machine, double amount) {
System.out.println("Select a product first"); }
    public void dispense(VendingMachine machine) { System.out.println("Select a
product first"); }
    public void cancel(VendingMachine machine) { System.out.println("Nothing to
cancel"); }
}

class CollectingPaymentState implements VendingMachineState {
    public void selectProduct(VendingMachine machine, String code) {
System.out.println("Already selecting a product"); }
    public void insertMoney(VendingMachine machine, double amount) {
        machine.addPayment(amount);
        if (machine.getInsertedAmount() >= machine.getSelectedProduct().price) {
            machine.setState(new DispensingState());
            machine.dispense(); // auto-trigger dispense once fully paid
        }
    }
    public void dispense(VendingMachine machine) { System.out.println("Insufficient
payment"); }
    public void cancel(VendingMachine machine) {
        System.out.println("Refunding: " + machine.getInsertedAmount());
        machine.reset();
```

```
        machine.setState(new IdleState());
    }
}

class DispensingState implements VendingMachineState {
    public void selectProduct(VendingMachine machine, String code) {
System.out.println("Currently dispensing"); }
    public void insertMoney(VendingMachine machine, double amount) {
System.out.println("Currently dispensing"); }
    public void dispense(VendingMachine machine) {
        Product product = machine.getSelectedProduct();
        double change = machine.getInsertedAmount() - product.price;
        machine.getInventory().decrementStock(product.code);
        System.out.println("Dispensing " + product.name + ", change: " + change);
        machine.reset();
        machine.setState(new IdleState());
    }
    public void cancel(VendingMachine machine) { System.out.println("Cannot cancel
while dispensing"); }
}

class VendingMachine {
    private VendingMachineState state = new IdleState();
    private final Inventory inventory;
    private Product selectedProduct;
    private double insertedAmount = 0;

    VendingMachine(Inventory inventory) { this.inventory = inventory; }

    public void selectProduct(String code) { state.selectProduct(this, code); }
    public void insertMoney(double amount) { state.insertMoney(this, amount); }
    public void dispense() { state.dispense(this); }
    public void cancel() { state.cancel(this); }

    void setState(VendingMachineState state) { this.state = state; }
    void setSelectedProduct(Product p) { this.selectedProduct = p; }
    Product getSelectedProduct() { return selectedProduct; }
    void addPayment(double amount) { this.insertedAmount += amount; }
    double getInsertedAmount() { return insertedAmount; }
    Inventory getInventory() { return inventory; }
    void reset() { this.selectedProduct = null; this.insertedAmount = 0; }
}
```

Extensibility Discussion

- **New state** (e.g. `MaintenanceState` disabling purchases for restocking) — implement `VendingMachineState`; the `VendingMachine` context class needs zero changes since it already just delegates every action to `state.*()`.
- **Multiple payment methods** (coins, card) — introduce a `PaymentMethod` interface/ Strategy alongside the state machine, so `CollectingPaymentState`

delegates the actual payment-processing mechanics to a pluggable strategy rather than assuming cash only.

- **Refund exact change in specific denominations** — extend the dispensing logic with a `ChangeDispenser` component (greedy coin-denomination algorithm) — isolated from the state machine's transition logic itself.

---

50. LLD: Movie Ticket Booking

Problem

Design the class model for booking movie seats for a specific showtime (the LLD counterpart to the HLD case study §31 — here the focus is the class/object model rather than distributed-systems concerns like handling massive concurrent contention).

Requirements

- A `Movie` has multiple `Show`s (showtime + theater + screen).
- Each `Show` has a `SeatMap` with individual `Seat`s (different types: regular/premium).
- Book one or more specific seats for a show; prevent double-booking of the same seat.
- Support cancellation.

Identify Entities

`Movie`, `Show`, `Screen`, `Seat` (+ `SeatType`), `Booking`, `BookingManager`.

Design Patterns Used

- **Factory Method** implicitly for creating different `Seat` types with different pricing.
- **Facade** (`BookingManager`) coordinating seat-locking + booking creation behind a simple interface, echoing §21.8 and the checkout facade example.

Full Java Code

```
import java.time.LocalDateTime;
import java.util.*;
import java.util.concurrent.locks.ReentrantLock;

enum SeatType { REGULAR, PREMIUM }
enum SeatStatus { AVAILABLE, BOOKED }

class Seat {
    final String seatId; final SeatType type; final double price;
    private SeatStatus status = SeatStatus.AVAILABLE;
    private final ReentrantLock lock = new ReentrantLock();

    Seat(String seatId, SeatType type, double price) { this.seatId = seatId;
this.type = type; this.price = price; }

    // Atomic check-and-set, mirroring the HLD case study's DB-level atomic update
(§31) --
    // here enforced via a per-seat lock instead of a database transaction.
    public boolean tryBook() {
        lock.lock();
        try {
            if (status == SeatStatus.AVAILABLE) { status = SeatStatus.BOOKED;
return true; }
            return false;
        } finally { lock.unlock(); }
    }
    public void release() {
        lock.lock();
        try { status = SeatStatus.AVAILABLE; } finally { lock.unlock(); }
    }
    public boolean isAvailable() { return status == SeatStatus.AVAILABLE; }
}

class Show {
    final String showId; final String movieTitle; final LocalDateTime startTime;
    private final Map<String, Seat> seats = new LinkedHashMap<>();

    Show(String showId, String movieTitle, LocalDateTime startTime, List<Seat>
seatList) {
        this.showId = showId; this.movieTitle = movieTitle; this.startTime =
startTime;
        for (Seat s : seatList) seats.put(s.seatId, s);
    }
    public Seat getSeat(String seatId) { return seats.get(seatId); }
    public List<Seat> getAvailableSeats() {
        return seats.values().stream().filter(Seat::isAvailable).toList();
    }
}

class Booking {
    final String bookingId; final String showId; final List<String> seatIds; final
double totalAmount;
    Booking(String bookingId, String showId, List<String> seatIds, double
totalAmount) {
        this.bookingId = bookingId; this.showId = showId; this.seatIds = seatIds;
this.totalAmount = totalAmount;
    }
```

```
}

// Facade (§21.8): coordinates seat-locking across multiple seats atomically (all-or-nothing,
// directly addressing the group-booking follow-up question from the HLD case study §31)
class BookingManager {
    private final Map<String, Show> shows = new HashMap<>();
    private int bookingCounter = 0;

    public void addShow(Show show) { shows.put(show.showId, show); }

    public Optional<Booking> bookSeats(String showId, List<String> seatIds) {
        Show show = shows.get(showId);
        List<Seat> lockedSeats = new ArrayList<>();
        // Try to lock all requested seats; if any fails, roll back all successfully-locked ones
        for (String seatId : seatIds) {
            Seat seat = show.getSeat(seatId);
            if (seat.tryBook()) {
                lockedSeats.add(seat);
            } else {
                lockedSeats.forEach(Seat::release); // rollback -- all-or-nothing group booking
                return Optional.empty();
            }
        }
        double total = lockedSeats.stream().mapToDouble(s -> s.price).sum();
        Booking booking = new Booking("B" + (++bookingCounter), showId, seatIds, total);
        return Optional.of(booking);
    }

    public void cancelBooking(String showId, List<String> seatIds) {
        Show show = shows.get(showId);
        seatIds.forEach(id -> show.getSeat(id).release());
    }
}
```

Extensibility Discussion

- **Dynamic pricing** (e.g. premium seats cost more at peak showtimes) — introduce a `PricingStrategy` (§21.5) computing `price` at booking time rather than a fixed field on `Seat` — an additive change to `bookSeats`, not a restructuring.
- **Seat holds with expiry** (matching the HLD case study's hold/confirm two-phase flow, §31) — extend `Seat`'s state to a 3-state model (`AVAILABLE`/`HELD`/`BOOKED`) with an expiry timestamp, and add a scheduled cleanup task — directly mirrors the state-machine extensibility pattern already seen in §49.
- **Multiple screens/theaters per movie** — already naturally supported by having `Show` reference a specific screen/theater; no structural change needed, just

populating more `Show` instances.

---

51. LLD: Snake and Ladder

Problem

Design the class model for a Snake and Ladder board game supporting multiple players, a configurable board with snakes and ladders, and turn-based dice-roll gameplay.

Requirements

- Configurable board size, with snakes (move backward) and ladders (move forward) at specific cell positions.
- Multiple players take turns rolling a die and moving.
- First player to reach (or exceed, per house rules — configurable) the final cell wins.

Identify Entities

`Board`, `Cell` (+ optional snake/ladder), `Dice`, `Player`, `Game` (orchestrator).

Design Patterns Used

- **Strategy** for dice behavior (a standard 6-sided die vs a "loaded"/testable die) — makes the game engine trivially unit-testable by injecting a deterministic dice implementation.

Full Java Code

```
import java.util.*;

class Board {
    private final int size;
    private final Map<Integer, Integer> jumps = new HashMap<>(); // cell ->
```

```
destination cell (snake or ladder)

    Board(int size) { this.size = size; }

    public void addSnake(int head, int tail) {
        if (head <= tail) throw new IllegalArgumentException("Snake head must be
above its tail");
        jumps.put(head, tail);
    }
    public void addLadder(int bottom, int top) {
        if (bottom >= top) throw new IllegalArgumentException("Ladder bottom must
be below its top");
        jumps.put(bottom, top);
    }

    // Resolve final position after landing on a cell (applies snake/ladder if
present)
    public int resolvePosition(int position) { return jumps.getOrDefault(position,
position); }
    public int getSize() { return size; }
}

// Strategy pattern (§21.5): dice roll behavior is pluggable (e.g. for
deterministic testing)
interface Dice {
    int roll();
}
class StandardDice implements Dice {
    private final Random random = new Random();
    public int roll() { return random.nextInt(6) + 1; }
}

class Player {
    final String name;
    int position = 0;
    Player(String name) { this.name = name; }
}

class Game {
    private final Board board;
    private final Dice dice;
    private final Queue<Player> turnOrder;

    Game(Board board, Dice dice, List<Player> players) {
        this.board = board; this.dice = dice;
        this.turnOrder = new LinkedList<>(players);
    }

    // Plays until one player wins; returns the winning player.
    public Player play() {
        while (true) {
            Player current = turnOrder.poll();
            int roll = dice.roll();
            int newPosition = current.position + roll;
            if (newPosition > board.getSize()) {
                // Overshoot: stay in place (common house rule) instead of winning
                turnOrder.add(current);
                continue;
            }
```

```
            newPosition = board.resolvePosition(newPosition);
            current.position = newPosition;
            System.out.println(current.name + " rolled " + roll + ", now at " +
newPosition);
            if (newPosition == board.getSize()) return current;
            turnOrder.add(current);
        }
    }
}
```

Extensibility Discussion

- **Multiple dice / different board sizes / different overshoot rules** — `Dice` is already a swappable strategy (e.g. a `TwoDiceSum implements Dice`); overshoot behavior could similarly be extracted into an `OvershootRule` strategy if multiple house-rule variants need to be supported simultaneously.
- **Power-ups/special cells** (e.g. "skip next turn," "roll again") beyond simple snakes/ ladders — generalize `Board`'s `jumps` map into a more general `Map<Integer, CellEffect>` where `CellEffect` is an interface with an `apply(Player, Game)` method — the `Game` loop's cell-resolution step calls this uniformly instead of a hardcoded position lookup.
- **Concurrent/networked multiplayer** — the `Game` class as written is synchronous/ single-threaded (fine for a local/turn-based console game); a networked version would need to separate "whose turn is it" state management from the actual gameplay loop and likely introduce a state machine per game session (reusing the State pattern, §21.10, similar to §49's vending machine).

52. LLD: Library Management System

Problem

Design the class model for a library system: catalog of books (with multiple copies), members who can borrow/return/reserve books, and fine calculation for overdue returns.

Requirements

- A `Book` (title/author/ISBN) can have multiple physical `BookCopy` instances.

- Members can check out an available copy, return it, and reserve a book that's currently fully checked out (notified when a copy becomes available).
- Overdue returns incur a fine (pluggable calculation).

Identify Entities

`Book`, `BookCopy`, `Member`, `Loan` (a checkout record), `Reservation`, `Library` (orchestrator), `FinePolicy` (Strategy).

Design Patterns Used

- **Strategy** for fine calculation (flat rate vs escalating per day overdue).
- **Observer** (§21.4) for reservation notifications — when a copy is returned, notify the next member in the reservation queue.

Full Java Code

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;

enum CopyStatus { AVAILABLE, CHECKED_OUT }

class Book {
    final String isbn; final String title; final String author;
    Book(String isbn, String title, String author) { this.isbn = isbn; this.title =
title; this.author = author; }
}

class BookCopy {
    final String copyId; final Book book;
    CopyStatus status = CopyStatus.AVAILABLE;
    BookCopy(String copyId, Book book) { this.copyId = copyId; this.book = book; }
}

// Observer pattern (§21.4): members reserve a book and are notified when a copy
frees up
interface ReservationObserver {
    void onCopyAvailable(BookCopy copy);
}
class Member implements ReservationObserver {
    final String memberId; final String name;
    Member(String memberId, String name) { this.memberId = memberId; this.name =
name; }
    public void onCopyAvailable(BookCopy copy) {
        System.out.println("Notifying " + name + ": a copy of '" + copy.book.title
```

```
+ "' is now available");
    }
}

class Loan {
    final String loanId; final BookCopy copy; final Member member;
    final LocalDate checkoutDate; final LocalDate dueDate;
    LocalDate returnDate;
    Loan(String loanId, BookCopy copy, Member member, LocalDate checkoutDate,
LocalDate dueDate) {
        this.loanId = loanId; this.copy = copy; this.member = member;
        this.checkoutDate = checkoutDate; this.dueDate = dueDate;
    }
}

// Strategy pattern (§21.5): fine calculation is pluggable
interface FinePolicy {
    double calculateFine(Loan loan, LocalDate returnDate);
}
class DailyFlatFinePolicy implements FinePolicy {
    private final double perDayRate;
    DailyFlatFinePolicy(double perDayRate) { this.perDayRate = perDayRate; }
    public double calculateFine(Loan loan, LocalDate returnDate) {
        long overdueDays = Math.max(0, ChronoUnit.DAYS.between(loan.dueDate,
returnDate));
        return overdueDays * perDayRate;
    }
}

class Library {
    private final Map<String, List<BookCopy>> catalog = new HashMap<>(); // isbn ->
copies
    private final Map<String, Queue<ReservationObserver>> reservationQueues = new
HashMap<>();
    private final Map<String, Loan> activeLoans = new HashMap<>(); // copyId ->
Loan
    private final FinePolicy finePolicy;
    private int loanCounter = 0;

    Library(FinePolicy finePolicy) { this.finePolicy = finePolicy; }

    public void addCopy(BookCopy copy) {
        catalog.computeIfAbsent(copy.book.isbn, k -> new ArrayList<>()).add(copy);
    }

    public Optional<Loan> checkout(String isbn, Member member) {
        List<BookCopy> copies = catalog.getOrDefault(isbn, List.of());
        for (BookCopy copy : copies) {
            if (copy.status == CopyStatus.AVAILABLE) {
                copy.status = CopyStatus.CHECKED_OUT;
                Loan loan = new Loan("L" + (++loanCounter), copy, member,
LocalDate.now(), LocalDate.now().plusDays(14));
                activeLoans.put(copy.copyId, loan);
                return Optional.of(loan);
            }
        }
        return Optional.empty(); // caller can then call reserve()
    }
```

```
    public void reserve(String isbn, Member member) {
        reservationQueues.computeIfAbsent(isbn, k -> new LinkedList<>
()).add(member);
    }

    public double returnCopy(String copyId) {
        Loan loan = activeLoans.remove(copyId);
        loan.returnDate = LocalDate.now();
        loan.copy.status = CopyStatus.AVAILABLE;
        double fine = finePolicy.calculateFine(loan, loan.returnDate);

        // Notify next reservation in queue, if any (Observer pattern in action)
        Queue<ReservationObserver> queue =
reservationQueues.get(loan.copy.book.isbn);
        if (queue != null && !queue.isEmpty()) {
            queue.poll().onCopyAvailable(loan.copy);
        }
        return fine;
    }
}
```

Extensibility Discussion

- **New fine policy** (e.g. escalating rate after 7 days, or a maximum cap) — implement a new `FinePolicy`; zero changes to `Library`/`Loan`.
- **E-books / digital copies with no physical inventory limit** — introduce a `DigitalBookCopy` alongside `BookCopy` (both implementing a common `Copy` interface) with different availability semantics (effectively unlimited concurrent "checkouts") — Library logic would branch on copy type or use polymorphism to avoid `if/else` chains.
- **Multiple libraries/branches sharing a catalog** — `Library` becomes `Branch`, and a new `LibrarySystem` aggregates multiple branches, routing checkout requests to whichever branch has an available copy nearest the member — an additive structural change layered on top of the existing `Library` class rather than a rewrite.

---

53. LLD: Tic-Tac-Toe & Chess

Problem

Design class models for two classic board games — Tic-Tac-Toe (simple, for warm-up) and Chess (complex, for a deeper LLD discussion) — illustrating how the same core

"board game" skeleton scales from trivial to highly complex, and where polymorphism earns its keep.

Tic-Tac-Toe — Requirements & Code

Simple 3x3 grid, two players (X/O) alternate placing marks, win by 3-in-a-row (row/column/ diagonal), or a draw if the board fills without a winner.

```
import java.util.*;

enum Mark { X, O, EMPTY }

class TicTacToeBoard {
    private final Mark[][] grid;
    private final int size;

    TicTacToeBoard(int size) {
        this.size = size;
        grid = new Mark[size][size];
        for (Mark[] row : grid) Arrays.fill(row, Mark.EMPTY);
    }

    public boolean placeMark(int row, int col, Mark mark) {
        if (grid[row][col] != Mark.EMPTY) return false;
        grid[row][col] = mark;
        return true;
    }

    public Mark checkWinner() {
        for (int i = 0; i < size; i++) {
            if (allSame(grid[i])) return grid[i][0];                // row
            Mark[] col = new Mark[size];
            for (int j = 0; j < size; j++) col[j] = grid[j][i];
            if (allSame(col)) return col[0];                         // column
        }
        Mark[] diag1 = new Mark[size], diag2 = new Mark[size];
        for (int i = 0; i < size; i++) { diag1[i] = grid[i][i]; diag2[i] = grid[i]
[size - 1 - i]; }
        if (allSame(diag1)) return diag1[0];
        if (allSame(diag2)) return diag2[0];
        return Mark.EMPTY; // no winner yet
    }

    private boolean allSame(Mark[] line) {
        return line[0] != Mark.EMPTY && Arrays.stream(line).allMatch(m -> m ==
line[0]);
    }

    public boolean isFull() {
        return Arrays.stream(grid).flatMap(Arrays::stream).noneMatch(m -> m ==
Mark.EMPTY);
```

```
    }
}
```

Chess — Requirements

Chess needs a fundamentally richer model: each piece type has unique movement rules, moves must be validated against check/checkmate, and special rules (castling, en passant, promotion) apply. This is a deliberately common "LLD escalation" interview pattern — start simple (Tic-Tac-Toe), then the interviewer says "now make it Chess" to see how your design adapts to much greater complexity.

Identify Entities (Chess)

`Piece` (abstract, + King/Queen/Rook/Bishop/Knight/Pawn subtypes), `Board`, `Position`, `Move`, `Player`, `Game`.

Design Patterns Used (Chess)

- **Polymorphism/Template pattern** — each `Piece` subtype implements its own `getValidMoves()`, avoiding a giant `switch` on piece type scattered through the codebase — the single most important design decision, directly demonstrating the Open/Closed Principle (§20.2): adding a new custom piece type (e.g. a variant-chess "Archbishop") requires zero changes to `Board` or `Game`.

Full Java Code (Chess — Core Skeleton)

```
import java.util.*;

enum Color { WHITE, BLACK }

class Position {
    final int row, col;
    Position(int row, int col) { this.row = row; this.col = col; }
    boolean isValid() { return row >= 0 && row < 8 && col >= 0 && col < 8; }
    @Override public boolean equals(Object o) {
        if (!(o instanceof Position p)) return false;
        return row == p.row && col == p.col;
    }
    @Override public int hashCode() { return Objects.hash(row, col); }
}
```

```
abstract class Piece {
    final Color color;
    Position position;
    Piece(Color color, Position position) { this.color = color; this.position =
position; }
    // Each subtype defines its own movement rules -- the polymorphic core of the
design
    public abstract List<Position> getValidMoves(Board board);
}

class Rook extends Piece {
    Rook(Color color, Position position) { super(color, position); }
    public List<Position> getValidMoves(Board board) {
        List<Position> moves = new ArrayList<>();
        int[][] directions = {{1,0},{-1,0},{0,1},{0,-1}};
        for (int[] dir : directions) {
            int r = position.row, c = position.col;
            while (true) {
                r += dir[0]; c += dir[1];
                Position candidate = new Position(r, c);
                if (!candidate.isValid()) break;
                Piece occupant = board.getPieceAt(candidate);
                if (occupant == null) { moves.add(candidate); continue; }
                if (occupant.color != this.color) moves.add(candidate); // capture
                break; // blocked either way (own piece, or just captured)
            }
        }
        return moves;
    }
}

class Knight extends Piece {
    Knight(Color color, Position position) { super(color, position); }
    public List<Position> getValidMoves(Board board) {
        int[][] jumps = {{2,1},{2,-1},{-2,1},{-2,-1},{1,2},{1,-2},{-1,2},{-1,-2}};
        List<Position> moves = new ArrayList<>();
        for (int[] jump : jumps) {
            Position candidate = new Position(position.row + jump[0], position.col
+ jump[1]);
            if (candidate.isValid()) {
                Piece occupant = board.getPieceAt(candidate);
                if (occupant == null || occupant.color != this.color)
moves.add(candidate);
            }
        }
        return moves;
    }
}
// Bishop, Queen, King, Pawn follow the same pattern (each implements getValidMoves
per its
// own rules) -- omitted here for brevity since the pattern is now established.

class Board {
    private final Map<Position, Piece> pieces = new HashMap<>();
    public Piece getPieceAt(Position pos) { return pieces.get(pos); }
    public void placePiece(Piece piece) { pieces.put(piece.position, piece); }
    public void movePiece(Position from, Position to) {
        Piece piece = pieces.remove(from);
```

```
        piece.position = to;
        pieces.put(to, piece); // overwrites/captures any piece that was at `to`
    }
}

class Game {
    private final Board board;
    private Color currentTurn = Color.WHITE;

    Game(Board board) { this.board = board; }

    public boolean makeMove(Position from, Position to) {
        Piece piece = board.getPieceAt(from);
        if (piece == null || piece.color != currentTurn) return false;
        if (!piece.getValidMoves(board).contains(to)) return false;
        board.movePiece(from, to);
        // A full implementation would check: does this move leave the mover's own
king in
        // check? (illegal if so) -- and after the move, is the opponent now in
        // check/checkmate/stalemate? Omitted here to keep the skeleton focused on
the
        // core polymorphic movement-validation design.
        currentTurn = (currentTurn == Color.WHITE) ? Color.BLACK : Color.WHITE;
        return true;
    }
}
```

Extensibility Discussion

- **New piece variants** (chess960/fairy chess pieces) — implement a new `Piece` subclass with its own `getValidMoves` — zero changes to `Board` or `Game`, directly demonstrating why polymorphism was chosen over a type-tag + switch-statement approach from the start.
- **Check/checkmate detection** — layer on top of `getValidMoves` by simulating each candidate move and checking whether the mover's own king would then be attacked by any opposing piece's valid moves — an additive validation step in `makeMove`, not a restructuring of the piece-movement model.
- **Move history / undo / PGN export** — introduce a `Move` record type and an append-only `List<Move>` on `Game` — supports undo (replay all moves except the last) and serialization, without touching `Piece`/`Board`.
- **Contrast with Tic-Tac-Toe:** Tic-Tac-Toe's board is simple enough that a flat 2D array
    - direct win-checking logic is entirely appropriate — reaching for heavy OOP/polymorphism there would be over-engineering. Chess's combinatorial rule complexity is exactly what justifies the richer `Piece` polymorphism — a

good, explicit point to make if an interviewer asks "why didn't you use the same simple approach for both?"

54. LLD: Logging Framework

Problem

Design a simple, extensible logging framework (the class-design essence of libraries like Log4j/SLF4J): log messages at different severity levels, route them to multiple destinations (console, file, network), with configurable formatting and level filtering.

Requirements

- Support standard log levels (DEBUG < INFO < WARN < ERROR) with level-based filtering (e.g. a logger configured for `WARN` ignores `DEBUG`/`INFO` calls entirely, cheaply).
- Support multiple simultaneous output destinations ("appenders") — e.g. log to both console and a file at once.
- Configurable message formatting (timestamp, level, message — customizable pattern).
- Extensible: adding a new output destination (e.g. send logs to a network service) should not require modifying the core `Logger` class.

Identify Entities

`LogLevel`, `LogMessage`, `Appender` (+ concrete: Console/File), `Formatter`, `Logger`.

Design Patterns Used

- **Strategy** for both `Formatter` (how a message is rendered to text) and the level-filtering logic.
- **Observer**-adjacent: a `Logger` can have multiple `Appender`s attached, each independently receiving every log event that passes the level filter (conceptually identical to the Observer pattern §21.4, applied to log events instead of generic state changes).

- **Singleton** commonly applied to a global `LoggerFactory`/registry (omitted for brevity — same pattern as §21.1/§44's `ParkingLot`).

Full Java Code

```
import java.time.LocalDateTime;
import java.util.*;

enum LogLevel {
    DEBUG(0), INFO(1), WARN(2), ERROR(3);
    final int severity;
    LogLevel(int severity) { this.severity = severity; }
}

class LogMessage {
    final LogLevel level; final String message; final LocalDateTime timestamp;
    LogMessage(LogLevel level, String message) {
        this.level = level; this.message = message; this.timestamp =
LocalDateTime.now();
    }
}

// Strategy pattern (§21.5): formatting is pluggable
interface Formatter {
    String format(LogMessage msg);
}
class SimpleFormatter implements Formatter {
    public String format(LogMessage msg) {
        return "[" + msg.timestamp + "] [" + msg.level + "] " + msg.message;
    }
}

// Appenders: each represents a different output destination -- new appenders can
be added
// without modifying Logger at all (Open/Closed Principle, §20.2)
interface Appender {
    void append(String formattedMessage);
}
class ConsoleAppender implements Appender {
    public void append(String formattedMessage) {
System.out.println(formattedMessage); }
}
class FileAppender implements Appender {
    private final List<String> fileLines = new ArrayList<>(); // stand-in for real
file I/O
    public void append(String formattedMessage) { fileLines.add(formattedMessage);
}
    public List<String> getLines() { return fileLines; }
}

class Logger {
    private final LogLevel minLevel;
    private final Formatter formatter;
    private final List<Appender> appenders = new ArrayList<>();
```

```
    Logger(LogLevel minLevel, Formatter formatter) { this.minLevel = minLevel;
this.formatter = formatter; }

    public void addAppender(Appender appender) { appenders.add(appender); }

    private void log(LogLevel level, String message) {
        if (level.severity < minLevel.severity) return; // cheap early filtering,
no formatting
                                                        // or appender work
wasted on filtered-out levels
        LogMessage logMessage = new LogMessage(level, message);
        String formatted = formatter.format(logMessage);
        for (Appender appender : appenders) appender.append(formatted);
    }

    public void debug(String message) { log(LogLevel.DEBUG, message); }
    public void info(String message) { log(LogLevel.INFO, message); }
    public void warn(String message) { log(LogLevel.WARN, message); }
    public void error(String message) { log(LogLevel.ERROR, message); }
}
```

Extensibility Discussion

- **New destination** (e.g. `NetworkAppender` shipping logs to a central aggregation service — directly connecting back to the HLD case study §43) — implement `Appender`; zero changes to `Logger`.
- **Per-appender log levels** (e.g. console shows `INFO+`, but the file captures `DEBUG+`) — extend `Appender` with its own level threshold, checked in `Logger.log()` per-appender instead of one global `minLevel` — a small, additive change to the loop.
- **Structured (JSON) logging** — implement a `JsonFormatter implements Formatter` — directly supports the HLD case study's recommendation (§43) that structured logging at the source is preferable to fragile downstream text parsing; this LLD's `Formatter` abstraction is precisely what would make that switch a one-line configuration change in a real application, rather than a rewrite of every log call site.
- **Async/non-blocking appenders** (avoid blocking the calling thread on slow I/O, e.g. network appenders) — wrap a slow `Appender` in a decorator (§21.6) that enqueues messages onto an internal queue processed by a background thread — an additive wrapper, not a change to `Logger` or the wrapped appender's own logic.

---

55. LLD: Food Delivery (Swiggy/Zomato)

Problem

Design the class model for a food delivery app's core ordering flow: browsing a restaurant's menu, placing an order, and tracking order status through preparation and delivery (the LLD counterpart to concepts touched on in the checkout §32 and ride-sharing §30 HLD case studies — here the focus is the order/state class model, not distributed- systems scaling concerns).

Requirements

- Restaurants have a menu of `MenuItem`s (with availability toggling).
- A customer places an `Order` containing multiple menu items with quantities.
- Order progresses through a well-defined lifecycle: `PLACED` → `CONFIRMED` → `PREPARING` → `OUT_FOR_DELIVERY` → `DELIVERED` (or `CANCELLED` at certain points) — a clean State-pattern application (§21.10), directly reusing the same technique as the vending machine (§49) and the order-lifecycle example introduced in §21.10 itself.
- Assign a delivery partner once the order is ready.

Identify Entities

`Restaurant`, `MenuItem`, `OrderItem`, `Order`, `OrderState` (+ concrete states), `DeliveryPartner`, `OrderManager`.

Design Patterns Used

- **State** (§21.10) for order lifecycle — the primary pattern, directly reused from its original introduction.
- **Observer** (§21.4) for notifying the customer on each state transition (echoing the notification-system HLD case study, §26, at the object-model level).

Full Java Code

```
import java.util.*;

class MenuItem {
    final String itemId; final String name; final double price;
    boolean available = true;
    MenuItem(String itemId, String name, double price) { this.itemId = itemId;
this.name = name; this.price = price; }
}

class Restaurant {
    final String restaurantId; final String name;
    private final Map<String, MenuItem> menu = new HashMap<>();
    Restaurant(String restaurantId, String name) { this.restaurantId =
restaurantId; this.name = name; }
    public void addMenuItem(MenuItem item) { menu.put(item.itemId, item); }
    public MenuItem getMenuItem(String itemId) { return menu.get(itemId); }
}

class OrderItem {
    final MenuItem item; final int quantity;
    OrderItem(MenuItem item, int quantity) { this.item = item; this.quantity =
quantity; }
    double getSubtotal() { return item.price * quantity; }
}

// Observer pattern (§21.4): customer is notified on every order state transition
interface OrderObserver {
    void onStatusChanged(Order order, String newStatus);
}
class CustomerNotifier implements OrderObserver {
    public void onStatusChanged(Order order, String newStatus) {
        System.out.println("Order " + order.orderId + " is now: " + newStatus);
    }
}

// State pattern (§21.10): order lifecycle, directly reusing the pattern from
§21.10/§49
interface OrderState {
    void confirm(Order order);
    void startPreparing(Order order);
    void dispatchForDelivery(Order order, DeliveryPartner partner);
    void markDelivered(Order order);
    void cancel(Order order);
}
class PlacedState implements OrderState {
    public void confirm(Order order) { order.setState(new ConfirmedState());
order.notifyObservers("CONFIRMED"); }
    public void startPreparing(Order order) { System.out.println("Must confirm
before preparing"); }
    public void dispatchForDelivery(Order order, DeliveryPartner p) {
System.out.println("Not ready for delivery"); }
    public void markDelivered(Order order) { System.out.println("Not out for
delivery yet"); }
    public void cancel(Order order) { order.setState(new CancelledState());
order.notifyObservers("CANCELLED"); }
}
class ConfirmedState implements OrderState {
```

```
    public void confirm(Order order) { System.out.println("Already confirmed"); }
    public void startPreparing(Order order) { order.setState(new PreparingState());
order.notifyObservers("PREPARING"); }
    public void dispatchForDelivery(Order order, DeliveryPartner p) {
System.out.println("Still preparing"); }
    public void markDelivered(Order order) { System.out.println("Not out for
delivery yet"); }
    public void cancel(Order order) { order.setState(new CancelledState());
order.notifyObservers("CANCELLED"); }
}
class PreparingState implements OrderState {
    public void confirm(Order order) { System.out.println("Already confirmed"); }
    public void startPreparing(Order order) { System.out.println("Already
preparing"); }
    public void dispatchForDelivery(Order order, DeliveryPartner p) {
        order.setDeliveryPartner(p);
        order.setState(new OutForDeliveryState());
        order.notifyObservers("OUT_FOR_DELIVERY");
    }
    public void markDelivered(Order order) { System.out.println("Not out for
delivery yet"); }
    public void cancel(Order order) { System.out.println("Cannot cancel once
preparing has started"); }
}
class OutForDeliveryState implements OrderState {
    public void confirm(Order order) { System.out.println("Already confirmed"); }
    public void startPreparing(Order order) { System.out.println("Already
preparing"); }
    public void dispatchForDelivery(Order order, DeliveryPartner p) {
System.out.println("Already out for delivery"); }
    public void markDelivered(Order order) { order.setState(new DeliveredState());
order.notifyObservers("DELIVERED"); }
    public void cancel(Order order) { System.out.println("Cannot cancel once out
for delivery"); }
}
class DeliveredState implements OrderState {
    public void confirm(Order order) {}
    public void startPreparing(Order order) {}
    public void dispatchForDelivery(Order order, DeliveryPartner p) {}
    public void markDelivered(Order order) { System.out.println("Already
delivered"); }
    public void cancel(Order order) { System.out.println("Cannot cancel a delivered
order"); }
}
class CancelledState implements OrderState {
    public void confirm(Order order) { System.out.println("Order is cancelled"); }
    public void startPreparing(Order order) { System.out.println("Order is
cancelled"); }
    public void dispatchForDelivery(Order order, DeliveryPartner p) {
System.out.println("Order is cancelled"); }
    public void markDelivered(Order order) { System.out.println("Order is
cancelled"); }
    public void cancel(Order order) { System.out.println("Already cancelled"); }
}

class DeliveryPartner {
    final String partnerId; final String name;
    DeliveryPartner(String partnerId, String name) { this.partnerId = partnerId;
this.name = name; }
```

```
}

class Order {
    final String orderId; final Restaurant restaurant; final List<OrderItem> items;
    private OrderState state = new PlacedState();
    private DeliveryPartner deliveryPartner;
    private final List<OrderObserver> observers = new ArrayList<>();

    Order(String orderId, Restaurant restaurant, List<OrderItem> items) {
        this.orderId = orderId; this.restaurant = restaurant; this.items = items;
    }

    public void addObserver(OrderObserver observer) { observers.add(observer); }
    void notifyObservers(String status) { observers.forEach(o ->
o.onStatusChanged(this, status)); }
    void setState(OrderState state) { this.state = state; }
    void setDeliveryPartner(DeliveryPartner partner) { this.deliveryPartner =
partner; }

    public void confirm() { state.confirm(this); }
    public void startPreparing() { state.startPreparing(this); }
    public void dispatchForDelivery(DeliveryPartner partner) {
state.dispatchForDelivery(this, partner); }
    public void markDelivered() { state.markDelivered(this); }
    public void cancel() { state.cancel(this); }

    public double getTotal() { return
items.stream().mapToDouble(OrderItem::getSubtotal).sum(); }
}

class OrderManager {
    private int orderCounter = 0;
    public Order placeOrder(Restaurant restaurant, List<OrderItem> items) {
        Order order = new Order("O" + (++orderCounter), restaurant, items);
        order.addObserver(new CustomerNotifier());
        return order;
    }
}
```

Extensibility Discussion

- **New lifecycle state** (e.g. a `PickupState` for takeaway orders that skips delivery entirely) — implement a new `OrderState`; the `Order` context class is unchanged since it already delegates every transition through `state.*()`, exactly mirroring §49's discussion.
- **Multiple observers** (customer app, restaurant dashboard, delivery-partner app all watching the same order) — `Order.addObserver` already supports any number of observers; no structural change needed, just registering more `OrderObserver` implementations.

- **Order modification before confirmation** — add an `addItem`/`removeItem` method on `Order`, guarded by checking the current state allows modification (e.g. only in `PlacedState`) — naturally expressed by delegating the check into the state classes themselves, keeping the "what's allowed when" logic centralized in one place per state rather than scattered `if` checks in `Order`.
- **Refunds on cancellation** — introduce a `RefundService` collaborator invoked from `CancelledState`'s transition logic (or via an additional `OrderObserver` reacting specifically to the `"CANCELLED"` notification) — kept as a separate concern rather than baked directly into the state classes, following the Single Responsibility Principle (§20.2).